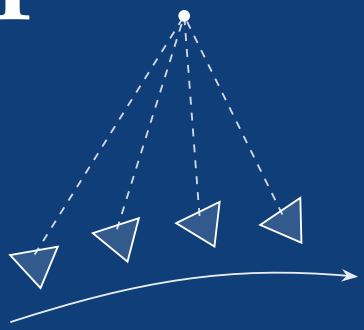
From Zero to MSCKF

Lecture Notes on Visual-Inertial Odometry

Lie theory · error-state Kalman filtering · multi-view geometry · the Multi-State Constraint Kalman Filter



These notes develop monocular visual-inertial odometry from first principles. They start with the linear algebra, probability and rotation geometry that state estimation is built on, develop calculus on the rotation group SO(3), derive the error-state Kalman filter for an inertial navigation system, build the geometric computer-vision front end, and assemble everything into the Multi-State Constraint Kalman Filter (MSCKF) evaluated on the EuRoC MAV benchmark.

Every convention matches the frozen specification of the `vio` project (`AGENTS.md`): Hamilton quaternions, body-to-world attitude q_{wb}, and *local (right) perturbations* in every Jacobian.

Edition. This edition *omits* the worked solutions of the project derivation exercises, so that F_c, G_c, H and the cloning Jacobian J are yours to derive (Guardrail 4 of `AGENTS.md`). Each *Derivation exercise* box gives the method, hints and a numerical test that tells you when you are right.

Course notes for the `vio` project

October 2, 2026

Preface

What these notes are

Visual-inertial odometry (VIO) estimates the motion of a rigid body, such as a drone, a phone, a headset or a car, by fusing an inertial measurement unit (IMU) with a camera. It is the navigation core of most modern augmented-reality devices and of many autonomous aerial vehicles. VIO is also one of the best ways to learn state estimation properly, because it forces you to deal with almost every hard topic in the field at once: nonlinear dynamics, estimation on curved spaces, sensor noise modeling, multi-rate fusion, unobservable directions, outliers, numerical conditioning, and real-time computation.

The notes are written in the style of a graduate course. Each chapter states its learning objectives, develops the theory with definitions, propositions and proofs, illustrates it with worked examples and figures, flags the pitfalls that cost practitioners weeks of debugging, and closes with key takeaways and exercises. All formulas and conventions match the `vio` project specification in `AGENTS.md`. Every identity, Jacobian and growth law stated here has been checked numerically, and the figures are generated by `scripts/make_figures.py`, so you can reproduce them and change their parameters.

Who should read them

The assumed background is

- multivariable calculus (Taylor series, chain rule, partial derivatives);
- introductory linear algebra (matrices, determinants, eigenvalues), which Chapter 2 revisits in the depth state estimation needs;
- basic probability (mean, variance, Gaussian distribution), revisited in Chapter 3;
- for the implementation notes, working knowledge of modern C++ and Eigen.

No prior exposure to Lie groups, Kalman filtering or multi-view geometry is assumed.

How the notes are organized

Part I, Mathematical Foundations. The vocabulary of the whole subject: linear algebra and the factorizations that make filters numerically sound, probability and Gaussian algebra, rotations and quaternions, and Lie theory, the calculus of the rotation group.

Part II, Estimation Theory. The Kalman filter from Bayes' rule, its nonlinear extension, and the error-state formulation that makes estimation on a manifold clean.

Part III, Inertial Navigation. IMU physics and noise, strapdown propagation, and the linearized error dynamics that drive covariance growth.

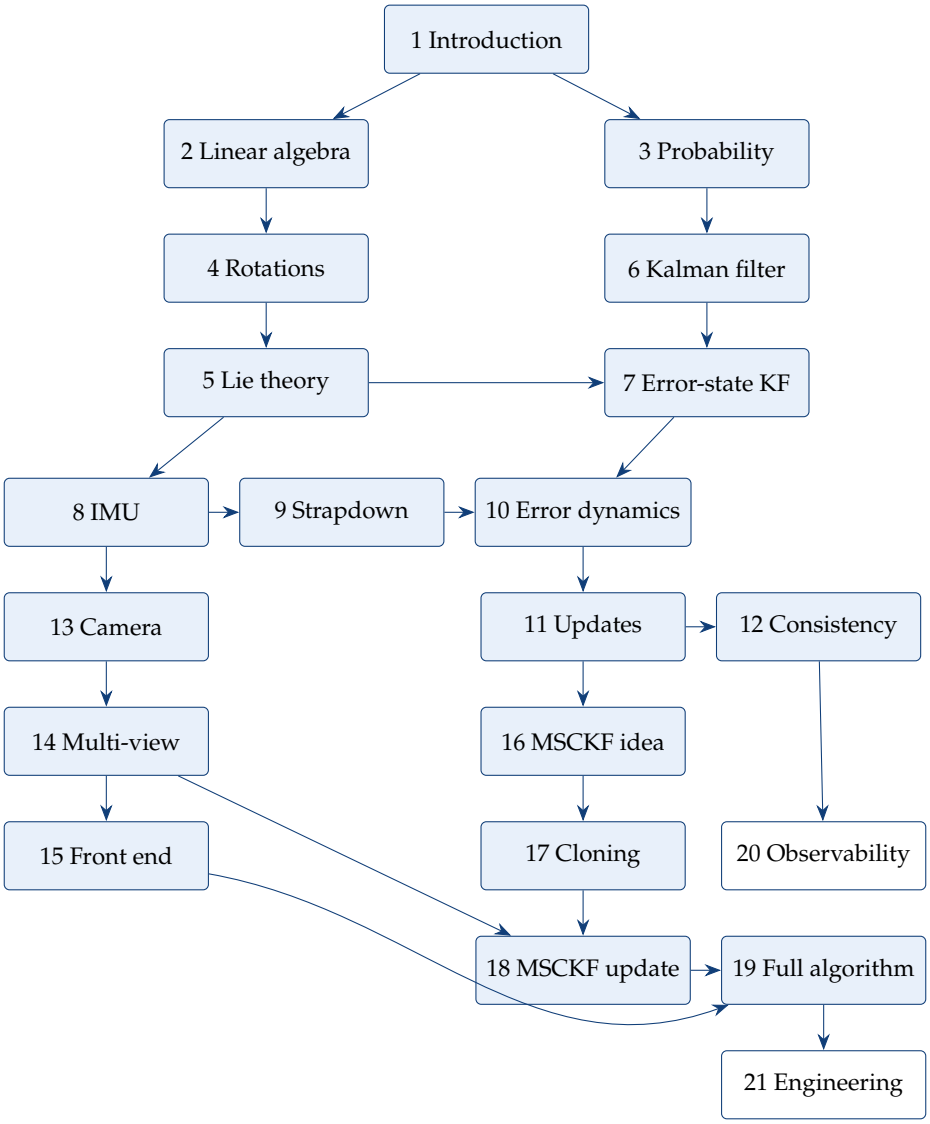


Figure 1: Chapter dependencies. Arrows point from prerequisites to the chapters that use them. Chapters 8–12 can be read before Part V, but the MSCKF chapters need both branches.

Part IV, The Error-State Filter in Practice. Measurement updates, gating, injection and reset, and the statistics that tell you whether the filter is lying to you.

Part V, Geometric Computer Vision. Camera models, epipolar geometry, triangulation, corner detection and optical-flow tracking.

Part VI, The MSCKF. Stochastic cloning, the visual measurement model, left-nullspace projection, and the complete algorithm.

Part VII, Beyond the Baseline. Observability and consistency, and engineering a deterministic real-time implementation in C++20.

Reading the boxes

Five kinds of callout recur throughout:

Key idea: the one-sentence version

The idea to remember even if you forget the derivation.

Frozen convention: project specification

A rule from `AGENTS.md` that is frozen for the whole project. These are never changed silently. A change is a logged design decision.

Pitfall: what goes wrong in practice

A mistake that is easy to make, hard to see, and expensive to debug.

Implementation note: from math to code

How an equation becomes deterministic, allocation-free C++20 with Eigen.

Project link: Milestone N

Where the material meets a project milestone and its exit criteria.

A sixth box, the **Derivation exercise**, marks the Jacobians that the project asks you to derive yourself: F_c, G_c, the measurement Jacobians H, and the cloning Jacobian J. The surrounding text teaches the method on closely related examples, and the box gives hints plus a finite-difference test that tells you when your answer is right. The worked solutions are collected in an appendix that exists only in the *solutions* edition of these notes (this edition omits it).

Milestones and chapters

Milestone	Topic	Chapters
M1	Build harness, sanitizers, allocation guard	21
M2	Lie group math on SO(3)	4, 5
M3	EuRoC dataset streamer	8, Appendix D
M4	Nominal IMU kinematics	9
M5	Error covariance propagation	10
M6	Pose update, injection and reset	7, 11
M7	Telemetry and consistency	12
M8	Feature detection and optical flow	13, 15
M9	Epipolar geometry and outlier rejection	14, 15
M10a	Stochastic cloning and sliding window	16, 17
M10b	MSCKF visual update	18, 19, 20

Sources and acknowledgments

These notes stand on a body of excellent literature. The treatment of quaternions and the error-state filter follows the conventions of Solà [44]; Lie theory follows Solà, Deray and Atchuthan [45] and Barfoot [3]; the MSCKF is due to Mourikis and Roumeliotis [37] and

was refined by Li and Mourikis [31]; much practical wisdom comes from OpenVINS [17] and MSCKF-VIO [48]. The annotated bibliography at the end points to the best places to go deeper on each topic.

Contents

Preface i

I Mathematical Foundations 1

1 The Visual-Inertial Odometry Problem 2

1.1 What odometry is . . . 2
1.2 An analogy: the blindfolded walk . . . 2
1.3 The sensors and their weaknesses . . . 3
1.4 Why fusion is hard . . . 5
1.5 A map of the field . . . 6
1.6 Why the MSCKF . . . 6
1.7 The estimator at a glance . . . 7
1.8 The EuRoC benchmark . . . 8
Exercises . . . 9

2 Linear Algebra for State Estimation 10

2.1 Vectors, coordinates and the cross product . . . 10
2.2 The four fundamental subspaces . . . 11
2.3 Symmetric and positive-definite matrices . . . 12
2.4 Matrix factorizations . . . 13
2.5 Solving linear systems and least squares . . . 15
2.6 Block matrices and the Schur complement . . . 15
2.7 The matrix exponential . . . 16
2.8 Derivatives and Jacobians . . . 16
2.9 Floating-point arithmetic . . . 17
Exercises . . . 18

3 Probability and Gaussian Algebra **20**

3.1 Random vectors, mean and covariance . . . 20
3.2 The multivariate Gaussian . . . 21
3.3 Marginals and conditionals . . . 21
3.4 Bayesian estimation . . . 22
3.5 Nonlinear transformations and linearization . . . 23
3.6 The chi-squared distribution . . . 24
3.7 Continuous-time noise . . . 24
3.8 Consistency . . . 25
Exercises . . . 26

4 Frames, Rotations and Quaternions **27**

4.1 Frames and notation . . . 27
4.2 Rotation matrices and SO(3) . . . 28
4.3 Axis-angle and Rodrigues' formula . . . 29
4.4 Euler angles and gimbal lock . . . 30
4.5 Quaternions . . . 30
4.6 Rigid-body transformations . . . 33
4.7 Rotational kinematics . . . 33
Exercises . . . 35

5 Lie Theory for Rotations **36**

5.1 Why estimation needs Lie theory . . . 36
5.2 Groups, manifolds and Lie groups . . . 37
5.3 The Lie algebra and tangent spaces . . . 37
5.4 The exponential map . . . 38
5.5 The logarithm map . . . 40
5.6 The adjoint: moving a perturbation across a rotation . . . 41
5.7 Composition and the Baker–Campbell–Hausdorff formula . . . 42
5.8 Perturbations, $\boxplus$ and $\boxminus$ . . . 42
5.9 Jacobians on manifolds . . . 43
5.10 Uncertainty on SO(3) . . . 45
5.11 SE(3) and the choice of state manifold . . . 46
5.12 Verifying manifold Jacobians numerically . . . 46
Exercises . . . 47

II Estimation Theory **48**

6 The Kalman Filter **49**

6.1 Linear-Gaussian state-space models . . . 49
6.2 The Bayes filter . . . 49
6.3 Derivation . . . 50
6.4 Properties . . . 51
6.5 The Joseph form . . . 51
6.6 Implementing the update . . . 52
6.7 From continuous to discrete time . . . 53
6.8 A worked example . . . 54
Exercises . . . 55

7 The Extended and Error-State Kalman Filters **56**

7.1 The extended Kalman filter . . . 56
7.2 Filtering on a manifold . . . 57
7.3 The ESKF cycle . . . 57
7.4 Linearized error dynamics: the recipe . . . 58
7.5 Measurement updates on the manifold . . . 59
7.6 Injection and reset . . . 59
7.7 The complete error-state filter . . . 60
Exercises . . . 61

III Inertial Navigation **62**

8 Inertial Sensors and Their Errors **63**

8.1 How MEMS inertial sensors work . . . 63
8.2 Specific force: what an accelerometer measures . . . 63
8.3 The measurement model . . . 64
8.4 Error sources . . . 65
8.5 Noise parameters and their units . . . 65
8.6 The Allan variance . . . 66
8.7 Tuning in practice . . . 67
8.8 The EuRoC IMU stream . . . 67
Exercises . . . 68

9 Strapdown Propagation of the Nominal State **69**

9.1 Continuous-time kinematics . . . 69
9.2 Exact solution for constant inputs . . . 70
9.3 Numerical integration schemes . . . 70
9.4 Timing . . . 72
9.5 Initialization . . . 72
9.6 Sidebar: IMU preintegration . . . 73
Exercises . . . 74

10 Error-State Dynamics and Covariance Propagation **75**

10.1 True and nominal systems . . . 75
10.2 Deriving the error dynamics . . . 76
10.3 Discretization . . . 77
10.4 Propagating the full covariance . . . 78
10.5 How uncertainty grows . . . 78
10.6 Validating covariance propagation . . . 80
Exercises . . . 80

IV The Error-State Filter in Practice **82**

11 Measurement Updates, Gating and the Pose Sensor **83**

11.1 The generic update . . . 83
11.2 Worked example: position with a lever arm . . . 83
11.3 Worked example: a direction sensor . . . 84
11.4 The pose sensor of Milestone 6 . . . 84
11.5 Gating . . . 85
11.6 Testing the update . . . 86
Exercises . . . 87

12 Consistency and Evaluation **88**

12.1 What consistency means . . . 88
12.2 Normalized estimation error squared . . . 88
12.3 Normalized innovation squared . . . 89
12.4 Diagnosing inconsistency . . . 90
12.5 Trajectory accuracy . . . 91
Exercises . . . 92

V Geometric Computer Vision **93**

13 Camera Models **94**

13.1 The pinhole model . . . 94
13.2 Lens distortion . . . 94
13.3 The measurement model . . . 96
13.4 The projection Jacobian . . . 96
13.5 Extrinsics and time offset . . . 97
Exercises . . . 98

14 Multi-View Geometry **99**

14.1 Epipolar geometry . . . 99
14.2 Estimating relative motion . . . 100
14.3 RANSAC . . . 101
14.4 Degeneracies . . . 101
14.5 Triangulation . . . 102
Exercises . . . 104

15 The Visual Front End **106**

15.1 What makes a good feature . . . 106
15.2 Lucas–Kanade optical flow . . . 107
15.3 Making tracking robust . . . 108
15.4 The feature life cycle . . . 109
15.5 Engineering the front end . . . 110
Exercises . . . 110

VI The Multi-State Constraint Kalman Filter **112**

16 From EKF-SLAM to the MSCKF **113**

16.1 EKF-SLAM . . . 113
16.2 The multi-state constraint . . . 113
16.3 Why past poses must be in the state . . . 114
16.4 The MSCKF state . . . 115
16.5 Relation to sliding-window optimization . . . 115
16.6 Variants . . . 115
Exercises . . . 116

17 Stochastic Cloning and the Sliding Window **117**

17.1 A scalar example . . . 117
17.2 State augmentation . . . 118
17.3 The project's clone . . . 118
17.4 Clones in propagation and updates . . . 119
17.5 Marginalizing the oldest clone . . . 119
17.6 Window policy and size . . . 120
Exercises . . . 120

18 The MSCKF Visual Update **122**

18.1 A feature track . . . 122
18.2 Residuals and Jacobians . . . 122
18.3 The problem with the feature . . . 123
18.4 Left-nullspace projection . . . 124
18.5 Gating, stacking and compression . . . 125
18.6 The update . . . 126
Exercises . . . 127

19 The Complete Algorithm **128**

19.1 The event loop . . . 128
19.2 Initialization and the first seconds . . . 128
19.3 Computational budget . . . 129
19.4 Software architecture . . . 130
19.5 Debugging guide . . . 130
19.6 What to expect on EuRoC . . . 131
Exercises . . . 132

VII Beyond the Baseline **133**

20 Observability and Consistency of VIO **134**

20.1 Observability . . . 134
20.2 The four unobservable directions of VIO . . . 135
20.3 Why the standard EKF becomes overconfident . . . 135
20.4 Remedies . . . 136
20.5 Degenerate motions . . . 136
Exercises . . . 137

21 Engineering a Real-Time Estimator in C++20 **138**

21.1 Determinism and the memory tiers . . . 138

21.2 Eigen pitfalls . . . 139

21.3 Time and events . . . 140

21.4 Numerical robustness checklist . . . 140

21.5 Testing . . . 141

21.6 Debugging a diverging filter . . . 141

21.7 Going further . . . 142

Exercises . . . 142

Appendices **143**

A Notation and Conventions **144**

B Formula Sheet **146**

C Statistical and Numerical Tables **149**

D The EuRoC MAV Dataset **151**

Further Reading **153**

References **154**

Part I

Mathematical Foundations

Chapter 1

The Visual-Inertial Odometry Problem

In this chapter

- State precisely what visual-inertial odometry estimates and what it cannot estimate.
- Understand why an IMU alone drifts, and quantify how fast.
- Understand why a monocular camera alone cannot recover metric scale.
- Place the MSCKF among the main families of VIO algorithms.
- See the complete estimator pipeline that the rest of the notes build piece by piece.

1.1 What odometry is

Odometry is the incremental estimation of a body's motion from its own sensors, without a prior map and without absolute positioning such as GNSS. The word comes from the wheel odometers of cars, which count wheel rotations. *Visual-inertial odometry* uses an IMU and one or more cameras rigidly attached to the body.

At every time t we want the *state*

$$x(t) = \big(p_{wb}(t),\ v_{wb}(t),\ q_{wb}(t),\ b_a(t),\ b_g(t) \big), \tag{1.1}$$

namely the position and velocity of the body in a world frame, its orientation, and the slowly varying biases of the accelerometer and gyroscope. We also want an honest measure of how uncertain that estimate is. Chapter 4 makes the frames and symbols precise. For now, w is a fixed, gravity-aligned world frame and b is the body frame of the IMU.

Odometry differs from *SLAM* (simultaneous localization and mapping) in one important way. A SLAM system builds a persistent map and recognizes places it has seen before (*loop closure*), which lets it cancel accumulated drift. An odometry system has no such memory: its error grows without bound, slowly if it is good and quickly if it is bad. The goal of VIO is to make that growth as slow as possible, typically well below 1% of the distance traveled, while running in real time on modest hardware.

1.2 An analogy: the blindfolded walk

Imagine standing blindfolded in an unfamiliar gymnasium and being asked to walk a 20-meter square and return to the start. You would rely on your inner ear to sense turning and on the feeling in your legs to judge distance. Integrating these internal sensations into a running estimate of your position is *dead reckoning*.

For the first few steps the estimate is good. But your sensors are imperfect: a 90-degree turn comes out as 87 degrees, and your strides are a little shorter than you think. Each new step is added on top of a slightly wrong estimate, so the small errors accumulate. After a few minutes you might believe you are back at the start while actually standing several meters away. This accumulation is called *drift*.

Now suppose you may lift the blindfold for a tenth of a second every few seconds. The room is dark except for a few glowing stickers on the walls. You do not know where the stickers are, but by watching how they shift between glimpses you can infer how you moved, and correct the walking estimate. That is visual-inertial odometry: the inner ear is the IMU, the glimpses are camera images, and the stickers are visual features.

Two details of the analogy carry over exactly. First, the glimpses are *relative*: they tell you how you moved between glimpses, not where you are in some absolute sense, so drift is slowed but never eliminated. Second, you need both senses. Without the glimpses, drift is fast. Without the inner ear, a single glimpse cannot tell whether a sticker is small and near or large and far.

1.3 The sensors and their weaknesses

1.3.1 The inertial measurement unit

An IMU contains a three-axis gyroscope and a three-axis accelerometer. Neither measures position.

- The **gyroscope** measures the angular velocity ω of the body, in rad/s, expressed in the body frame.
- The **accelerometer** measures *specific force*: the non-gravitational force per unit mass acting on the sensor. At rest on a table it reads +9.81 m/s^2 *upward*, because the table pushes up on it, and in free fall it reads zero. Chapter 8 explains this carefully.

IMUs are fast (typically 100–1000 Hz), cheap, self-contained, insensitive to lighting and texture, and their measurements are *metric*: an acceleration in m/s^2 carries an absolute scale. But to get position you must integrate angular rate once to get attitude, then rotate the specific force into the world frame, add gravity, and integrate twice. Any error in the sensor or in the attitude is integrated as well.

1.3.2 How fast inertial errors grow

Three error sources dominate short-term inertial drift. The derivations are one-liners, and the resulting growth laws (Figure 1.1) are worth memorizing.

1. **Accelerometer bias b_a.** A constant acceleration error integrated twice gives a position error $\frac{1}{2} b_a t^2$.
2. **Initial tilt error $\delta\theta_0$.** If the estimated attitude is tilted by a small angle $\delta\theta_0$, the 9.81 m/s^2 gravity compensation is applied along a slightly wrong axis. A component $g\,\delta\theta_0$ of gravity leaks into the horizontal acceleration, and the position error grows as $\frac{1}{2} g\,\delta\theta_0\,t^2$.

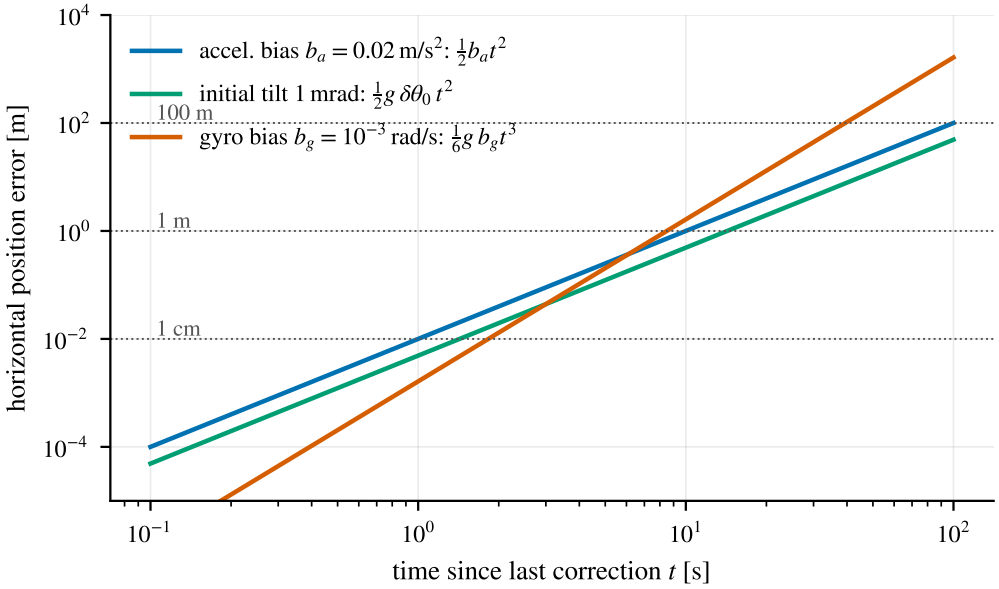


Figure 1.1: Deterministic dead-reckoning drift from three error sources of a consumer-grade IMU. A gyro bias of 10^{-3} rad/s ($\approx 0.06\,^\circ$/s) produces more than a meter of error after ten seconds and hundreds of meters after one minute. The cubic law is why attitude accuracy dominates inertial navigation.

3. **Gyroscope bias b_g.** Now the tilt itself grows linearly, $\delta\theta(t) = b_g t$, so the leaked acceleration is $g b_g t$, and integrating twice gives $\frac{1}{6} g\, b_g t^3$.

With $b_g = 10^{-3}$ rad/s, a modest value for a MEMS gyroscope, the gyro term alone gives 1.6 m after 10 s and 353 m after 60 s. Random sensor noise adds error that grows in a similar way. Chapter 10 derives that the position *standard deviation* due to accelerometer white noise grows as $t^{3/2}$, due to gyroscope white noise as $t^{5/2}$, and due to the gyroscope bias random walk as $t^{7/2}$.

Key idea: inertial navigation is an attitude problem

Because gravity is large (about 9.81 m/s^2) compared with typical accelerometer errors, a tilt of one milliradian costs as much as an accelerometer bias of about 10 mm/s^2. Position accuracy is therefore limited mostly by attitude accuracy. Any aiding sensor that constrains roll and pitch, as a camera does, pays off enormously.

1.3.3 The camera

A camera measures the *directions* (bearings) of light rays arriving from the scene. A point p_f in the world appears in the image at the projection of its camera-frame coordinates $p_c = (X_c, Y_c, Z_c)$:

$$z = \begin{bmatrix} X_c / Z_c \\ Y_c / Z_c \end{bmatrix}. \tag{1.2}$$

Because images are measurements of fixed scene geometry, they do not drift the way integrated inertial data does. But a single camera has a fundamental blind spot.

	IMU	Monocular camera
Rate	100–1000 Hz	10–60 Hz
Measures	angular rate, specific force	bearings to scene points
Scale	metric	unknown (Proposition 1.1)
Drift	fast (t^2 to t^3)	slow, relative to the scene
Gravity direction	observable (at rest or on average)	not observable
Fails when	always drifts; saturates on impacts	blur, darkness, no texture, pure rotation

Table 1.1: The IMU and the camera fail in complementary ways.

Proposition 1.1 (Monocular scale ambiguity). *Let a camera with poses (R_i, p_i), $i = 1, \dots, M$, observe points $p_{f,j}$. For any $s > 0$, the scaled configuration with poses $(R_i, s\,p_i)$ and points $s\,p_{f,j}$ produces exactly the same images.*

Proof. In camera i, the scaled point has coordinates $R_i^\mathsf{T}(s\,p_{f,j} - s\,p_i) = s\,R_i^\mathsf{T}(p_{f,j} - p_i)$. The projection divides the first two coordinates by the third, so the factor s cancels. ∎

A camera alone therefore recovers the trajectory and the scene only up to an unknown global scale. A photograph of a toy car from 10 cm can be identical to a photograph of a real car from 4 m. Cameras also fail under motion blur, in darkness, against texture-less surfaces such as white walls, and in scenes dominated by moving objects.

1.3.4 Why the combination works

Table 1.1 summarizes the complementarity. The camera bounds the inertial drift by constraining the motion between images. The IMU supplies metric scale, the direction of gravity (and hence absolute roll and pitch), high-rate motion between frames, and robustness during brief visual outages. Scale becomes observable because the accelerometer measures acceleration in $\mathrm{m/s^2}$: scaling a trajectory by s scales its acceleration by s, which the accelerometer would notice. The exception is unaccelerated motion. At constant velocity, the scale is unobservable again, which is one of several *degenerate motions* studied in Chapter 20.

Even fused, four directions remain unobservable: the three components of global position, and the rotation about the gravity vector (global yaw). Neither sensor senses where the origin is or which way north is. VIO estimates these relative to the starting pose, and they drift.

1.4 Why fusion is hard

The difficulty of VIO comes from several sources acting at once.

1. **Nonlinearity.** Rotations enter the equations through products and trigonometric functions, and the camera projection divides by depth.
2. **Curved state spaces.** Orientations do not live in a vector space. You cannot add two rotations, and every estimator must deal with this (Chapter 5).

3. **Multi-rate, asynchronous data.** The IMU runs at 200 Hz and the camera at 20 Hz, and every measurement must be processed in timestamp order.
4. **Unknown scene.** The 3D positions of the visual features are unknown and must be estimated, or eliminated, along with the motion.
5. **Outliers.** Feature tracks fail, latch onto moving objects, or slide along edges.
6. **Unobservability.** Some directions of the state cannot be determined. A careless estimator can become falsely confident about them, which is the consistency problem of Chapter 20.
7. **Real time.** All of this must run at camera rate, deterministically, on embedded processors.

1.5 A map of the field

VIO algorithms differ along three main axes.

Loosely vs. tightly coupled. A loosely coupled system runs a vision-only pipeline that produces up-to-scale poses and fuses those poses with the IMU. A tightly coupled system feeds the raw visual measurements (feature positions in the image) directly into a single estimator together with the inertial data. Tight coupling is more accurate and more robust, because the IMU helps the vision part and correlations are handled correctly. All modern high-accuracy systems, and this project, are tightly coupled.

Filtering vs. optimization. *Filtering* approaches (EKF-based) process each measurement once, keep a Gaussian belief over a bounded state, and discard the past. They are fast and have predictable cost. *Optimization* approaches solve a nonlinear least-squares problem over a sliding window of recent states, relinearizing as needed, and summarize discarded states as a prior (marginalization). They are typically more accurate per frame but more expensive and more complex. The two families are closer than they appear: the MSCKF is equivalent to a single Gauss–Newton iteration over a sliding window of poses with the landmarks marginalized (Chapter 18).

Feature-based vs. direct. Feature-based systems extract and track sparse keypoints and use their image coordinates as measurements. Direct systems use pixel intensities themselves (photometric error).

1.6 Why the MSCKF

In EKF-SLAM every tracked landmark adds three variables to the state. Covariance updates cost $O(n^2)$ to $O(n^3)$ in the state size n, so a few hundred landmarks quickly exhaust a real-time budget, and landmark initialization from a single camera is awkward because depth is unknown at first sight.

System	Family	Notes
EKF-SLAM / MonoSLAM [10]	filter, landmarks in state	cost grows quadratically with landmarks
MSCKF [37, 31]	filter, poses in state	landmarks eliminated by nullspace projection; this project
MSCKF-VIO [48]	filter (stereo MSCKF)	open-source, fast flight; good engineering reference
OpenVINS [17]	filter (MSCKF + SLAM features)	research platform with excellent documentation
ROVIO [5]	filter, direct	photometric patches in an iterated EKF
OKVIS [30]	optimization, keyframes	sliding-window bundle adjustment
VINS-Mono [39]	optimization	preintegration, robust initialization, loop closure
ORB-SLAM3 [8]	optimization + map	full visual-inertial SLAM with relocalization

Table 1.2: Representative visual-inertial estimators.

The **Multi-State Constraint Kalman Filter** of Mourikis and Roumeliotis [37] keeps instead a sliding window of past IMU poses, called *clones*, taken at image times. When a feature has been tracked across several clones, its observations constrain those poses relative to one another. The feature's 3D position is triangulated temporarily, then algebraically eliminated from the linearized equations by projecting onto the left nullspace of its Jacobian (Chapter 18). The landmark never enters the state. The result:

- cost linear in the number of features and bounded by the window size;
- no need to initialize landmark depth inside the filter;
- all the geometric information of each feature track is used exactly once.

These properties make the MSCKF the canonical filter-based VIO and an excellent vehicle for learning, because it requires every ingredient of the field to be understood and nothing more.

1.7 The estimator at a glance

Figure 1.2 shows the whole estimator. Between images, each IMU sample advances the nominal state with the nonlinear kinematics and grows the error covariance. When an image arrives, the filter propagates exactly to the image timestamp, appends a clone of the current pose, and updates the feature tracks. Tracks that have ended, or whose oldest observation is about to leave the window, are triangulated and converted into constraints between clones. The update corrects the IMU state and all clones together, the correction is injected into the nominal state, and the oldest clone is removed when the window is full.

Project link: the road map

The project builds this pipeline from the bottom up: the Lie group toolkit (M2), the data stream (M3), inertial propagation (M4–M5), a filter tested against synthetic pose

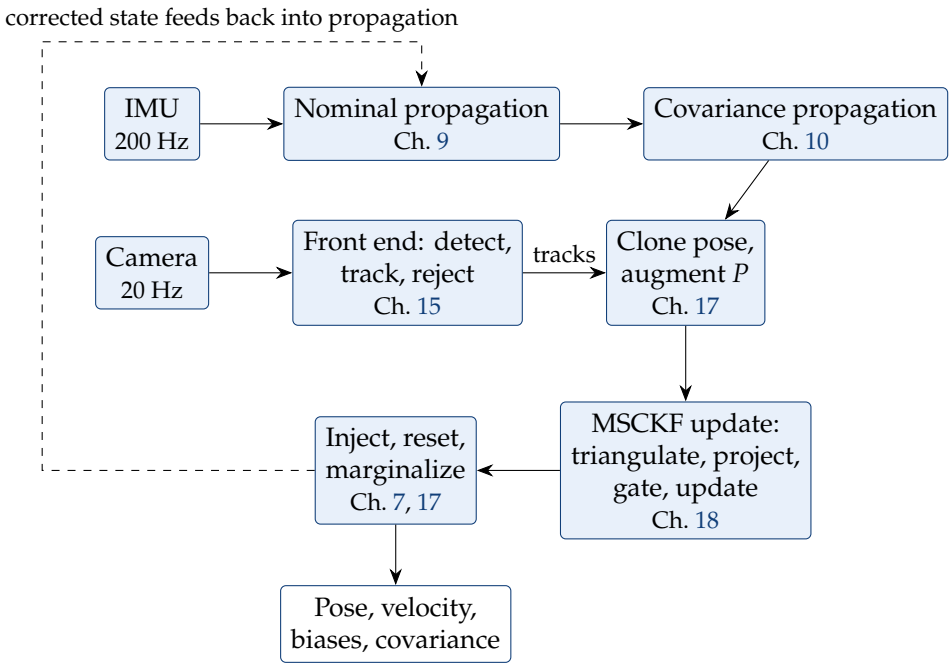


Figure 1.2: Data flow of the monocular MSCKF built in these notes.

measurements (M6–M7), the visual front end (M8–M9), and finally the MSCKF (M10a–M10b). Each layer is tested in isolation before the next is added, so that any later failure can be localized.

1.8 The EuRoC benchmark

The European Robotics Challenge (EuRoC) MAV dataset [7] is the standard VIO benchmark. A hexacopter carries a stereo pair of global-shutter cameras (752×480, 20 Hz) and an ADIS16448 MEMS IMU (200 Hz), hardware-synchronized and calibrated. Eleven sequences fly through an industrial machine hall (MH_01 to MH_05) and a motion-capture room (V1_01 to V2_03), graded easy, medium and difficult by speed, lighting and texture. Ground truth comes from a laser tracker or a motion-capture system. Appendix D documents the file formats and calibration values the project uses.

Key takeaways

- VIO estimates pose, velocity and IMU biases from an IMU and a camera, with no map and no absolute positioning, and its error grows without bound.
- Inertial drift is dominated by attitude error: a gyro bias produces position error growing like t^3.
- A monocular camera observes the scene only up to scale. The accelerometer supplies the scale, provided the body accelerates.
- Global position and yaw remain unobservable even with both sensors.
- The MSCKF keeps a sliding window of past poses rather than landmarks, which

bounds its cost while using every feature track.

Exercises

1.1 A MEMS accelerometer has a bias of 5 mg ($1\,\text{mg} = 9.81 \times 10^{-3}$ m/s^2). How long can you dead-reckon before the position error exceeds 1 m? What gyro bias produces the same error at the same time?

1.2 Prove that the gravity leak produced by a constant tilt error $\delta\theta_0$ is $g\,\delta\theta_0$ to first order. What is the exact expression, and at what tilt does the first-order formula err by 1%?

1.3 Extend Proposition 1.1: show that applying a common rigid transformation (R, t) to all camera poses and points also leaves the images unchanged. How many unobservable degrees of freedom does a monocular camera have? Which of them does an IMU remove, and why?

1.4 A drone hovers perfectly still for 30 s. Which states can a VIO system estimate during that time, and which cannot be distinguished from each other? (Revisit this after Chapter 20.)

1.5 Read the abstract and introduction of [37]. In two sentences, state the problem with EKF-SLAM that the paper claims to solve, and the key idea of the solution.

Chapter 2

Linear Algebra for State Estimation

In this chapter

- Master the cross-product matrix $[a]_\times$ and the identities used in every Jacobian derivation.
- Understand the four fundamental subspaces, especially the *left nullspace* on which the MSCKF rests.
- Know when and why to use Cholesky, LDL$^\mathsf{T}$, QR and SVD, and why a filter should never form an explicit inverse.
- Use block matrices, Schur complements and the Woodbury identity fluently.
- Understand the matrix exponential and floating-point arithmetic well enough to write numerically robust code.

Most of the computation in a Kalman filter is linear algebra: products of covariance matrices, solutions of linear systems, and factorizations. This chapter collects the tools the rest of the notes rely on and explains the numerical reasoning behind them. Readers fluent in numerical linear algebra can skim it, but should not skip Sections 2.1, 2.2 and 2.9.

2.1 Vectors, coordinates and the cross product

2.1.1 Vectors versus their coordinates

A physical vector, such as a velocity or the direction of gravity, exists independently of any coordinate system. To compute with it we express it in a *frame*: an origin plus three orthonormal, right-handed axes. The same physical vector $\vec{v}$ has different coordinate triples $v_w \in \mathbb{R}^3$ and $v_b \in \mathbb{R}^3$ in the world and body frames, and the two are related by a rotation matrix, $v_w = R_{wb}\, v_b$ (Chapter 4). Confusing a vector with its coordinates in some frame is the root of most frame bugs. Throughout these notes, a subscript on a coordinate vector names the frame it is expressed in.

For coordinate vectors $a, b \in \mathbb{R}^3$ we use the standard inner product $a^\mathsf{T} b = \sum_i a_i b_i$, the Euclidean norm $\|a\| = \sqrt{a^\mathsf{T} a}$, and the cross product $a \times b$. The cross product is bilinear, anti-commutative ($a \times b = -b \times a$), and orthogonal to both arguments. Its norm is $\|a\|\|b\| \sin \angle(a, b)$.

2.1.2 The cross-product (skew-symmetric) matrix

Because $a \times b$ is linear in b, it can be written as a matrix-vector product.

Definition 2.1 (Skew-symmetric matrix). For $a = (a_1, a_2, a_3)^\mathsf{T}$, the *cross-product matrix* of a is

$$[a]_\times = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix}, \qquad \text{so that} \qquad [a]_\times b = a \times b \quad \forall\, b \in \mathbb{R}^3. \tag{2.1}$$

It is skew-symmetric: $[a]_\times^\mathsf{T} = -[a]_\times$. Conversely, every 3×3 skew-symmetric matrix S equals $[s]_\times$ for the unique vector $s = (S_{32}, S_{13}, S_{21})^\mathsf{T}$. We write $s = S^\vee$ for this inverse operation, the *vee* map. In Lie theory, $[\cdot]_\times$ is also written $(\cdot)^\wedge$ and called the *hat* map.

The following identities are used constantly in derivations. Learn to recognize them on sight.

Proposition 2.2 (Cross-product identities). *For $a, b, c \in \mathbb{R}^3$ and $R \in \mathrm{SO}(3)$ (a rotation matrix):*

$$[a]_\times b = -[b]_\times a, \tag{2.2}$$
$$[a]_\times a = 0, \tag{2.3}$$
$$[a]_\times^2 = aa^\mathsf{T} - \|a\|^2 I, \tag{2.4}$$
$$[a]_\times^3 = -\|a\|^2 [a]_\times, \tag{2.5}$$
$$[a]_\times [b]_\times = ba^\mathsf{T} - (a^\mathsf{T} b)\, I, \tag{2.6}$$
$$[a \times b]_\times = [a]_\times [b]_\times - [b]_\times [a]_\times = ba^\mathsf{T} - ab^\mathsf{T}, \tag{2.7}$$
$$[Ra]_\times = R[a]_\times R^\mathsf{T}, \tag{2.8}$$
$$R\,(a \times b) = (Ra) \times (Rb). \tag{2.9}$$

Proof. (2.2) and (2.3) restate $a \times b = -b \times a$ and $a \times a = 0$. For (2.6), apply both sides to an arbitrary c and use the "BAC–CAB" rule $a \times (b \times c) = b\,(a^\mathsf{T} c) - c\,(a^\mathsf{T} b)$: $[a]_\times [b]_\times c = a \times (b \times c) = ba^\mathsf{T} c - (a^\mathsf{T} b)c$. Setting $b = a$ gives (2.4), and multiplying (2.4) by $[a]_\times$ and using (2.3) gives (2.5). Subtracting (2.6) from its version with a, b swapped gives the second equality of (2.7). The first follows from the Jacobi identity $a \times (b \times c) + b \times (c \times a) + c \times (a \times b) = 0$. For (2.9), the cross product is defined by orientation and lengths, both preserved by a proper rotation. Finally (2.8) follows from (2.9): $[Ra]_\times c = (Ra) \times c = (Ra) \times (RR^\mathsf{T} c) = R\big(a \times R^\mathsf{T} c\big) = R[a]_\times R^\mathsf{T} c$. ∎

Key idea: moving a vector out of a cross product

Identity (2.2) is how unknowns are isolated in Jacobian derivations. A perturbation appears as $[\delta\theta]_\times v$, and the Jacobian with respect to $\delta\theta$ is read off after rewriting it as $-[v]_\times\, \delta\theta$.

Identity (2.5) is what makes closed-form rotation formulas possible: every power of $[a]_\times$ reduces to a multiple of $[a]_\times$ or $[a]_\times^2$, so infinite series in $[a]_\times$ collapse to two terms (Chapter 4).

2.2 The four fundamental subspaces

Let $A \in \mathbb{R}^{m \times n}$ have rank r. Four subspaces are associated with it:

Subspace	Definition	Lives in	Dimension
column space (range)	$\mathrm{range}(A) = \{Ax\}$	$\mathbb{R}^m$	r
nullspace	$\mathrm{null}(A) = \{x : Ax = 0\}$	$\mathbb{R}^n$	$n - r$
row space	$\mathrm{range}(A^\mathsf{T})$	$\mathbb{R}^n$	r
left nullspace	$\mathrm{null}(A^\mathsf{T}) = \{y : y^\mathsf{T}A = 0\}$	$\mathbb{R}^m$	$m - r$

Theorem 2.3 (Fundamental theorem of linear algebra)**.** $\mathrm{null}(A) = \mathrm{range}(A^\mathsf{T})^\perp$ *and* $\mathrm{null}(A^\mathsf{T}) = \mathrm{range}(A)^\perp$*. Consequently* $\mathbb{R}^n = \mathrm{range}(A^\mathsf{T}) \oplus \mathrm{null}(A)$ *and* $\mathbb{R}^m = \mathrm{range}(A) \oplus \mathrm{null}(A^\mathsf{T})$*, with orthogonal direct sums.*

Proof. $y \in \mathrm{null}(A^\mathsf{T}) \iff A^\mathsf{T}y = 0 \iff x^\mathsf{T}A^\mathsf{T}y = 0\ \forall x \iff (Ax)^\mathsf{T}y = 0\ \forall x \iff y \perp \mathrm{range}(A)$. The first statement is the same argument applied to A^T. ∎

The *left nullspace* deserves special attention because the MSCKF is built on it. Suppose a stacked measurement depends on an unwanted variable f through a tall matrix $H_f \in \mathbb{R}^{m\times 3}$ of full column rank:

$$r = H_x\,\delta x + H_f\,\delta f + n. \tag{2.10}$$

Take any matrix $A \in \mathbb{R}^{m\times(m-3)}$ whose columns form an orthonormal basis of $\mathrm{null}(H_f^\mathsf{T})$. Then $A^\mathsf{T}H_f = 0$, and multiplying through by A^T gives $A^\mathsf{T}r = A^\mathsf{T}H_x\,\delta x + A^\mathsf{T}n$: the unwanted variable has disappeared. Chapter 18 shows that no information about δx is lost in the process.

Definition 2.4 (Orthogonal projector)**.** A matrix Π is an orthogonal projector if $\Pi^2 = \Pi$ and $\Pi^\mathsf{T} = \Pi$. If the columns of Q are an orthonormal basis of a subspace $\mathcal{S}$, the projector onto $\mathcal{S}$ is $\Pi_\mathcal{S} = QQ^\mathsf{T}$. If A has full column rank, the projector onto $\mathrm{range}(A)$ is $A(A^\mathsf{T}A)^{-1}A^\mathsf{T}$, and $I - \Pi_\mathcal{S}$ projects onto $\mathcal{S}^\perp$.

For the matrix A spanning the left nullspace of H_f, these facts combine into an identity we will use to prove the MSCKF's optimality:

$$AA^\mathsf{T} = I - H_f(H_f^\mathsf{T}H_f)^{-1}H_f^\mathsf{T}. \tag{2.11}$$

2.3 Symmetric and positive-definite matrices

Covariance matrices are symmetric and positive semi-definite, and much of the numerical care in a filter goes into keeping them that way.

Definition 2.5. A symmetric matrix $P = P^\mathsf{T} \in \mathbb{R}^{n\times n}$ is *positive semi-definite* (PSD, $P \succeq 0$) if $x^\mathsf{T}Px \geq 0$ for all x, and *positive definite* (PD, $P \succ 0$) if $x^\mathsf{T}Px > 0$ for all $x \neq 0$. We write $P_1 \succeq P_2$ (the Loewner order) if $P_1 - P_2 \succeq 0$.

Theorem 2.6 (Spectral theorem)**.** *Every real symmetric matrix factors as* $P = U\Lambda U^\mathsf{T}$ *with* U *orthogonal and* $\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_n)$ *real.* $P \succeq 0$ *if and only if all* $\lambda_i \geq 0$*, and* $P \succ 0$ *if and only if all* $\lambda_i > 0$*.*

The geometric picture is an ellipsoid. For $P \succ 0$, the set $\{x : x^\mathsf{T}P^{-1}x \leq 1\}$ is an ellipsoid with principal axes along the columns of U and semi-axis lengths $\sqrt{\lambda_i}$. This is the "one-sigma" uncertainty ellipsoid of a Gaussian with covariance P (Chapter 3).

Proposition 2.7 (Properties of PSD matrices). *Let $P, Q \succeq 0$ and let A be any matrix of compatible size. Then*

1. *$APA^\mathsf{T} \succeq 0$ (congruence preserves semi-definiteness);*
2. *$P + Q \succeq 0$, and $P + Q \succ 0$ if either is PD;*
3. *$P_{ii} \geq 0$ and $|P_{ij}| \leq \sqrt{P_{ii}P_{jj}}$;*
4. *every principal submatrix of P is PSD.*

Proof. (1) $x^\mathsf{T}APA^\mathsf{T}x = (A^\mathsf{T}x)^\mathsf{T}P(A^\mathsf{T}x) \geq 0$. (2) is immediate. (3) and (4) follow from (1) with A a selection matrix, using the 2×2 determinant for the off-diagonal bound. ∎

Property (1) explains why the covariance updates $\Phi P \Phi^\mathsf{T} + Q$ and the Joseph form (Chapter 6) preserve semi-definiteness exactly in exact arithmetic: they are sums of congruences of PSD matrices. The simpler update $(I - KH)P$ is *not* a congruence, which is why it can lose positive semi-definiteness under rounding.

2.4 Matrix factorizations

2.4.1 Cholesky and LDL$^\mathsf{T}$

Theorem 2.8 (Cholesky factorization). *A symmetric matrix S is positive definite if and only if $S = LL^\mathsf{T}$ for a unique lower triangular L with positive diagonal.*

The factor is computed column by column:

$$L_{jj} = \Big(S_{jj} - \sum_{k<j} L_{jk}^2\Big)^{1/2}, \qquad L_{ij} = \frac{1}{L_{jj}}\Big(S_{ij} - \sum_{k<j} L_{ik}L_{jk}\Big), \quad i > j, \tag{2.12}$$

at a cost of about $n^3/3$ flops, half that of an LU factorization. The algorithm is backward stable without pivoting, and it fails (a negative argument of the square root) exactly when S is not numerically positive definite. That makes it a useful diagnostic too. The variant $S = LDL^\mathsf{T}$ with unit-diagonal L and diagonal D avoids square roots, and with symmetric pivoting (Eigen's `LDLT`) it also handles semi-definite matrices gracefully.

Cholesky factors *whiten* correlated quantities. If $n \sim \mathcal{N}(0, S)$ and $S = LL^\mathsf{T}$, then $L^{-1}n \sim \mathcal{N}(0, I)$. This is how Mahalanobis distances are computed stably: $r^\mathsf{T}S^{-1}r = \|L^{-1}r\|^2$, with one triangular solve.

2.4.2 QR factorization and Householder reflections

Definition 2.9 (Householder reflection). For a nonzero $v \in \mathbb{R}^m$, the matrix $H_v = I - 2\,\dfrac{vv^\mathsf{T}}{v^\mathsf{T}v}$ reflects vectors across the hyperplane orthogonal to v. It is symmetric and orthogonal, $H_v = H_v^\mathsf{T} = H_v^{-1}$.

Given any $x \in \mathbb{R}^m$, choosing $v = x + \operatorname{sign}(x_1)\|x\|\,e_1$ gives $H_v x = -\operatorname{sign}(x_1)\|x\|\,e_1$: a single reflection zeroes every entry of x below the first. (The sign choice avoids cancellation when

x is already nearly parallel to e_1.) Applying such reflections to the columns of $A \in \mathbb{R}^{m\times n}$, $m \geq n$, one after another produces

$$H_n \cdots H_2 H_1 A = \begin{bmatrix} R_1 \\ 0 \end{bmatrix} \quad \Longleftrightarrow \quad A = Q \begin{bmatrix} R_1 \\ 0 \end{bmatrix} = \begin{bmatrix} Q_1 & Q_2 \end{bmatrix} \begin{bmatrix} R_1 \\ 0 \end{bmatrix} = Q_1 R_1, \tag{2.13}$$

with $Q = H_1 H_2 \cdots H_n$ orthogonal, $R_1 \in \mathbb{R}^{n\times n}$ upper triangular, $Q_1 \in \mathbb{R}^{m\times n}$ and $Q_2 \in \mathbb{R}^{m\times(m-n)}$. The cost is about $2mn^2 - \frac{2}{3}n^3$ flops.

Proposition 2.10 (Nullspaces from QR). *If A has full column rank n and $A = Q_1 R_1$ as in (2.13), then the columns of Q_1 are an orthonormal basis of* range(A) *and the columns of Q_2 are an orthonormal basis of the left nullspace* null(A^T). *In particular $Q_2^\mathsf{T} A = 0$.*

Proof. $Q^\mathsf{T} A = [R_1;\ 0]$, so $Q_2^\mathsf{T} A = 0$. The columns of Q_2 are orthonormal and there are $m - n$ of them, which is the dimension of null(A^T). ∎

Implementation note: never form Q

Q is $m \times m$ and dense, but it is defined by only n reflection vectors. Libraries store those vectors compactly in the lower triangle of the factored matrix. To compute $Q^\mathsf{T} B$ for some other matrix B, apply the n reflections to B one at a time, at a cost of $O(mnk)$ for $B \in \mathbb{R}^{m\times k}$. Forming Q explicitly would cost $O(m^2 n)$ and m^2 storage. In Eigen, `HouseholderQR::householderQ()` returns a lazy `HouseholderSequence` that can be applied to B without materializing Q. Whether a particular call allocates must be checked with the allocation guard, not assumed.

A *Givens rotation* zeroes a single entry by rotating two rows. It is the tool of choice when the matrix is already nearly triangular or very sparse. The original MSCKF paper suggests Givens rotations for its nullspace projection, because the feature Jacobian has a sparse block structure.

2.4.3 The singular value decomposition

Theorem 2.11 (SVD). *Every $A \in \mathbb{R}^{m\times n}$ factors as $A = U\Sigma V^\mathsf{T}$ with U, V orthogonal and Σ diagonal with entries $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$, the* singular values.

The SVD reveals everything about A at once. The rank is the number of nonzero singular values; the first r columns of U and V span the column and row spaces, and the remaining columns span the left nullspace and the nullspace. The *condition number* $\kappa(A) = \sigma_1/\sigma_n$ measures how much relative errors in the data can be amplified in the solution of a linear system. The minimum-norm least-squares solution of $Ax \approx b$ is $x = A^+ b$ with the pseudo-inverse $A^+ = V\Sigma^+ U^\mathsf{T}$. The SVD is the most robust of the factorizations and also the most expensive. In this project it appears in the linear triangulation of Chapter 14 and in diagnostics.

2.5 Solving linear systems and least squares

Pitfall: explicit inverses

Formulas such as $K = PH^\mathsf{T}S^{-1}$ are notation, not algorithms. Computing S^{-1} and then multiplying costs about three times as much as solving, is less accurate, and destroys structure. Factor S once (Cholesky or LDL$^\mathsf{T}$) and solve: $SK^\mathsf{T} = HP^\mathsf{T}$ gives K^T by two triangular substitutions per column. Writing `S.ldlt().solve(Identity)` is still an explicit inverse in disguise.

Least squares. Given an overdetermined system $Ax \approx b$ with $A \in \mathbb{R}^{m\times n}$ of full column rank, the least-squares solution minimizes $\|Ax - b\|^2$. Setting the gradient to zero gives the *normal equations* $A^\mathsf{T}Ax = A^\mathsf{T}b$. They are cheap but square the condition number, $\kappa(A^\mathsf{T}A) = \kappa(A)^2$, which can lose half the significant digits. The QR approach is more stable: with $A = Q_1R_1$, $\|Ax - b\|^2 = \|R_1x - Q_1^\mathsf{T}b\|^2 + \|Q_2^\mathsf{T}b\|^2$, so $x = R_1^{-1}Q_1^\mathsf{T}b$ by back substitution, and the residual norm is $\|Q_2^\mathsf{T}b\|$.

Weighted least squares. If the errors have covariance W, minimize $(Ax - b)^\mathsf{T}W^{-1}(Ax - b)$. With $W = LL^\mathsf{T}$ this is ordinary least squares on the whitened system $L^{-1}Ax \approx L^{-1}b$.

2.6 Block matrices and the Schur complement

Estimation constantly partitions vectors and matrices into blocks: IMU state versus clones, poses versus landmarks. Block elimination is the basic tool.

Proposition 2.12 (Block factorization and inverse). *Let* $M = \begin{bmatrix} A & B \\ C & D \end{bmatrix}$ *with* A *invertible, and define the* Schur complement *of* A *in* M, $M/A := D - CA^{-1}B$. *Then*

$$M = \begin{bmatrix} I & 0 \\ CA^{-1} & I \end{bmatrix} \begin{bmatrix} A & 0 \\ 0 & M/A \end{bmatrix} \begin{bmatrix} I & A^{-1}B \\ 0 & I \end{bmatrix}, \qquad \det M = \det A \cdot \det(M/A), \tag{2.14}$$

and if M/A *is invertible,*

$$M^{-1} = \begin{bmatrix} A^{-1} + A^{-1}B\,(M/A)^{-1}\,CA^{-1} & -A^{-1}B\,(M/A)^{-1} \\ -(M/A)^{-1}CA^{-1} & (M/A)^{-1} \end{bmatrix}. \tag{2.15}$$

Proof. Multiply out the three factors. The inverse is the product of the inverses of the three factors in reverse order, and the inverses of the unit-triangular block factors are obtained by negating their off-diagonal blocks. ∎

For a symmetric M with $A \succ 0$: $M \succ 0$ if and only if $M/A \succ 0$. The Schur complement is the algebraic form of *eliminating* a block of variables. Chapter 3 shows it is exactly Gaussian marginalization in information form and conditioning in covariance form.

Theorem 2.13 (Woodbury matrix identity). *For invertible A and C and conformable U, V,*

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U\left(C^{-1} + VA^{-1}U\right)^{-1}VA^{-1}. \tag{2.16}$$

Proof. Write the block matrix $\begin{bmatrix} A & U \\ V & -C^{-1} \end{bmatrix}$ and compute the top-left block of its inverse in two ways with Proposition 2.12: eliminating A first gives the right-hand side, and eliminating $-C^{-1}$ first gives $(A + UCV)^{-1}$. ∎

Woodbury connects the two forms of the Kalman update (Chapter 6). It says that a low-rank update of a matrix gives a low-rank update of its inverse, which is why an update with m measurements needs only an $m \times m$ inverse no matter how large the state is.

2.7 The matrix exponential

Definition 2.14. For $A \in \mathbb{R}^{n\times n}$, $\exp(A) = \sum_{k=0}^{\infty} \frac{A^k}{k!} = I + A + \frac{1}{2}A^2 + \frac{1}{6}A^3 + \cdots$. The series converges absolutely for every A.

Proposition 2.15 (Properties of the matrix exponential).

1. $x(t) = \exp(tA)\,x_0$ *is the unique solution of* $\dot{x} = Ax$, $x(0) = x_0$.
2. $\frac{\mathrm{d}}{\mathrm{d}t}\exp(tA) = A\exp(tA) = \exp(tA)\,A$.
3. $\exp(A)^{-1} = \exp(-A)$, *and* $\det\exp(A) = e^{\operatorname{tr} A}$.
4. $\exp(TAT^{-1}) = T\exp(A)\,T^{-1}$ *for invertible* T.
5. *If* $AB = BA$ *then* $\exp(A + B) = \exp(A)\exp(B)$. ***In general this fails.***

The failure of property 5 for non-commuting matrices is the algebraic root of everything subtle about rotations: two rotations generated by $[a]_\times$ and $[b]_\times$ do not compose into the rotation generated by $[a + b]_\times$. Chapter 5 measures the discrepancy precisely.

Example 2.16 (Nilpotent generator: the double integrator). For $F = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$, $F^2 = 0$, so the series terminates: $\exp(F\Delta t) = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}$. This is the exact transition matrix of position and velocity under zero acceleration. The inertial error dynamics of Chapter 10 have a similar nilpotent structure, so their exponentials can be written in closed form.

2.8 Derivatives and Jacobians

Definition 2.17 (Jacobian layout). For $f : \mathbb{R}^n \to \mathbb{R}^m$, the Jacobian $\frac{\partial f}{\partial x} \in \mathbb{R}^{m\times n}$ has entries $\left[\frac{\partial f}{\partial x}\right]_{ij} = \frac{\partial f_i}{\partial x_j}$ (numerator layout), so that $f(x + \delta) = f(x) + \frac{\partial f}{\partial x}\delta + o(\|\delta\|)$. Column j is the sensitivity of the output to the j-th input direction. This is exactly what a finite-difference test computes column by column.

With this layout the chain rule reads $\frac{\partial(g\circ f)}{\partial x} = \frac{\partial g}{\partial y}\big|_{y=f(x)} \frac{\partial f}{\partial x}$, a product of matrices in the same order as the composition. A few derivatives recur:

$$\frac{\partial(Ax)}{\partial x} = A, \qquad \frac{\partial(a \times x)}{\partial x} = [a]_\times, \qquad \frac{\partial(x \times a)}{\partial x} = -[a]_\times, \qquad \frac{\partial(x^\mathsf{T} S x)}{\partial x} = x^\mathsf{T}(S + S^\mathsf{T}). \tag{2.17}$$

Chapter 5 extends Jacobians to functions whose inputs or outputs are rotations.

2.9 Floating-point arithmetic

An IEEE 754 `double` stores a sign, an 11-bit exponent, and a 53-bit significand (52 stored bits plus an implicit leading one). Two constants matter:

- the *machine epsilon* $\varepsilon_{\text{mach}} = 2^{-52} \approx 2.22 \times 10^{-16}$, the gap between 1 and the next larger double (`std::numeric_limits<double>::epsilon()`);
- the *unit roundoff* $u = 2^{-53} \approx 1.11 \times 10^{-16}$: every basic operation satisfies $\mathrm{fl}(a \circ b) = (a \circ b)(1 + \delta)$ with $|\delta| \le u$.

Relative accuracy is therefore excellent for a single operation. Trouble comes from *cancellation*: subtracting two nearly equal numbers that each carry a rounding error leaves a small result whose relative error can be enormous.

Example 2.18 (Catastrophic cancellation in $1-\cos\theta$). For $\theta = 10^{-5}$, $\cos\theta = 1 - 5\times10^{-11} + \dots$. The stored value of $\cos\theta$ has an absolute error up to $u \approx 10^{-16}$, so $1 - \cos\theta \approx 5 \times 10^{-11}$ is computed with relative error up to $10^{-16}/(5 \times 10^{-11}) \approx 2 \times 10^{-6}$. Dividing by θ^2 to form $(1-\cos\theta)/\theta^2 \approx 1/2$ keeps that relative error. At $\theta = 10^{-8}$, $\cos\theta$ rounds to exactly 1, and the computed value is 0 instead of $1/2$. The Taylor expansion $\frac{1}{2} - \frac{\theta^2}{24} + \cdots$ has no cancellation at all. Figure 5.2 in Chapter 5 shows this effect for all the coefficient functions of the rotation group.

Pitfall: time stamps in floating point

EuRoC timestamps are nanoseconds since the Unix epoch, about 1.4×10^{18}, which lies between 2^{60} and 2^{61}. A `double` near that value can only represent multiples of $2^{60-52} = 256$ ns. Converting absolute timestamps to `double` therefore quantizes them to 256 ns, and accumulating time by repeated addition of Δt drifts further. Keep absolute time in 64-bit integers (`std::chrono::nanoseconds`) and convert only differences to `double` (AGENTS.md C5).

Finite-difference step size. A finite-difference derivative is itself a cancellation. For the central difference $\frac{f(x+h)-f(x-h)}{2h}$, the truncation error is about $\frac{h^2}{6}|f'''|$ and the rounding error about $\frac{u|f|}{h}$. Their sum is minimized at

$$h^\star = \left(\frac{3\,u\,|f|}{|f'''|}\right)^{1/3} \approx 10^{-5} \text{ to } 10^{-6} \quad \text{(for quantities of order one)}, \tag{2.18}$$

with a total error of order $u^{2/3} \approx 10^{-11}$. For the one-sided difference the optimal step is $\sim \sqrt{u} \approx 10^{-8}$ and the error is only $\sim 10^{-8}$. This is why Jacobian tests use central differences

with $h \approx 10^{-6}$ and tolerances around 10^{-6} to 10^{-8}, scaled to the magnitude of the quantities involved.

Conditioning versus stability. A problem is *ill-conditioned* if small changes in its data produce large changes in its exact answer. An algorithm is *unstable* if it produces much larger errors than the conditioning of the problem justifies. Computing the rotation angle from a rotation matrix with arccos near zero angle is an unstable algorithm for a well-conditioned problem (Chapter 5). Triangulating a point seen with almost no parallax is an ill-conditioned problem that no algorithm can rescue (Chapter 14).

Key takeaways

- $[a]_\times b = a \times b = -[b]_\times a$, $[a]_\times^3 = -\|a\|^2[a]_\times$, and $[Ra]_\times = R[a]_\times R^\mathsf{T}$ are the workhorses of every rotation derivation.
- The left nullspace of a tall matrix is the orthogonal complement of its column space. A full QR factorization delivers an orthonormal basis for it.
- Covariance updates should be congruences (APA^T) plus PSD terms, so that positive semi-definiteness is preserved structurally.
- Factor and solve; never form an explicit inverse, and never form Q explicitly.
- Schur complements eliminate variables; Woodbury converts between large and small inverses.
- Cancellation, not single operations, destroys floating-point accuracy. Time stamps belong in 64-bit integers.

Exercises

2.1 Prove (2.7) directly from (2.6), and verify the Jacobi identity numerically for random vectors.

2.2 Show that the eigenvalues of $[a]_\times$ are 0 and $\pm i\|a\|$. Which eigenvector belongs to 0?

2.3 Let $H_f \in \mathbb{R}^{2M\times 3}$ have full column rank. Compute the dimension of its left nullspace for $M = 2, 3, 10$. Prove (2.11).

2.4 Show that if $P \succ 0$ then $P - KSK^\mathsf{T} \succeq 0$ for $S = HPH^\mathsf{T} + R$, $K = PH^\mathsf{T}S^{-1}$ and $R \succ 0$. (Hint: Schur complement of a suitable 2×2 block matrix.)

2.5 Implement Householder QR in Python for a 10×3 matrix and check Proposition 2.10 numerically. Then compare the accuracy of solving a least-squares problem with $\kappa(A) = 10^8$ by the normal equations and by QR.

2.6 Use Theorem 2.13 to show that $\left(P^{-1} + H^\mathsf{T}R^{-1}H\right)^{-1} = P - PH^\mathsf{T}(HPH^\mathsf{T} + R)^{-1}HP$. Which side is cheaper to evaluate when P is 75×75 and R is 2×2? When R is 500×500?

2.7 What is the spacing of doubles near 2^{30} seconds expressed in nanoseconds? Would storing time as `double` seconds since the epoch be acceptable for a 200 Hz IMU?

2.8 For $f(x) = \sin x$ at $x = 1$, plot the error of forward and central differences versus $h \in [10^{-16}, 10^{-1}]$ on log axes and locate the optimal step sizes. Compare with the formulas above.

Chapter 3

Probability and Gaussian Algebra

In this chapter

- Propagate means and covariances through linear and linearized maps.
- Marginalize and condition Gaussians in both covariance and information form, and know which operation is cheap in which form.
- Connect Bayesian estimation, maximum a posteriori estimation and weighted least squares.
- Understand the χ^2 distribution that underlies gating and consistency tests.
- Model continuous-time sensor noise and random walks, and derive how integrated noise grows.

A filter is a machine for maintaining a probability distribution over the state. Kalman filters restrict that distribution to be Gaussian, which reduces the whole machinery to the manipulation of means and covariance matrices. This chapter develops that algebra.

3.1 Random vectors, mean and covariance

For a random vector $x \in \mathbb{R}^n$ with density $p(x)$, the mean and covariance are

$$\mu = \mathbb{E}[x] = \int x\, p(x)\, \mathrm{d}x, \qquad \Sigma = \mathrm{Cov}(x) = \mathbb{E}\big[(x-\mu)(x-\mu)^\mathsf{T}\big]. \tag{3.1}$$

Σ is symmetric and positive semi-definite: for any a, $a^\mathsf{T}\Sigma a = \mathbb{E}[(a^\mathsf{T}(x-\mu))^2] \geq 0$. Its diagonal holds the variances σ_i^2 and its off-diagonal entries the covariances. The correlation coefficient $\rho_{ij} = \Sigma_{ij}/(\sigma_i\sigma_j) \in [-1, 1]$ measures how strongly two components move together. The cross-covariance of two random vectors is $\Sigma_{xy} = \mathbb{E}[(x-\mu_x)(y-\mu_y)^\mathsf{T}]$.

Proposition 3.1 (Affine transformations). *If $y = Ax + b$ then $\mathbb{E}[y] = A\mu + b$ and $\mathrm{Cov}(y) = A\Sigma A^\mathsf{T}$. If x and w are uncorrelated, $\mathrm{Cov}(Ax + Bw) = A\Sigma_x A^\mathsf{T} + B\Sigma_w B^\mathsf{T}$.*

Proof. Linearity of expectation gives the mean. Then $\mathrm{Cov}(y) = \mathbb{E}[A(x-\mu)(x-\mu)^\mathsf{T}A^\mathsf{T}] = A\Sigma A^\mathsf{T}$. With uncorrelated x and w, the cross terms vanish. ∎

This one proposition is the covariance propagation of the Kalman filter ($P \leftarrow \Phi P \Phi^\mathsf{T} + Q$), the covariance of a stacked measurement, and the augmentation formula for stochastic cloning ($P \leftarrow JPJ^\mathsf{T}$).

3.2 The multivariate Gaussian

Definition 3.2 (Gaussian distribution). $x \sim \mathcal{N}(\mu, \Sigma)$ with $\Sigma \succ 0$ has density

$$p(x) = \frac{1}{\sqrt{(2\pi)^n \det \Sigma}} \exp\Big(-\tfrac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)\Big). \tag{3.2}$$

The quantity $d_M^2(x) = (x-\mu)^\top \Sigma^{-1}(x-\mu)$ is the squared *Mahalanobis distance* of x from the mean.

The Mahalanobis distance measures distance in units of standard deviations along each principal axis, so it is the natural way to ask whether a vector is "far" from a distribution. The level sets $d_M^2 = c$ are the ellipsoids of the previous chapter.

The Gaussian is closed under every operation a Kalman filter performs. Affine maps of Gaussians are Gaussian, and marginals, conditionals and products of Gaussian densities are Gaussian. This is why a Gaussian belief can be propagated exactly through a linear system with Gaussian noise, and why the Kalman filter is exact in that case.

Sampling. To draw from $\mathcal{N}(\mu, \Sigma)$, factor $\Sigma = LL^\top$ (Cholesky), draw $s \sim \mathcal{N}(0, I)$, and return $\mu + Ls$. By Proposition 3.1 its covariance is $LL^\top = \Sigma$. Monte Carlo tests of covariance propagation (Milestone 5) use exactly this.

Information form. A Gaussian can equally be parameterized by the *information matrix* $\Lambda = \Sigma^{-1}$ and the *information vector* $\eta = \Sigma^{-1}\mu$. Then $-\log p(x) = \tfrac{1}{2}x^\top \Lambda x - \eta^\top x + \text{const}$. Information form is natural for combining independent pieces of evidence, since their information adds, and it is the form that optimization-based estimators work in.

3.3 Marginals and conditionals

Partition a jointly Gaussian vector as

$$\begin{bmatrix} a \\ b \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \mu_a \\ \mu_b \end{bmatrix}, \begin{bmatrix} \Sigma_{aa} & \Sigma_{ab} \\ \Sigma_{ba} & \Sigma_{bb} \end{bmatrix} \right), \qquad \Lambda = \Sigma^{-1} = \begin{bmatrix} \Lambda_{aa} & \Lambda_{ab} \\ \Lambda_{ba} & \Lambda_{bb} \end{bmatrix}. \tag{3.3}$$

Theorem 3.3 (Gaussian marginalization and conditioning). *1. The marginal of a is $\mathcal{N}(\mu_a, \Sigma_{aa})$.*

2. The conditional of a given b is $\mathcal{N}(\mu_{a|b}, \Sigma_{a|b})$ with

$$\mu_{a|b} = \mu_a + \Sigma_{ab}\Sigma_{bb}^{-1}(b - \mu_b), \qquad \Sigma_{a|b} = \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}. \tag{3.4}$$

Proof. Define $\tilde{a} = a - \Sigma_{ab}\Sigma_{bb}^{-1} b$. Then $\operatorname{Cov}(\tilde{a}, b) = \Sigma_{ab} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{bb} = 0$, so $\tilde{a}$ and b are uncorrelated, and being jointly Gaussian they are independent. By Proposition 3.1, $\tilde{a} \sim \mathcal{N}(\mu_a - \Sigma_{ab}\Sigma_{bb}^{-1}\mu_b,\ \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba})$. Conditioning on b leaves the distribution of $\tilde{a}$ unchanged, and $a = \tilde{a} + \Sigma_{ab}\Sigma_{bb}^{-1} b$, which gives (3.4). The marginal follows from Proposition 3.1 with the selection matrix $A = [I \ \ 0]$. ∎

The conditional covariance $\Sigma_{a|b}$ is the Schur complement Σ/Σ_{bb}. Applying Proposition 2.12 to Σ shows that $\Lambda_{aa} = \Sigma_{a|b}^{-1}$ and that the marginal information matrix is a Schur complement of Λ. The two parameterizations therefore swap the roles of the two operations:

	covariance form (μ, Σ)	information form (η, Λ)
marginalize out b	Σ_{aa} (**delete rows/columns**)	$\Lambda_{aa} - \Lambda_{ab}\Lambda_{bb}^{-1}\Lambda_{ba}$ (Schur compl.)
condition on b	$\Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}$ (Schur compl.)	Λ_{aa} (**delete rows/columns**)

Key idea: marginalization is free in covariance form

A Kalman filter stores covariances, so discarding a variable (the oldest clone of the MSCKF window) is literally deleting its rows and columns from P. Optimization-based estimators store information matrices, so the same operation requires a Schur complement, which creates fill-in and the "marginalization prior" of sliding-window smoothers.

3.4 Bayesian estimation

Estimation combines a *prior* $p(x)$ with a *likelihood* $p(z \mid x)$, the probability of the measurement given the state, through Bayes' rule:

$$p(x \mid z) = \frac{p(z \mid x)\, p(x)}{p(z)} \propto p(z \mid x)\, p(x). \tag{3.5}$$

The posterior $p(x \mid z)$ is the complete answer. Point estimates summarize it:

- the *minimum mean-square error* (MMSE) estimate $\hat{x} = \mathbb{E}[x \mid z]$ minimizes $\mathbb{E}[\|x - \hat{x}\|^2 \mid z]$;
- the *maximum a posteriori* (MAP) estimate maximizes $p(x \mid z)$;
- the *maximum likelihood* (ML) estimate maximizes $p(z \mid x)$ and ignores the prior.

Proposition 3.4 (Linear-Gaussian case). *If $x \sim \mathcal{N}(\mu, P)$ and $z = Hx + n$ with $n \sim \mathcal{N}(0, R)$ independent of x, the posterior is Gaussian, MMSE and MAP coincide, and the estimate solves the weighted least-squares problem*

$$\hat{x} = \arg\min_x \ \|x - \mu\|_{P^{-1}}^2 + \|z - Hx\|_{R^{-1}}^2, \qquad \|e\|_W^2 := e^{\mathsf{T}} W e. \tag{3.6}$$

Proof. $-\log p(x \mid z) = \frac{1}{2}\|x - \mu\|_{P^{-1}}^2 + \frac{1}{2}\|z - Hx\|_{R^{-1}}^2 + \text{const}$ is a convex quadratic in x, so the posterior is Gaussian. A Gaussian's mode equals its mean. ∎

Chapter 6 solves (3.6) in closed form and obtains the Kalman update. The same objective with nonlinear $h(x)$ in place of Hx, iterated over a window of states, is the cost function of optimization-based VIO.

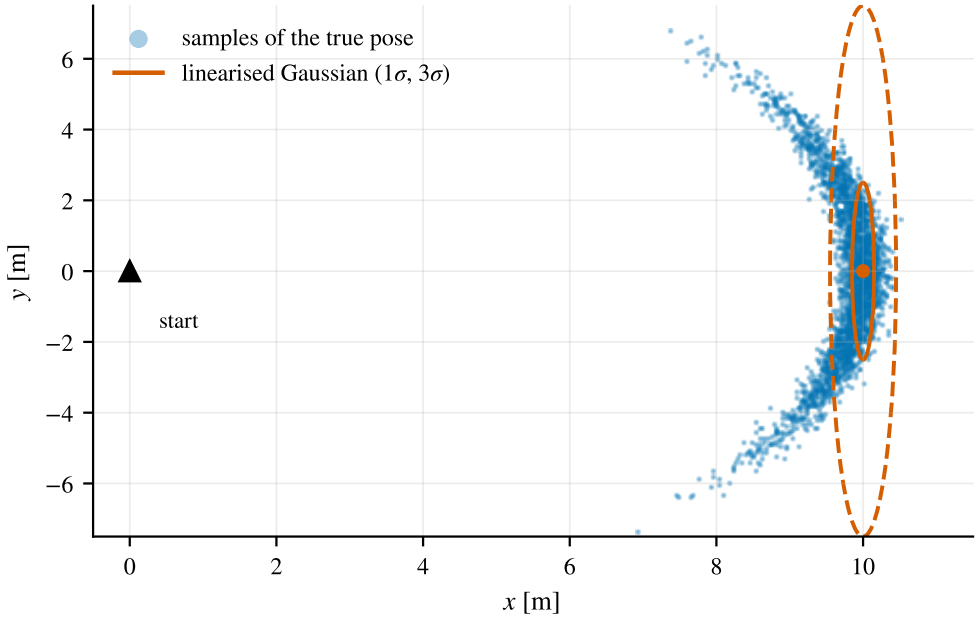


Figure 3.1: A robot drives 10 m with a heading uncertainty of $\sigma_\psi = 0.25$ rad. The true distribution of its final position (samples) is a curved "banana". The linearized Gaussian (ellipses) matches its first two moments only locally, puts probability where the robot cannot be, and misses the curvature. Errors of this kind are why an estimator must keep its errors small, the central design principle of the error-state filter.

3.5 Nonlinear transformations and linearization

If $y = f(x)$ with f nonlinear and $x \sim \mathcal{N}(\mu, \Sigma)$, then y is generally *not* Gaussian. The extended Kalman filter approximates it with a Gaussian by linearizing f about the mean:

$$y \approx f(\mu) + J(x - \mu), \quad J = \left.\frac{\partial f}{\partial x}\right|_{\mu} \qquad \Longrightarrow \qquad y \mathrel{\dot\sim} \mathcal{N}\big(f(\mu),\, J\Sigma J^{\mathsf{T}}\big). \tag{3.7}$$

The approximation is good when f is nearly linear over the region where x has most of its probability mass, a few standard deviations around μ. A second-order expansion shows that curvature also biases the mean by approximately $\frac{1}{2}\sum_i e_i \operatorname{tr}(\nabla^2 f_i\, \Sigma)$, which first-order linearization ignores.

Figure 3.1 shows a classic failure. When heading uncertainty is large, positions are distributed along an arc, and the linearized Gaussian is a poor description. Two remedies are available. One is to reduce the uncertainty that must be linearized: the error-state filter of Chapter 7 keeps the estimation error, not the state, as the quantity it linearizes, and continually resets it to zero. The other is to choose coordinates in which the distribution *is* close to Gaussian. A Gaussian on the heading and travelled distance, pushed through the exact kinematics, reproduces the banana. Chapter 5 develops that idea for rotations.

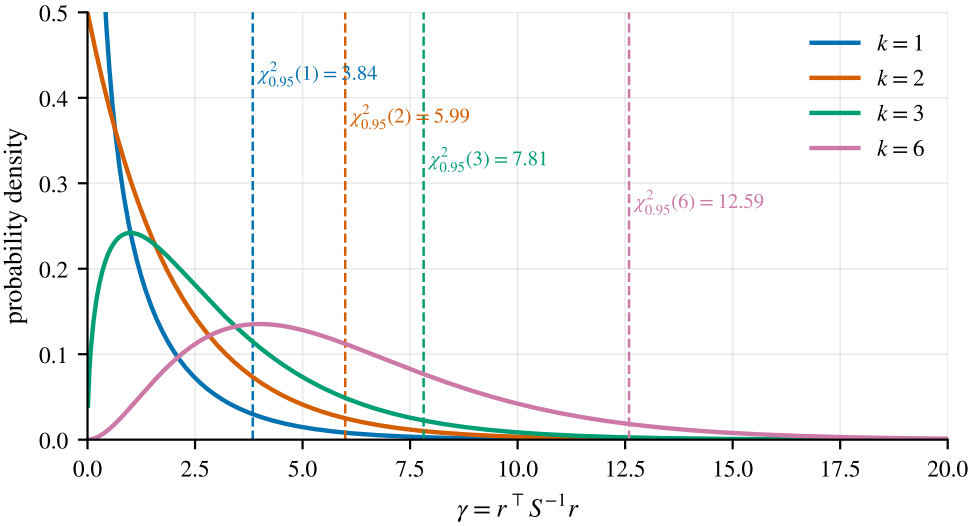


Figure 3.2: χ^2 densities and their 95% quantiles. A 2D image measurement whose normalized innovation squared exceeds 5.99 would occur by chance only 5% of the time if the filter's covariance were correct.

3.6 The chi-squared distribution

Definition 3.5. If $s_1, \dots, s_k$ are independent standard normal variables, $\gamma = \sum_{i=1}^{k} s_i^2$ has the χ^2 distribution with k degrees of freedom, written $\gamma \sim \chi^2_k$. It has mean k and variance $2k$.

Theorem 3.6 (Mahalanobis distance of a Gaussian). *If $x \sim \mathcal{N}(\mu, \Sigma)$ in $\mathbb{R}^k$ with $\Sigma \succ 0$, then $(x-\mu)^\top \Sigma^{-1}(x-\mu) \sim \chi^2_k$.*

Proof. With $\Sigma = LL^\top$, the whitened vector $s = L^{-1}(x-\mu)$ is $\mathcal{N}(0, I)$ by Proposition 3.1, and $(x-\mu)^\top \Sigma^{-1}(x-\mu) = s^\top s = \sum s_i^2$. ∎

This theorem is the basis of two tools. *Gating* (Chapter 11) rejects a measurement whose normalized innovation squared $r^\top S^{-1} r$ exceeds the 95% (or 99%) quantile of $\chi^2_{\dim r}$. *Consistency testing* (Chapter 12) checks that the normalized estimation error squared behaves like a χ^2 variable over time and over Monte Carlo runs. Because independent χ^2 variables add (degrees of freedom add), the sum of N independent NEES values of an n-dimensional state is χ^2_{Nn}. That gives exact confidence bounds for averages. Appendix C tabulates the quantiles.

3.7 Continuous-time noise

IMU noise is specified in continuous time, and getting its units right is essential.

Definition 3.7 (Continuous-time white noise). A zero-mean process $w(t) \in \mathbb{R}^n$ is white with *power spectral density* (PSD) Q_c if $\mathbb{E}[w(t)\, w(s)^\top] = Q_c\, \delta(t-s)$. For a scalar process we write $Q_c = \sigma^2$ and call σ the *noise density*.

White noise is an idealization with infinite variance. Its meaningful properties are those of its integrals. If w is measured in some unit $[u]$, then σ has units $[u]/\sqrt{\text{Hz}} = [u]\sqrt{\text{s}}$, because

$\delta(t-s)$ has units of $1/\mathrm{s}$. For a gyroscope, $[u] = \mathrm{rad/s}$, so σ_g is in $\mathrm{rad/s}/\sqrt{\mathrm{Hz}}$, exactly the units of `gyroscope_noise_density` in EuRoC's `sensor.yaml`.

Proposition 3.8 (Integrated white noise). *Let w be scalar white noise with density σ. The m-fold integral $I_m(t) = \int_0^t \int_0^{t_1} \cdots \int_0^{t_{m-1}} w(t_m)\,\mathrm{d}t_m \cdots \mathrm{d}t_1$ has zero mean and*

$$\operatorname{Var}\big(I_m(t)\big) = \sigma^2 \,\frac{t^{2m-1}}{(2m-1)\,\big((m-1)!\big)^2}. \tag{3.8}$$

For $m = 1, 2, 3, 4$ this gives $\sigma^2 t$, $\sigma^2 t^3/3$, $\sigma^2 t^5/20$ and $\sigma^2 t^7/252$.

Proof. By Cauchy's formula for repeated integration, $I_m(t) = \int_0^t \frac{(t-s)^{m-1}}{(m-1)!}\, w(s)\,\mathrm{d}s$. For a linear functional $\int g(s)w(s)\,\mathrm{d}s$ of white noise, the variance is $\iint g(s)g(s')\,\sigma^2\delta(s-s')\,\mathrm{d}s\,\mathrm{d}s' = \sigma^2 \int g(s)^2\,\mathrm{d}s$. Here $\int_0^t (t-s)^{2m-2}\,\mathrm{d}s/((m-1)!)^2 = t^{2m-1}/((2m-1)((m-1)!)^2)$. ∎

The single integral I_1 is the *Wiener process* or *random walk*, whose standard deviation grows as $\sqrt{t}$. A bias modeled as $\dot{b} = w$ is a random walk. Integrating accelerometer white noise twice into position gives $\sigma_a^2 t^3/3$, the analytic check of Milestone 5. Each further integration adds two powers of t to the variance. Chapter 10 uses this proposition to produce the drift laws of Figure 10.1.

From continuous to discrete noise. A digital sensor reports samples every Δt. If each sample is the average of the white-noise input over its interval (which an anti-aliasing filter approximates), the sample noise has variance

$$\operatorname{Var}\left(\frac{1}{\Delta t}\int_0^{\Delta t} w(s)\,\mathrm{d}s\right) = \frac{\sigma^2}{\Delta t}, \qquad \sigma_{\text{discrete}} = \frac{\sigma}{\sqrt{\Delta t}}. \tag{3.9}$$

For EuRoC's gyroscope, $\sigma_g = 1.6968\times10^{-4}\ \mathrm{rad/s}/\sqrt{\mathrm{Hz}}$ at 200 Hz gives a per-sample standard deviation of $2.4\times10^{-3}\ \mathrm{rad/s}$. Conversely, a random walk $\dot{b} = w$ discretized over Δt gains variance $\sigma_b^2\Delta t$ per step: $b_{k+1} = b_k + \sqrt{\Delta t}\,\sigma_b\, s_k$, $s_k \sim \mathcal{N}(0,1)$. The two conversions go in opposite directions; mixing them up is a classic tuning error.

3.8 Consistency

Definition 3.9 (Consistent estimator). An estimator that reports $(\hat{x}, P)$ is *consistent* if its error $e = x - \hat{x}$ satisfies $\mathbb{E}[e] = 0$ and $\mathbb{E}[ee^{\mathsf{T}}] = P$. It is *overconfident* if the actual error covariance exceeds P, and *underconfident* if it is smaller.

Consistency matters more than it first appears. An overconfident filter trusts its own prediction too much, so it gives new measurements too little weight. Its errors then grow, which makes it even more inconsistent, and it eventually rejects valid measurements in its gate. Most catastrophic filter failures in practice begin as gradual overconfidence. Chapter 12 develops the tests and Chapter 20 explains the structural cause in VIO.

Key takeaways

- Affine maps propagate covariances as $A\Sigma A^\mathsf{T}$. This single fact drives prediction, cloning and stacking.
- In covariance form, marginalization deletes rows and columns and conditioning takes a Schur complement. Information form is the reverse.
- For linear-Gaussian models, the Bayesian posterior mean is the weighted least-squares solution.
- Linearization approximates a nonlinear transformation by a Gaussian, and fails when uncertainty is large relative to curvature.
- The Mahalanobis distance of a Gaussian vector is χ^2 with dim degrees of freedom.
- White-noise densities have units of $[u]/\sqrt{\text{Hz}}$. The m-fold integral of white noise has variance growing like t^{2m-1}.

Exercises

3.1 Let $x \sim \mathcal{N}(0, \operatorname{diag}(1, 4))$ and $y = Ax$ with A a rotation by $30°$. Compute $\operatorname{Cov}(y)$ and sketch the 1σ ellipses of x and y.

3.2 Prove that $\Sigma_{a|b} \preceq \Sigma_{aa}$: conditioning never increases uncertainty in the Loewner order. When is the inequality an equality?

3.3 Derive the information-form marginal $\Lambda_{aa} - \Lambda_{ab}\Lambda_{bb}^{-1}\Lambda_{ba}$ from Proposition 2.12, and show it equals Σ_{aa}^{-1}.

3.4 Solve (3.6) by setting the gradient to zero, and show that the solution can be written $\hat{x} = \mu + K(z - H\mu)$ for $K = (P^{-1} + H^\mathsf{T}R^{-1}H)^{-1}H^\mathsf{T}R^{-1}$.

3.5 For $x \sim \mathcal{N}(0, \sigma^2)$ and $y = x^2$, compute the exact mean of y and its first-order linearized prediction. Explain the discrepancy.

3.6 Verify Proposition 3.8 by Monte Carlo for $m = 2$ and $m = 3$, simulating at 200 Hz for 10 s with 1000 runs.

3.7 An accelerometer has density $\sigma_a = 2 \times 10^{-3}$ m/s$^2/\sqrt{\text{Hz}}$. A user samples it at 1 kHz instead of 200 Hz. How does the per-sample noise change? How does the position uncertainty after 10 s of integration change? Explain why.

3.8 Show that the 95% quantile of χ_k^2 divided by k tends to 1 as $k \to \infty$, and explain what that implies for gating high-dimensional stacked measurements.

Chapter 4

Frames, Rotations and Quaternions

In this chapter

- Read and write frame-annotated transformations such as R_{wb}, p_{wb} and T_{bc} without ambiguity, and catch frame bugs with the adjacency rule.
- Derive Rodrigues' formula and understand the axis-angle (rotation vector) representation.
- See exactly why Euler angles are banned from the estimator.
- Work fluently with Hamilton unit quaternions, and avoid the Eigen and EuRoC ordering traps.
- Derive the kinematic equations $\dot{R} = R[\omega]_\times$ and $\dot{q} = \frac{1}{2} q \otimes [0, \omega]$.

4.1 Frames and notation

A *frame* is an origin together with three orthonormal, right-handed axes. Three frames appear in this project (Figure 4.1).

Frozen convention: frames, AGENTS.md C1

- W, the **world** frame: fixed, gravity-aligned with z pointing up, the frame of the ground truth (EuRoC calls it `R`). Gravity is $g_w = [0, 0, -9.81]^\mathsf{T}$ m/s^2.
- B, the **body** frame: the IMU frame `imu0` (EuRoC's `S` in `imu0`).
- C, the left **camera** frame `cam0`: origin at the optical center, z along the optical axis, x to the right and y down in the image.

R_{wb} (or q_{wb}) maps body-frame coordinates to world-frame coordinates, $v_w = R_{wb} v_b$. p_{wb} is the position of the origin of B expressed in W. v_{wb} is the velocity of B relative to W, expressed in W. $T_{bc} = (R_{bc}, p_{bc})$ maps camera points into the body frame, $p_b = R_{bc} p_c + p_{bc}$; it is `T_BS` from `cam0/sensor.yaml`. A world-to-body rotation is never used as a state.

4.1.1 Reading subscripts

The naming rule `<quantity>_<to><from>` makes products self-checking. R_{wb} takes a vector *from B to W*. Composition chains frames like dominoes:

$$R_{wc} = R_{wb}\, R_{bc}, \qquad v_w = R_{wb}\, R_{bc}\, v_c. \tag{4.1}$$

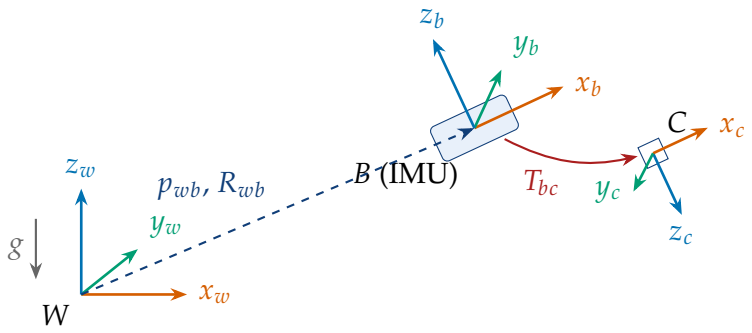


Figure 4.1: The world frame W (gravity-aligned, z up), the body/IMU frame B moving with the vehicle, and the camera frame C rigidly attached to the body through the constant extrinsic calibration T_{bc}.

Key idea: the adjacency rule

In a valid product, adjacent subscripts match: $R_{w\underline{b}}\, R_{\underline{b}c}$. A product such as $R_{wb}R_{cb}$ has mismatched neighbors and is a bug. The fix is a transpose, $R_{wb}R_{cb}^{\mathsf{T}} = R_{wb}R_{bc}$, since $R_{bc} = R_{cb}^{-1} = R_{cb}^{\mathsf{T}}$. Apply the rule to every line of code that multiplies rotations or transforms points.

4.1.2 Active and passive views

A rotation matrix can be read in two ways. In the *passive* view, R_{wb} re-expresses the same physical vector in another frame: v_w and v_b are coordinates of one arrow. In the *active* view, R_{wb} rotates the world axes onto the body axes: its columns are the body axes x_b, y_b, z_b expressed in world coordinates. Both readings are the same matrix:

$$R_{wb} = \begin{bmatrix} x_b^{(w)} & y_b^{(w)} & z_b^{(w)} \end{bmatrix}. \tag{4.2}$$

This gives a quick sanity check when debugging. The third column of R_{wb} is the body z-axis in world coordinates. For a level vehicle it is $[0, 0, 1]^{\mathsf{T}}$.

4.2 Rotation matrices and SO(3)

Definition 4.1 (Special orthogonal group). $\mathrm{SO}(3) = \{R \in \mathbb{R}^{3\times 3} : R^{\mathsf{T}}R = I,\ \det R = +1\}$.

Orthogonality ($R^{\mathsf{T}}R = I$) means R preserves lengths and angles. $\det R = +1$ excludes reflections, which have det $= -1$ and would turn a right-handed frame into a left-handed one. The set is closed under multiplication and inversion, $R^{-1} = R^{\mathsf{T}}$, so it is a *group*. A rotation matrix has nine entries but only three degrees of freedom, since orthonormality imposes six constraints. This redundancy is why SO(3) is a curved three-dimensional surface inside $\mathbb{R}^9$, not a vector space. The sum of two rotation matrices, for example, is not a rotation matrix.

The elementary rotations about the coordinate axes are

$$R_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_\alpha & -s_\alpha \\ 0 & s_\alpha & c_\alpha \end{bmatrix}, \quad R_y(\alpha) = \begin{bmatrix} c_\alpha & 0 & s_\alpha \\ 0 & 1 & 0 \\ -s_\alpha & 0 & c_\alpha \end{bmatrix}, \quad R_z(\alpha) = \begin{bmatrix} c_\alpha & -s_\alpha & 0 \\ s_\alpha & c_\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}, \tag{4.3}$$

with $c_\alpha = \cos\alpha$ and $s_\alpha = \sin\alpha$. Matrix multiplication of rotations does not commute: $R_x(90°)R_y(90°) \neq R_y(90°)R_x(90°)$. Try it with a book.

4.3 Axis-angle and Rodrigues' formula

Theorem 4.2 (Euler's rotation theorem). *Every $R \in \mathrm{SO}(3)$ is a rotation by some angle $\theta \in [0, \pi]$ about some unit axis u, the eigenvector of R with eigenvalue 1.*

The pair (u, θ) is the *axis-angle* representation, and $\phi = \theta u \in \mathbb{R}^3$ is the *rotation vector*. Its direction is the axis and its length is the angle. The rotation vector is the central object of the next chapter.

Theorem 4.3 (Rodrigues' rotation formula). *The rotation by angle θ about the unit axis u is*

$$R = I + \sin\theta\,[u]_\times + (1 - \cos\theta)\,[u]_\times^2 = \cos\theta\, I + (1 - \cos\theta)\, uu^\mathsf{T} + \sin\theta\,[u]_\times. \tag{4.4}$$

Geometric proof. Split any vector into components parallel and perpendicular to the axis, $v = v_\parallel + v_\perp$ with $v_\parallel = uu^\mathsf{T}v$. Rotation leaves $v_\parallel$ unchanged and rotates $v_\perp$ by θ within the plane spanned by $v_\perp$ and $u \times v_\perp = u \times v$:

$$Rv = v_\parallel + \cos\theta\, v_\perp + \sin\theta\, u \times v.$$

Substituting $v_\perp = v - uu^\mathsf{T}v = -[u]_\times^2 v$ (from (2.4) with $\|u\| = 1$) and $u \times v = [u]_\times v$ gives $Rv = (I + \sin\theta[u]_\times + (1 - \cos\theta)[u]_\times^2)v$. ∎

Algebraic proof. The rotation about u at unit angular rate satisfies $\dot{R} = [u]_\times R$ (Section 4.7), so $R(\theta) = \exp(\theta[u]_\times)$. With $[u]_\times^3 = -[u]_\times$ from (2.5), the exponential series regroups as

$$\exp(\theta[u]_\times) = I + \left(\theta - \tfrac{\theta^3}{3!} + \tfrac{\theta^5}{5!} - \cdots\right)[u]_\times + \left(\tfrac{\theta^2}{2!} - \tfrac{\theta^4}{4!} + \cdots\right)[u]_\times^2,$$

and the two series are $\sin\theta$ and $1 - \cos\theta$. ∎

Two consequences of (4.4) let us go back from R to (u, θ). Taking the trace (using $\mathrm{tr}[u]_\times = 0$ and $\mathrm{tr}[u]_\times^2 = -2$) and the skew-symmetric part,

$$\mathrm{tr}\, R = 1 + 2\cos\theta, \qquad R - R^\mathsf{T} = 2\sin\theta\,[u]_\times. \tag{4.5}$$

Chapter 5 shows that *how* these are combined numerically matters a great deal.

4.4 Euler angles and gimbal lock

Euler angles describe a rotation as three successive rotations about coordinate axes. The aerospace roll-pitch-yaw (ZYX) convention writes

$$R_{wb} = R_z(\psi)\, R_y(\vartheta)\, R_x(\varphi), \tag{4.6}$$

with yaw ψ, pitch ϑ and roll φ. They are intuitive, which is why they are used for display. Their fatal flaw appears in the kinematics. The relation between the body angular velocity measured by a gyroscope and the Euler angle rates is

$$\begin{bmatrix} \dot{\varphi} \\ \dot{\vartheta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} 1 & \sin\varphi \tan\vartheta & \cos\varphi \tan\vartheta \\ 0 & \cos\varphi & -\sin\varphi \\ 0 & \sin\varphi/\cos\vartheta & \cos\varphi/\cos\vartheta \end{bmatrix} \omega_b. \tag{4.7}$$

At $\vartheta = \pm 90°$, $\cos\vartheta = 0$ and the matrix blows up. Geometrically, when the nose points straight up, yaw and roll rotate about the same physical axis, so the three angles no longer describe three independent motions. This is *gimbal lock*, named after mechanical gimbal platforms that physically jam in this configuration. Near the singularity, small angular velocities produce enormous Euler-angle rates, and a filter that linearizes in Euler angles becomes numerically meaningless.

Frozen convention: AGENTS.md C2

Euler angles never appear in the estimator. They may appear only in telemetry and plots.

No three-parameter representation of SO(3) is free of singularities everywhere, a topological fact (the space SO(3) cannot be covered by a single chart). The solution adopted by modern estimators is to store attitude in a singularity-free *redundant* representation (a unit quaternion or a rotation matrix) and to use three parameters only for *small* deviations around the current estimate, where no singularity is near. That is the content of the next chapter.

4.5 Quaternions

4.5.1 Quaternion algebra

A quaternion is a four-component number $q = w + x\,i + y\,j + z\,k$, which we write as a scalar part and a vector part, $q = [w,\ v]$ with $v = (x, y, z)^{\mathsf{T}}$. Hamilton's multiplication rules are

$$i^2 = j^2 = k^2 = ijk = -1, \qquad \text{hence} \qquad ij = k,\ jk = i,\ ki = j,\ ji = -k, \ldots \tag{4.8}$$

Expanding the product of two quaternions with these rules gives

$$p \otimes q = \begin{bmatrix} p_w q_w - p_v^{\mathsf{T}} q_v \\ p_w q_v + q_w p_v + p_v \times q_v \end{bmatrix}. \tag{4.9}$$

The cross product makes the product non-commutative. Because it is linear in each factor, it can be written with matrices,

$$p \otimes q = [p]_L\, q = [q]_R\, p, \qquad [p]_L = \begin{bmatrix} p_w & -p_v^\mathsf{T} \\ p_v & p_w I + [p_v]_\times \end{bmatrix}, \quad [q]_R = \begin{bmatrix} q_w & -q_v^\mathsf{T} \\ q_v & q_w I - [q_v]_\times \end{bmatrix}, \tag{4.10}$$

where q on the right of $[p]_L\, q$ is the 4-vector $[q_w; q_v]$. The *conjugate* is $q^* = [w,\ -v]$, the norm is $\|q\| = \sqrt{q \otimes q^*} = \sqrt{w^2 + \|v\|^2}$, and $q^{-1} = q^*/\|q\|^2$. Products reverse under conjugation: $(p \otimes q)^* = q^* \otimes p^*$.

4.5.2 Unit quaternions represent rotations

Theorem 4.4 (Rotation by a unit quaternion). *For a unit quaternion $q = [w, v]$, the map $a \mapsto q \otimes [0, a] \otimes q^*$ sends pure quaternions to pure quaternions and acts on a as the rotation matrix*

$$R(q) = (w^2 - v^\mathsf{T} v)\, I + 2\, v v^\mathsf{T} + 2w\, [v]_\times = \begin{bmatrix} 1 - 2(y^2 + z^2) & 2(xy - wz) & 2(xz + wy) \\ 2(xy + wz) & 1 - 2(x^2 + z^2) & 2(yz - wx) \\ 2(xz - wy) & 2(yz + wx) & 1 - 2(x^2 + y^2) \end{bmatrix}. \tag{4.11}$$

The unit quaternion of a rotation by θ about the unit axis u is

$$q = \big[\cos(\theta/2),\ \sin(\theta/2)\, u\big]. \tag{4.12}$$

Proof. Expand $q \otimes [0, a] \otimes q^*$ twice with (4.9). The scalar part is 0 and the vector part is $(w^2 - v^\mathsf{T} v)a + 2(v^\mathsf{T} a)v + 2w\, v \times a$, which is (4.11). For (4.12), substitute $w = \cos\frac{\theta}{2}$, $v = \sin\frac{\theta}{2} u$ and use the double-angle identities: $w^2 - v^\mathsf{T} v = \cos\theta$, $2\sin^2\frac{\theta}{2} = 1 - \cos\theta$ and $2\sin\frac{\theta}{2}\cos\frac{\theta}{2} = \sin\theta$. The result is the second form of Rodrigues' formula (4.4). ∎

The *half* angle in (4.12) is the most common source of factor-of-two bugs. It arises because q appears twice in the sandwich product $q \otimes [0, a] \otimes q^*$.

Key idea: double cover

q and $-q$ give the same $R(q)$, since (4.11) is quadratic in q. Every rotation has exactly two unit quaternions, at opposite points of the unit sphere $S^3 \subset \mathbb{R}^4$. They correspond to rotating by θ about u, or by $2\pi - \theta$ about $-u$. Code that compares, averages, interpolates or takes logarithms of quaternions must account for this. The project canonicalizes to $w \geq 0$ before taking Log_q (AGENTS.md C2).

4.5.3 Composition and the Hamilton convention

With the Hamilton product, quaternion multiplication composes rotations in the same order as matrix multiplication:

$$R(p \otimes q) = R(p)\, R(q), \qquad q_{wc} = q_{wb} \otimes q_{bc} \ \leftrightarrow\ R_{wc} = R_{wb} R_{bc}. \tag{4.13}$$

The adjacency rule therefore applies to quaternions unchanged.

Unfortunately, two incompatible quaternion conventions are in wide use. Much of the classic VIO literature, including the original MSCKF paper [37] and the indirect Kalman filter report of Trawny and Roumeliotis [49], uses the *JPL* convention. Formulas cannot be copied between the two without translation [46].

	Hamilton (this project)	**JPL**
Multiplication rule	$ij = k$ ($ijk = -1$)	$ij = -k$ ($ijk = +1$)
Component order	usually $[w, x, y, z]$	usually $[x, y, z, w]$
Typical meaning	body-to-world (active), q_{wb}	world-to-body (passive), ${}^{B}_{G}\bar{q}$
Composition	$q_{wc} = q_{wb} \otimes q_{bc}$	${}^{C}_{G}\bar{q} = {}^{C}_{B}\bar{q} \otimes {}^{B}_{G}\bar{q}$
Used by	Eigen, ROS, Solà [44], Ceres, Sophus	MSCKF papers, Trawny [49], NASA

Table 4.1: Hamilton versus JPL quaternions. The JPL product of two quaternions equals the Hamilton product of the same numbers in reverse order.

Frozen convention: quaternions, AGENTS.md C2

Hamilton quaternions, `Eigen::Quaterniond`, always of unit norm, normalized after every multiplicative update. Never substitute JPL formulas, even when a paper you are reading uses them. Translate them first.

4.5.4 Quaternions in code and data

Pitfall: coefficient order

Three orderings meet at the project's input boundary:

- `Eigen::Quaterniond(w, x, y, z)`: the constructor takes w *first*;
- `q.coeffs()`: Eigen's storage is $[x, y, z, w]$, with w *last*;
- EuRoC CSV files store `qw, qx, qy, qz`, with w first.

Mapping a raw EuRoC row with `Eigen::Map<Quaterniond>` interprets it as $[x, y, z, w]$ and silently produces a different, valid-looking rotation. Always construct explicitly, `Eigen::Quaterniond(row[0], row[1], row[2], row[3])`, and cover the parser with a fixture test.

Other Eigen facts worth knowing: `q * v` for a `Vector3d` applies $R(q)$ to v; `q1 * q2` is the Hamilton product; `q.conjugate()` is the inverse of a unit quaternion; `q.normalize()` works in place while `q.normalized()` returns a copy; and `q.toRotationMatrix()` evaluates (4.11).

Normalization. Each multiplicative update of a quaternion rounds its four components, so $\|q\|$ performs a slow random walk away from 1. A non-unit quaternion inserted into (4.11) produces a matrix that scales vectors as well as rotating them. Dividing by the norm after every update costs a few flops and removes the problem. Milestone 4 checks that $|\|q\| - 1| < 10^{-12}$ after 10^6 updates.

From a rotation matrix to a quaternion. Inverting (4.11) naively via $w = \frac{1}{2}\sqrt{1 + \operatorname{tr} R}$ and $v = \frac{(R - R^{\mathsf{T}})^{\vee}}{4w}$ divides by a number that vanishes for 180° rotations. Robust implementations

(Shepperd's method, used by Eigen) first determine which of w, x, y, z has the largest magnitude from the diagonal of R, compute that one with a square root, and obtain the other three by division by it.

Interpolation. Ground truth and IMU data rarely share timestamps with images. The correct way to interpolate between two attitudes is along the shortest great arc (SLERP), which in the notation of the next chapter reads $q(\tau) = q_0 \otimes \operatorname{Exp}_q\big(\tau \operatorname{Log}_q(q_0^{-1} \otimes q_1)\big)$ for $\tau \in [0, 1]$. Component-wise linear interpolation followed by normalization is an acceptable approximation only for small angles.

4.6 Rigid-body transformations

A rigid transformation combines a rotation and a translation. Point coordinates transform as

$$p_w = R_{wb}\, p_b + p_{wb} \quad \Longleftrightarrow \quad \begin{bmatrix} p_w \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} R_{wb} & p_{wb} \\ 0 & 1 \end{bmatrix}}_{T_{wb}} \begin{bmatrix} p_b \\ 1 \end{bmatrix}. \tag{4.14}$$

The 4×4 homogeneous matrices form the group SE(3). Composition and inversion follow from block multiplication:

$$T_{wc} = T_{wb} T_{bc} = \begin{bmatrix} R_{wb} R_{bc} & R_{wb} p_{bc} + p_{wb} \\ 0 & 1 \end{bmatrix}, \qquad T_{wb}^{-1} = T_{bw} = \begin{bmatrix} R_{wb}^{\mathsf{T}} & -R_{wb}^{\mathsf{T}} p_{wb} \\ 0 & 1 \end{bmatrix}. \tag{4.15}$$

The camera pose in the world therefore is $R_{wc} = R_{wb} R_{bc}$ and $p_{wc} = p_{wb} + R_{wb} p_{bc}$, and a world point p_f has camera coordinates

$$p_c = R_{bc}^{\mathsf{T}}\big(R_{wb}^{\mathsf{T}}(p_f - p_{wb}) - p_{bc}\big), \tag{4.16}$$

which is the first half of the camera measurement model (AGENTS.md C6).

Pitfall: points versus directions

Translations apply to points, not to free vectors. A velocity, a gravity vector or a bearing direction transforms with the rotation only, $v_w = R_{wb} v_b$. In homogeneous coordinates directions carry a 0 in the fourth component, which drops the translation automatically.

4.7 Rotational kinematics

4.7.1 Angular velocity

Let $R(t) = R_{wb}(t)$ describe a rotating body. Differentiating $R^{\mathsf{T}} R = I$ gives $\dot{R}^{\mathsf{T}} R + R^{\mathsf{T}} \dot{R} = 0$, so $R^{\mathsf{T}} \dot{R}$ is skew-symmetric and equals $[\omega_b]_\times$ for a unique vector ω_b:

$$\dot{R}_{wb} = R_{wb}\, [\omega_b]_\times = [\omega_w]_\times\, R_{wb}, \qquad \omega_w = R_{wb}\, \omega_b, \tag{4.17}$$

where the second form uses (2.8). ω_b is the angular velocity of B relative to W, expressed in B, and it is exactly what a strapdown gyroscope measures (up to bias and noise). The two forms of (4.17) are the kinematic origin of the two perturbation conventions of the next chapter. A body-frame quantity multiplies on the right, and a world-frame quantity on the left.

Proposition 4.5 (Constant body rate). *If ω_b is constant on $[t_0, t_0 + \Delta t]$, then $R(t_0 + \Delta t) = R(t_0)\exp(\Delta t[\omega_b]_\times)$.*

Proof. $R(t) = R(t_0)\exp((t - t_0)[\omega_b]_\times)$ satisfies (4.17) by Proposition 2.15, and the solution is unique. ∎

This exact solution is the basis of the multiplicative attitude update used in every integration scheme of Chapter 9. The body rate is held constant over the step and the incremental rotation is applied on the right. Note that the solution is *not* $R(t_0) + \Delta t\, R(t_0)[\omega_b]_\times$, which would leave SO(3).

4.7.2 Quaternion kinematics

The quaternion counterpart of (4.17) is

$$\dot{q}_{wb} = \tfrac{1}{2}\, q_{wb} \otimes [0,\ \omega_b], \tag{4.18}$$

with constant-rate solution $q(t_0 + \Delta t) = q(t_0) \otimes \left[\cos\frac{\|\omega\|\Delta t}{2},\ \sin\frac{\|\omega\|\Delta t}{2}\,\frac{\omega}{\|\omega\|}\right]$. To derive (4.18), compose $q(t + \mathrm{d}t) = q(t) \otimes \mathrm{d}q$, where $\mathrm{d}q$ is the small body-frame rotation by $\omega_b\,\mathrm{d}t$. By (4.12), $\mathrm{d}q = [1,\ \frac{1}{2}\omega_b\,\mathrm{d}t] + O(\mathrm{d}t^2)$, so $q(t + \mathrm{d}t) - q(t) = q \otimes [0, \frac{1}{2}\omega_b]\,\mathrm{d}t$. The factor $\frac{1}{2}$ is the half angle again.

4.7.3 Derivatives of vectors in a rotating frame

Differentiating $v_w = R_{wb}v_b$ with (4.17) gives the *transport theorem*

$$\dot{v}_w = R_{wb}\left(\dot{v}_b + \omega_b \times v_b\right). \tag{4.19}$$

It is the source of Coriolis and centripetal terms when dynamics are written in a rotating frame. Our estimator keeps position and velocity in the (approximately inertial) world frame, so these terms do not appear in the state equations.

***Remark* 4.6** (Earth rotation). The world frame is fixed to the Earth, which rotates at 7.29×10^{-5} rad/s. A gyroscope senses this rate on top of the vehicle's motion. For the ADIS16448 in EuRoC this is comparable to the bias instability and well below the noise floor over short horizons, so the project neglects it. The static part of the Earth rate, in the body frame, is partly absorbed by the gyro bias estimate. High-grade inertial navigation systems model it explicitly.

Key takeaways

- R_{wb} maps body coordinates to world coordinates; its columns are the body axes in the world. Adjacent subscripts must match.

- Every rotation is an axis-angle rotation, $R = I + \sin\theta[u]_\times + (1-\cos\theta)[u]_\times^2$.
- Three-parameter representations have singularities. Euler angles stay out of the estimator.
- Hamilton unit quaternions compose like matrices. q and $-q$ are the same rotation, and the quaternion of a rotation by θ uses $\theta/2$.
- Eigen's constructor is w-first, its storage w-last, EuRoC is w-first.
- $\dot{R}_{wb} = R_{wb}[\omega_b]_\times$. With constant ω_b the update $R \leftarrow R\exp(\Delta t[\omega_b]_\times)$ is exact.

Exercises

4.1 Show that the columns of any $R \in \mathrm{SO}(3)$ are orthonormal and that $\det R = +1$ implies the third column is the cross product of the first two.

4.2 Verify $R_x(90°)R_y(90°) \neq R_y(90°)R_x(90°)$ by computing both products, and describe each rotation physically.

4.3 Derive (4.7) from $R = R_z(\psi)R_y(\vartheta)R_x(\varphi)$ and $[\omega_b]_\times = R^\mathsf{T}\dot{R}$.

4.4 Prove that $[p]_L[q]_R = [q]_R[p]_L$. What does this say about associativity of the quaternion product?

4.5 Show that $R(-q) = R(q)$ and compute the quaternion of a 270° rotation about z in its canonical form ($w \geq 0$). What rotation angle and axis does the canonical form describe?

4.6 Write a function that converts a EuRoC ground-truth row into p_{wb}, q_{wb}, v_{wb}, b_g and b_a. Design a fixture row whose values would expose a swapped w/z and swapped biases.

4.7 Given T_{bc} from `cam0/sensor.yaml` (Appendix D), compute the camera's optical axis in body coordinates. Which way does the camera point relative to the IMU?

4.8 Show that if ω_b varies in time, $R(t) = R(0)\exp\left(\int_0^t [\omega_b]_\times \,\mathrm{d}s\right)$ is in general *wrong*. Construct a counterexample with piecewise-constant rates (this is the "coning" error of Chapter 9).

Chapter 5

Lie Theory for Rotations

In this chapter

- Understand SO(3) as a curved manifold with a flat tangent space, and why an estimator needs both.
- Use the exponential and logarithm maps, in matrix and quaternion form, and implement them robustly.
- Distinguish local (right) and global (left) perturbations, and move perturbations across a rotation with the adjoint.
- Define Jacobians of functions of rotations, derive the right Jacobian J_r, and build a table of elementary Jacobians from which every Jacobian in the project can be assembled.
- Represent and propagate uncertainty on SO(3), and test manifold Jacobians numerically.

This chapter is the mathematical core of the notes. Its conclusions fit in a few lines (the $\boxplus/\boxminus$ operators of Section 5.8 and the Jacobian table of Table 5.1), but the reasoning behind them is what lets you derive any new Jacobian correctly. The treatment follows the "micro Lie theory" of Solà et al. [45], specialized to SO(3) with the conventions of this project. Barfoot [3] covers the same material in greater depth.

5.1 Why estimation needs Lie theory

A Kalman filter represents uncertainty as a mean and a covariance and corrects the mean by adding a vector. Both operations assume the state lives in a vector space. Attitude does not.

- **Addition leaves the space.** If $\hat{q}$ is a unit quaternion and δq a small 4-vector, $\hat{q} + \delta q$ is generally not unit length. The same holds for rotation matrices.
- **Redundant parameters give singular covariances.** A unit quaternion has four components but three degrees of freedom. A 4×4 covariance of a unit quaternion must be singular along the radial direction, and keeping it exactly singular under rounding is hopeless.
- **Minimal parameters have singularities.** Euler angles (Chapter 4) are minimal but singular at gimbal lock.

Key idea: the Earth and its tangent planes

The surface of the Earth is curved, yet a town map drawn on flat paper is perfectly accurate for a walk across town. The map is the *tangent plane* at your location. It works

as long as you do not walk too far, and when you move to a new town you draw a new map. An estimator does the same: it stores the attitude on the curved space SO(3) (the globe), and expresses small uncertainties and corrections in the flat 3D tangent space at the current estimate (the town map). The exponential map turns a step on the map into a point on the globe, and the logarithm map does the reverse.

5.2 Groups, manifolds and Lie groups

Definition 5.1 (Group). A *group* $(\mathcal{G}, \circ)$ is a set with an associative operation $\circ$, an identity element $\mathcal{E}$, and an inverse $\mathcal{X}^{-1}$ for every element.

Definition 5.2 (Lie group). A *Lie group* is a group that is also a smooth manifold, such that composition and inversion are smooth maps. A *matrix Lie group* is a Lie group whose elements are invertible matrices and whose operation is matrix multiplication.

A smooth n-dimensional manifold is a space that looks like $\mathbb{R}^n$ near every point: a sphere is a 2D manifold, SO(3) a 3D manifold. Being a group as well gives a Lie group a remarkable property. The neighborhood of *any* element looks exactly like the neighborhood of the identity, carried over by composition. All local calculus can therefore be done once, at the identity, and transported.

Group	Elements	Dimension	Role in VIO
$(\mathbb{R}^n, +)$	vectors	n	position, velocity, biases
SO(2)	planar rotations	1	toy examples
SO(3)	3×3 rotation matrices	3	attitude
S^3	unit quaternions	3	attitude (double cover of SO(3))
SE(3)	rigid transformations	6	poses, extrinsics

5.3 The Lie algebra and tangent spaces

Consider a smooth curve $R(t) \in \mathrm{SO}(3)$ with $R(0) = I$. Differentiating $R^\mathsf{T} R = I$ at $t = 0$ gives $\dot{R}(0)^\mathsf{T} + \dot{R}(0) = 0$: the velocity at the identity is skew-symmetric. The set of all such velocities is the *Lie algebra*

$$\mathfrak{so}(3) = \{\, \Omega \in \mathbb{R}^{3\times 3} : \Omega^\mathsf{T} = -\Omega \,\} = \{\, [\phi]_\times : \phi \in \mathbb{R}^3 \,\}, \tag{5.1}$$

a three-dimensional vector space identified with $\mathbb{R}^3$ by the hat and vee maps, $\phi^\wedge = [\phi]_\times$ and $[\phi]_\times^\vee = \phi$. Unlike SO(3) itself, the Lie algebra is flat: we can add its elements, scale them, and put Gaussian distributions on them.

At a general point R, the tangent space consists of the velocities of curves through R. From (4.17), every such velocity can be written in two ways,

$$\dot{R} = R\,[\omega_b]_\times = [\omega_w]_\times\, R, \qquad \omega_w = R\,\omega_b, \tag{5.2}$$

so the tangent space at R can be identified with $\mathbb{R}^3$ in two ways: by a vector expressed in the *local* (body) frame, multiplying on the right, or by a vector expressed in the *global* (world)

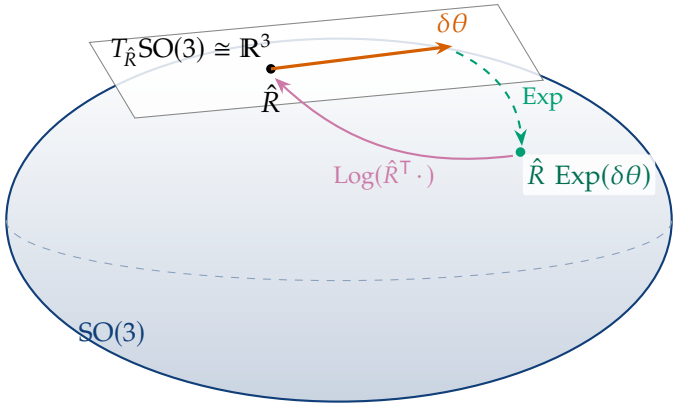


Figure 5.1: The manifold SO(3) and its tangent space at the estimate $\hat{R}$. A small error or correction $\delta\theta \in \mathbb{R}^3$ lives in the flat tangent space. The exponential map wraps it onto the manifold as $\hat{R}\,\mathrm{Exp}(\delta\theta)$, and the logarithm recovers it.

frame, multiplying on the left. This is the origin of the two perturbation conventions of Section 5.8. The project uses the local one.

5.4 The exponential map

Definition 5.3 (Exponential map of SO(3))**.** $\exp : \mathfrak{so}(3) \to \mathrm{SO}(3)$ is the matrix exponential. Composed with the hat map it gives the *capitalized* exponential

$$\mathrm{Exp} : \mathbb{R}^3 \to \mathrm{SO}(3), \qquad \mathrm{Exp}(\phi) = \exp([\phi]_\times). \tag{5.3}$$

By Rodrigues' formula (Theorem 4.3), with $\theta = \|\phi\|$,

$$\mathrm{Exp}(\phi) = I + \frac{\sin\theta}{\theta}\,[\phi]_\times + \frac{1-\cos\theta}{\theta^2}\,[\phi]_\times^2. \tag{5.4}$$

$\mathrm{Exp}(\phi)$ is the rotation by angle $\|\phi\|$ about the axis $\phi/\|\phi\|$. Its main properties are $\mathrm{Exp}(0) = I$, $\mathrm{Exp}(-\phi) = \mathrm{Exp}(\phi)^\mathsf{T}$, and $\mathrm{Exp}(s\phi)\,\mathrm{Exp}(t\phi) = \mathrm{Exp}((s+t)\phi)$: rotations about a *common* axis compose by adding angles. Exp is onto SO(3), and one-to-one on the open ball $\|\phi\| < \pi$.

Quaternion exponential. The unit quaternion corresponding to $\mathrm{Exp}(\phi)$ follows from (4.12):

$$\mathrm{Exp}_q(\phi) = \begin{bmatrix} \cos(\theta/2) \\ \dfrac{\sin(\theta/2)}{\theta}\,\phi \end{bmatrix}, \qquad R\big(\mathrm{Exp}_q(\phi)\big) = \mathrm{Exp}(\phi). \tag{5.5}$$

This is the quaternion exponential of the pure quaternion $[0, \phi/2]$; the half angle is the double cover again. Since $\mathrm{Exp}_q(\phi)$ has $w = \cos(\theta/2) \geq 0$ for $\theta \leq \pi$, its output is already in canonical form on that range.

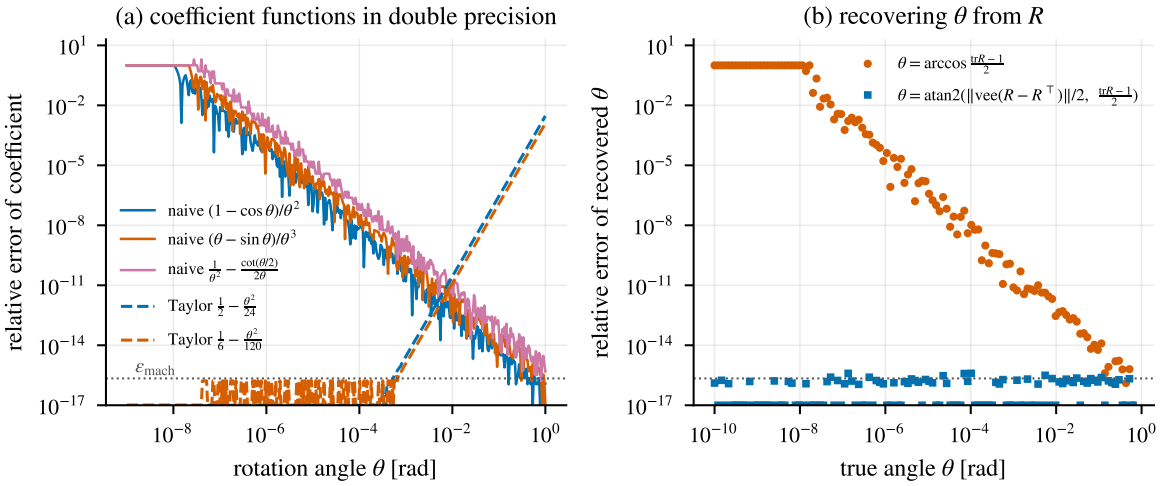


Figure 5.2: (a) Relative error of the coefficient functions evaluated naively in double precision (solid) and with a two-term Taylor series (dashed), against a 60-digit reference. The naive error grows like u/θ^2 as $\theta \to 0$, and the truncation error of the series grows like θ^4. (b) Recovering the angle from a rotation matrix: arccos loses accuracy like u/θ^2 and returns exactly zero below 10^{-8}, while the atan2 form is accurate to machine precision at every angle.

5.4.1 Numerical evaluation and the Taylor branches

Formulas (5.4) and (5.5), and the Jacobian formulas later in this chapter, are built from a handful of scalar *coefficient functions* of θ:

$$A(\theta) = \frac{\sin\theta}{\theta}, \quad B(\theta) = \frac{1-\cos\theta}{\theta^2}, \quad C(\theta) = \frac{\theta - \sin\theta}{\theta^3}, \quad D(\theta) = \frac{1}{\theta^2} - \frac{\cot(\theta/2)}{2\theta}. \tag{5.6}$$

All four are smooth, even functions with finite limits at $\theta = 0$, but their formulas read $0/0$ there. Their Taylor series are

$$\begin{aligned} A &= 1 - \tfrac{\theta^2}{6} + \tfrac{\theta^4}{120} - \cdots, & B &= \tfrac{1}{2} - \tfrac{\theta^2}{24} + \tfrac{\theta^4}{720} - \cdots, \\ C &= \tfrac{1}{6} - \tfrac{\theta^2}{120} + \tfrac{\theta^4}{5040} - \cdots, & D &= \tfrac{1}{12} + \tfrac{\theta^2}{720} + \tfrac{\theta^4}{30240} + \cdots, \end{aligned} \tag{5.7}$$

and for the quaternion map $\cos\frac{\theta}{2} = 1 - \frac{\theta^2}{8} + \frac{\theta^4}{384} - \cdots$ and $\frac{\sin(\theta/2)}{\theta} = \frac{1}{2} - \frac{\theta^2}{48} + \frac{\theta^4}{3840} - \cdots$.

There are two distinct numerical issues.

1. **Division by zero.** At $\theta = 0$ exactly (a perfectly still IMU, or a zero correction), every formula evaluates to `NaN`, and a single `NaN` propagates through P and destroys the filter. For tiny θ, θ^2 can also underflow to zero.
2. **Cancellation.** B, C and D subtract nearly equal quantities. As Figure 5.2(a) shows, their naive evaluation has relative error of order u/θ^2: about 10^{-8} at $\theta = 10^{-4}$, and complete loss below $\theta \approx 10^{-8}$. A and the quaternion coefficients involve no subtraction and are accurate everywhere except at $0/0$.

How much does the cancellation matter? It depends on what the coefficient multiplies. A coefficient with absolute error ϵ_c that multiplies $[\phi]_\times^k$ contributes an error of about $\epsilon_c \theta^k$ to the matrix. For B, C and D, $\epsilon_c \approx u/\theta^2$, so:

Function	cancelling coefficient	multiplies	naive absolute error
$\mathrm{Exp}(\phi)$	B	$[\phi]_\times^2$	$\approx u$ (harmless)
$J_r(\phi)$	B	$[\phi]_\times$	$\approx u/\theta$ (**harmful**: 3×10^{-9} at $\theta = 10^{-8}$)
$J_r(\phi)$	C	$[\phi]_\times^2$	$\approx u$
$J_r^{-1}(\phi)$	D	$[\phi]_\times^2$	$\approx u$

Key idea: why the Taylor branch exists, and how to choose its threshold

Every function needs a guard against $0/0$. Beyond that, the branch matters for accuracy wherever a cancelling coefficient multiplies a low power of $[\phi]_\times$, as in J_r. To choose the threshold θ_0 and the number of terms, balance the two error curves of Figure 5.2(a): keeping terms through θ^{2m} leaves a truncation error of order θ^{2m+2}, and the exact formula has a rounding error of order u/θ^2. Below the crossing point use the series, above it the formula. For two-term series ($m = 1$) the curves cross near $\theta \approx 3\times10^{-3}$, where both relative errors are about 10^{-12}; adding the θ^4 term moves the crossing to larger angles and lowers the error there. Remember to multiply by θ^k when translating a coefficient error into a matrix error. Record the choice and this justification next to the code (Milestone 2).

5.5 The logarithm map

Definition 5.4 (Logarithm map). $\mathrm{Log} : \mathrm{SO}(3) \to \mathbb{R}^3$ returns the rotation vector ϕ with $\|\phi\| \le \pi$ such that $\mathrm{Exp}(\phi) = R$.

For angles strictly between 0 and π the answer is unique. Combining the two equations of (4.5),

$$\theta = \mathrm{atan2}\Big(\tfrac{1}{2}\big\|(R - R^\mathsf{T})^\vee\big\|,\ \tfrac{1}{2}(\mathrm{tr}\,R - 1)\Big), \qquad \mathrm{Log}(R) = \frac{\theta}{2\sin\theta}(R - R^\mathsf{T})^\vee. \tag{5.8}$$

The atan2 form is essential. The common textbook formula $\theta = \arccos\big((\mathrm{tr}\,R - 1)/2\big)$ is correct mathematically but numerically unstable near $\theta = 0$, where arccos has infinite slope, and loses half the significant digits (Figure 5.2b). Three regimes need care in code.

1. **Small angles.** $\theta/(2\sin\theta) \to \frac{1}{2}$, so use $\mathrm{Log}(R) \approx \frac{1}{2}(1 + \frac{\theta^2}{6})(R - R^\mathsf{T})^\vee$.
2. **Generic angles.** Use (5.8) directly.
3. **Angles near π.** $\sin\theta \to 0$ and $(R - R^\mathsf{T})^\vee = 2\sin\theta\, u$ vanishes, so the axis cannot be recovered from the skew-symmetric part. Use the symmetric part instead: from (4.4), $\frac{1}{2}(R + R^\mathsf{T}) = \cos\theta\, I + (1 - \cos\theta)\, uu^\mathsf{T}$, so $uu^\mathsf{T} = \big(\frac{1}{2}(R + R^\mathsf{T}) - \cos\theta\, I\big)/(1 - \cos\theta)$. Take the column with the largest diagonal entry, normalize it to get $\pm u$, and fix the sign so that $u^\mathsf{T}(R - R^\mathsf{T})^\vee \ge 0$.

***Remark* 5.5** (why Log is ill-conditioned near π). Near $\theta = \pi$ two different effects combine. Algebraically, the axis must be extracted from a nearly vanishing skew-symmetric part, so the naïve formula amplifies rounding errors by $1/\sin\theta$. Topologically, Log is discontinuous at $\theta = \pi$ (Figure 5.3): a rotation by $\pi - \epsilon$ about u and one by $\pi - \epsilon$ about $-u$ are close in SO(3) but have Log values 2π apart. No implementation can remove the second effect. In an estimator it rarely matters, because Log is applied to *differences* between nearby rotations,

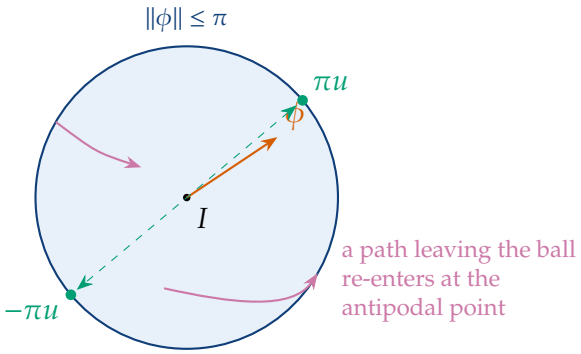


Figure 5.3: SO(3) pictured as the ball of rotation vectors of radius π, with antipodal points on the boundary identified (πu and $-\pi u$ are the same rotation). Log is smooth inside the ball but must jump when a rotation crosses $\theta = \pi$. Near the boundary a tiny change in R can flip the returned vector to the opposite side.

whose angles are small. Milestone 2 nevertheless tests the near-π regime, because a broken branch there produces `NaN`.

Quaternion logarithm. For a unit quaternion $q = [w, v]$, first canonicalize to $w \ge 0$ (replace q by $-q$ if needed), then

$$\operatorname{Log}_q(q) = 2\ \operatorname{atan2}(\|v\|, w)\,\frac{v}{\|v\|} \;\approx\; \frac{2}{w}\left(1 - \frac{\|v\|^2}{3w^2}\right) v \quad \text{for small } \|v\|. \tag{5.9}$$

Canonicalization guarantees $\|\operatorname{Log}_q(q)\| \le \pi$ and makes $\operatorname{Log}_q(q) = \operatorname{Log}_q(-q)$, so the double cover is invisible to the estimator. The atan2 is well-conditioned at all angles, including near π where $w \approx 0$: the quaternion logarithm does not suffer the algebraic ill-conditioning of the matrix version. The topological discontinuity at π remains.

5.6 The adjoint: moving a perturbation across a rotation

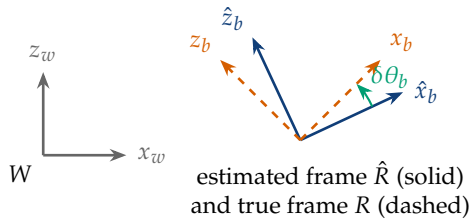
Proposition 5.6 (Adjoint of SO(3)). *For all $R \in \mathrm{SO}(3)$ and $\phi \in \mathbb{R}^3$,*

$$R\ \operatorname{Exp}(\phi)\,R^{\mathsf{T}} = \operatorname{Exp}(R\phi), \tag{5.10}$$

or equivalently $\operatorname{Exp}(\phi)\,R = R\ \operatorname{Exp}(R^{\mathsf{T}}\phi)$ and $R\ \operatorname{Exp}(\phi) = \operatorname{Exp}(R\phi)\,R$.

Proof. By (2.8), $R[\phi]_\times R^{\mathsf{T}} = [R\phi]_\times$. By property 4 of Proposition 2.15, $R\exp([\phi]_\times)R^{\mathsf{T}} = \exp(R[\phi]_\times R^{\mathsf{T}}) = \exp([R\phi]_\times)$. The equivalent forms follow by multiplying on the right or left by R or R^{T}. ∎

The map $\phi \mapsto R\phi$ is the *adjoint action* Ad_R, which for SO(3) is simply the rotation matrix itself. Geometrically, a rotation about an axis ϕ expressed in one frame equals the rotation about the same physical axis expressed in another frame. Equation (5.10) is the single most used identity in Jacobian derivations: it lets you slide a perturbation $\operatorname{Exp}(\delta)$ through other rotations until it reaches the position the convention requires.



Local: $R = \hat{R}\,\mathrm{Exp}(\delta\theta_b)$ **(this project)**
Global: $R = \mathrm{Exp}(\delta\theta_w)\hat{R}$
Same error: $\delta\theta_w = \hat{R}\,\delta\theta_b$
Covariances: $P_w = \hat{R}P_b\hat{R}^\mathsf{T}$

Figure 5.4: The local (right) perturbation measures the attitude error as a small rotation about the axes of the *estimated* body frame. The global (left) perturbation measures the same error about the world axes.

5.7 Composition and the Baker–Campbell–Hausdorff formula

Rotations about different axes do not commute, so $\mathrm{Exp}(a)\,\mathrm{Exp}(b) \neq \mathrm{Exp}(a+b)$ in general. The Baker–Campbell–Hausdorff (BCH) formula quantifies the discrepancy. For matrices X, Y near zero,

$$\log\big(\exp X \exp Y\big) = X + Y + \tfrac{1}{2}[X,Y] + \tfrac{1}{12}\big([X,[X,Y]] + [Y,[Y,X]]\big) + \cdots, \tag{5.11}$$

where $[X,Y] = XY - YX$. For $\mathfrak{so}(3)$, (2.7) gives $[[a]_\times,[b]_\times] = [a\times b]_\times$, so

$$\mathrm{Log}\big(\mathrm{Exp}(a)\,\mathrm{Exp}(b)\big) = a + b + \tfrac{1}{2}\,a\times b + \tfrac{1}{12}\big(a\times(a\times b) + b\times(b\times a)\big) + \cdots. \tag{5.12}$$

The first correction is second order. Two small rotations therefore compose approximately additively, $\mathrm{Exp}(a)\,\mathrm{Exp}(b) \approx \mathrm{Exp}(a+b)$, with an error of order $\|a\|\|b\|$. When only one of them is small, summing the BCH series in the large argument gives the two most important first-order formulas of this chapter:

$$\mathrm{Log}\big(\mathrm{Exp}(\phi)\;\mathrm{Exp}(\delta)\big) = \phi + J_r^{-1}(\phi)\,\delta + O(\|\delta\|^2), \tag{5.13}$$
$$\mathrm{Log}\big(\mathrm{Exp}(\delta)\;\mathrm{Exp}(\phi)\big) = \phi + J_l^{-1}(\phi)\,\delta + O(\|\delta\|^2), \tag{5.14}$$

with the Jacobians J_r and J_l defined in Section 5.9.

5.8 Perturbations, $\boxplus$ and $\boxminus$

Let $\hat{R}$ be an estimate and R the truth. There are two natural ways to write the small rotation separating them:

$$\underbrace{R = \hat{R}\;\mathrm{Exp}(\delta\theta_b)}_{\text{local (right) perturbation}} \qquad \underbrace{R = \mathrm{Exp}(\delta\theta_w)\,\hat{R}}_{\text{global (left) perturbation}}. \tag{5.15}$$

By Proposition 5.6 they describe the same error, related by $\delta\theta_w = \hat{R}\,\delta\theta_b$. $\delta\theta_b$ is the error expressed in the estimated body frame, $\delta\theta_w$ the same error expressed in the world frame. If $\delta\theta_b \sim \mathcal{N}(0, P_b)$, then $\delta\theta_w \sim \mathcal{N}(0, \hat{R}P_b\hat{R}^\mathsf{T})$.

Both conventions are mathematically valid and widely used, but they are not interchangeable. Every Jacobian, covariance and residual depends on the choice. The project uses the local one for three reasons.

1. **It matches the sensor.** The gyroscope measures ω_b in the body frame and (4.17) puts it on the right, so body-frame errors couple to body-frame quantities.
2. **It is decoupled from global yaw.** The local attitude error dynamics (Chapter 10) depend on $\hat{\omega}$ and the biases but not on the absolute heading.
3. **It is the convention of the standard ESKF reference [44],** which makes the literature easy to cross-check.

Frozen convention: retraction and local difference, AGENTS.md C4

For the full state $x = (p, v, q, b_a, b_g, \{p_i, q_i\})$ and $\delta x \in \mathbb{R}^{15+6N}$:

$$x = \hat{x} \boxplus \delta x: \quad p = \hat{p} + \delta p, \quad v = \hat{v} + \delta v, \quad q = \hat{q} \otimes \mathrm{Exp}_q(\delta\theta), \quad b = \hat{b} + \delta b,$$
$$\delta x = x \boxminus \hat{x}: \quad \delta\theta = \mathrm{Log}_q(\hat{q}^{-1} \otimes q), \quad \text{all other blocks subtract.}$$

Clone blocks follow the same rule. Every residual, Jacobian, injection and numerical-differentiation test uses these two operators and nothing else.

The operators satisfy the axioms that make them a consistent local parameterization:

$$x \boxplus 0 = x, \qquad (x \boxplus \delta) \boxminus x = \delta, \qquad x \boxplus (y \boxminus x) = y, \tag{5.16}$$

the second for $\|\delta\theta\| < \pi$ and the third for any y. The state is a *product manifold* $\mathbb{R}^3 \times \mathbb{R}^3 \times \mathrm{SO}(3) \times \mathbb{R}^3 \times \mathbb{R}^3 \times (\mathbb{R}^3 \times \mathrm{SO}(3))^N$. Each factor has its own $\boxplus$, and the composite operator acts block-wise.

5.9 Jacobians on manifolds

Definition 5.7 (Jacobian with respect to local perturbations)**.** Let $f : \mathcal{M} \to \mathcal{N}$ map between manifolds with $\boxplus/\boxminus$ operators. Its Jacobian at x is the matrix

$$J_x^f = \frac{\partial}{\partial \delta}\Big[f(x \boxplus \delta) \boxminus f(x)\Big]_{\delta=0}, \qquad \text{so that} \qquad f(x \boxplus \delta) = f(x) \boxplus \big(J_x^f\, \delta + O(\|\delta\|^2)\big). \tag{5.17}$$

When both spaces are vector spaces this is the ordinary Jacobian. The chain rule holds unchanged, $J_x^{g\circ f} = J_{f(x)}^g J_x^f$, so complicated Jacobians are products of simple ones. The column j of J_x^f is what a finite-difference test computes by perturbing direction j, which is how every such Jacobian is verified.

5.9.1 The right and left Jacobians of SO(3)

Definition 5.8 (Right and left Jacobians)**.** The *right Jacobian* $J_r(\phi)$ is the Jacobian of $\mathrm{Exp} : \mathbb{R}^3 \to \mathrm{SO}(3)$ with the local $\boxminus$ on SO(3); the *left Jacobian* $J_l(\phi)$ uses the global one:

$$\mathrm{Exp}(\phi + \delta) = \mathrm{Exp}(\phi)\ \mathrm{Exp}\big(J_r(\phi)\,\delta\big) + O(\|\delta\|^2) = \mathrm{Exp}\big(J_l(\phi)\,\delta\big)\ \mathrm{Exp}(\phi) + O(\|\delta\|^2). \tag{5.18}$$

$J_r(\phi)$ answers the question: if I change the rotation *vector* ϕ by δ, what small *body-frame* rotation does the rotation *matrix* undergo? Only for $\phi = 0$ is the answer δ itself.

Function	Jacobian w.r.t. local perturbation	Note
$f(R) = Rv$	$\dfrac{\partial f}{\partial \delta\theta} = -R[v]_\times$	rotate a body vector to world
$f(R) = R^{\mathsf{T}}v$	$\dfrac{\partial f}{\partial \delta\theta} = [R^{\mathsf{T}}v]_\times$	express a world vector in body
$f(R) = R^{-1}$	$J = -R$	output on SO(3) (local)
$f(R_1, R_2) = R_1 R_2$	$J_{R_1} = R_2^{\mathsf{T}}, \quad J_{R_2} = I$	output on SO(3) (local)
$f(\phi) = \mathrm{Exp}(\phi)$	$J = J_r(\phi)$	vector to SO(3)
$f(R) = \mathrm{Log}(R)$	$J = J_r^{-1}(\mathrm{Log}\, R)$	SO(3) to vector
$f(v) = Rv$	$J = R$	ordinary linear map

Table 5.1: Elementary Jacobians with respect to the local (right) perturbation $R \to R\,\mathrm{Exp}(\delta\theta)$. All other Jacobians in these notes are products of these by the chain rule. Each entry has been checked numerically by central differences.

Theorem 5.9 (Closed forms). *With $\theta = \|\phi\|$,*

$$J_r(\phi) = \int_0^1 \mathrm{Exp}(-s\phi)\,\mathrm{d}s = I - \frac{1-\cos\theta}{\theta^2}[\phi]_\times + \frac{\theta - \sin\theta}{\theta^3}[\phi]_\times^2, \tag{5.19}$$

$$J_r^{-1}(\phi) = I + \tfrac{1}{2}[\phi]_\times + \Big(\frac{1}{\theta^2} - \frac{1+\cos\theta}{2\theta\sin\theta}\Big)[\phi]_\times^2, \tag{5.20}$$

$$J_l(\phi) = J_r(-\phi) = J_r(\phi)^{\mathsf{T}} = \mathrm{Exp}(\phi)\,J_r(\phi), \qquad J_l^{-1}(\phi) = J_r^{-1}(-\phi). \tag{5.21}$$

The coefficient in (5.20) equals $D(\theta) = \frac{1}{\theta^2} - \frac{\cot(\theta/2)}{2\theta}$, which is finite at $\theta = \pi$ (where $D = 1/\pi^2$) and singular only at $\theta = 2\pi$.

Proof. The derivative of the matrix exponential in a direction Y is $\frac{\mathrm{d}}{\mathrm{d}\epsilon}\exp(X+\epsilon Y)\big|_0 = \int_0^1 \exp(sX)\,Y\exp((1-s)X)\,\mathrm{d}s$ (differentiate $\exp(X+\epsilon Y) = \lim(I + (X+\epsilon Y)/n)^n$, or verify that both sides satisfy the same linear ODE). With $X = [\phi]_\times$ and $Y = [\delta]_\times$,

$$\mathrm{Exp}(\phi)^{-1}\frac{\mathrm{d}}{\mathrm{d}\epsilon}\mathrm{Exp}(\phi+\epsilon\delta)\Big|_0 = \int_0^1 \mathrm{Exp}(-s\phi)\,[\delta]_\times\,\mathrm{Exp}(s\phi)\,\mathrm{d}s = \Big[\int_0^1 \mathrm{Exp}(-s\phi)\,\mathrm{d}s\;\delta\Big]_\times,$$

after substituting $s \to 1-s$ and using $R[\delta]_\times R^{\mathsf{T}} = [R\delta]_\times$. Comparing with $\mathrm{Exp}(\phi)\,\mathrm{Exp}(J_r\delta\,\epsilon) = \mathrm{Exp}(\phi)(I + \epsilon[J_r\delta]_\times) + O(\epsilon^2)$ gives the integral form. Inserting Rodrigues' formula for $\mathrm{Exp}(-s\phi)$ and integrating term by term gives the closed form. For the inverse, verify $J_r^{-1}J_r = I$ using $[\phi]_\times^3 = -\theta^2[\phi]_\times$ and $[\phi]_\times^4 = -\theta^2[\phi]_\times^2$, and use $\frac{1+\cos\theta}{\sin\theta} = \cot\frac{\theta}{2}$. The relations (5.21) follow from the definitions and Proposition 5.6. ∎

The small-angle expansions are $J_r(\phi) = I - \frac{1}{2}[\phi]_\times + \frac{1}{6}[\phi]_\times^2 + O(\theta^3)$ and $J_r^{-1}(\phi) = I + \frac{1}{2}[\phi]_\times + \frac{1}{12}[\phi]_\times^2 + O(\theta^4)$, with the coefficient series of (5.7). Equations (5.13) and (5.14) follow from (5.18) by inverting the relation.

5.9.2 Elementary Jacobians

The entries of Table 5.1 are derived in two or three lines each. The method is always the same: **perturb, expand to first order, isolate the perturbation, and read off the coefficient.**

Example 5.10 (Rotation action). For $f(R) = Rv$: $f(R\,\mathrm{Exp}(\delta\theta)) = R(I+[\delta\theta]_\times)v+O(\|\delta\theta\|^2) = Rv + R[\delta\theta]_\times v$. By (2.2), $[\delta\theta]_\times v = -[v]_\times\delta\theta$, so the Jacobian is $-R[v]_\times$.

Example 5.11 (Inverse action). For $f(R) = R^\mathsf{T}v$: $(R\,\mathrm{Exp}(\delta\theta))^\mathsf{T}v = \mathrm{Exp}(-\delta\theta)R^\mathsf{T}v \approx (I - [\delta\theta]_\times)R^\mathsf{T}v = R^\mathsf{T}v + [R^\mathsf{T}v]_\times\,\delta\theta$.

Example 5.12 (Inverse and composition). For $f(R) = R^{-1}$: $(R\,\mathrm{Exp}(\delta\theta))^{-1} = \mathrm{Exp}(-\delta\theta)R^\mathsf{T} = R^\mathsf{T}\,\mathrm{Exp}(-R\delta\theta)$ by (5.10), so $f(R)^{-1}f(R \boxplus \delta\theta) = \mathrm{Exp}(-R\delta\theta)$ and the Jacobian is $-R$. For $f = R_1R_2$ perturbed in R_1: $R_1\,\mathrm{Exp}(\delta\theta)R_2 = R_1R_2\,\mathrm{Exp}(R_2^\mathsf{T}\delta\theta)$, so $J_{R_1} = R_2^\mathsf{T}$. Perturbing R_2 appends $\mathrm{Exp}(\delta\theta)$ directly, so $J_{R_2} = I$.

Example 5.13 (Logarithm). For $f(R) = \mathrm{Log}(R)$ with $\phi = \mathrm{Log}\,R$: $\mathrm{Log}(R\,\mathrm{Exp}(\delta\theta)) = \mathrm{Log}(\mathrm{Exp}(\phi)\,\mathrm{Exp}(\delta\theta)) \approx \phi + J_r^{-1}(\phi)\delta\theta$ by (5.13).

Pitfall: first-order expansions and products

The expansion $\mathrm{Exp}(\delta\theta) \approx I + [\delta\theta]_\times$ is valid only when $\delta\theta$ is the *only* small quantity being differentiated. Products of two small quantities ($[\delta\theta]_\times\delta b$, or $[\delta\theta]_\times n$ with noise n) are second order and are dropped. Large quantities such as the nominal rotation, the angular rate or the specific force are never approximated.

5.10 Uncertainty on SO(3)

With the local perturbation, an uncertain attitude is the random rotation

$$R = \hat{R}\;\mathrm{Exp}(\delta\theta), \qquad \delta\theta \sim \mathcal{N}(0, P_\theta), \tag{5.22}$$

a "Gaussian on the manifold": the mean $\hat{R}$ lives on SO(3) and the covariance P_θ lives in the tangent space at $\hat{R}$, in body-frame coordinates. Its eigenvectors are body axes about which the attitude is most or least certain. This distribution is not a Gaussian in any fixed coordinates of SO(3), which is exactly why it captures curved distributions such as the banana of Figure 3.1 better than a Gaussian on Euler angles or matrix entries.

Propagation. If $y = f(R)$ and J is its manifold Jacobian, then to first order $y = f(\hat{R}) \boxplus \delta y$ with $\delta y \approx J\delta\theta \sim \mathcal{N}(0, JP_\theta J^\mathsf{T})$.

Compounding. For independent uncertain rotations $R_1 = \hat{R}_1\,\mathrm{Exp}(\delta_1)$ and $R_2 = \hat{R}_2\,\mathrm{Exp}(\delta_2)$, the composition is $R_1R_2 = \hat{R}_1\hat{R}_2\,\mathrm{Exp}(\hat{R}_2^\mathsf{T}\delta_1)\,\mathrm{Exp}(\delta_2) \approx \hat{R}_1\hat{R}_2\,\mathrm{Exp}(\hat{R}_2^\mathsf{T}\delta_1 + \delta_2)$, so

$$P_{12} = \hat{R}_2^\mathsf{T}P_1\hat{R}_2 + P_2. \tag{5.23}$$

***Remark* 5.14** (covariance entries rotate between axes). Because P_θ is expressed in body axes, it changes when the body rotates even if no information is gained or lost. Suppose the world yaw is uncertain and roll and pitch are perfectly known: in world coordinates $P_w = \sigma^2 e_z e_z^\mathsf{T}$. In body coordinates $P_b = \hat{R}^\mathsf{T}P_w\hat{R} = \sigma^2(\hat{R}^\mathsf{T}e_z)(\hat{R}^\mathsf{T}e_z)^\mathsf{T}$. If the vehicle rolls by 90°, the world vertical is the body y-axis, and the yaw uncertainty moves from the (z, z) to the (y, y) entry of P. Individual diagonal entries of the attitude covariance are therefore not monotonic during propagation (a Milestone 5 concept), although its trace and eigenvalues behave as expected.

5.11 SE(3) and the choice of state manifold

The rigid transformations of Chapter 4 also form a Lie group, SE(3), with $\operatorname{Exp}(\rho, \phi) = \begin{bmatrix} \operatorname{Exp}(\phi) & J_l(\phi)\rho \\ 0 & 1 \end{bmatrix}$. Perturbing a pose on SE(3) couples the translation error to the rotation error. The project does *not* do this: its state is the product manifold of Section 5.8, in which the position error is a plain world-frame vector and the attitude error is a local rotation, independently. Both choices are valid, but they produce different Jacobians and different covariances, and must never be mixed. A third choice, the extended pose group $\mathrm{SE}_2(3)$ for (R, v, p) together, underlies the *invariant* EKF, whose error dynamics have the striking property of not depending on the state at all (Chapter 20).

5.12 Verifying manifold Jacobians numerically

Every analytic Jacobian in the project is checked against a central difference built on the $\boxplus/\boxminus$ operators (AGENTS.md E6). Column j of the numerical Jacobian is

$$J_{:,j} \approx \frac{f(x \boxplus he_j) \boxminus f(x \boxplus (-he_j))}{2h}, \qquad h \approx 10^{-6}. \tag{5.24}$$

The central difference has truncation error $O(h^2)$ and rounding error $O(u/h)$ (Section 2.9), so analytic and numerical Jacobians should agree to about 10^{-8} relative to the magnitude of the entries.

Pitfall: tests that cannot fail

A Jacobian test evaluated at a trivial point proves little. At $R = I$, R and R^{T} are equal, so a transpose bug is invisible; with zero angular rate, terms multiplied by $\hat{\omega}$ vanish; with zero biases, bias-sign errors may hide. Always test at a random non-identity attitude, non-zero rates, accelerations and biases, and ask of every test: *would this still pass if I flipped the sign of X?* A good habit is to temporarily break the analytic Jacobian and confirm that the test fails.

Project link: Milestone 2

`so3.hpp` implements `skew`, `Exp/Log`, `Exp_q/Log_q`, `J_r` and `J_r_inv` as derived here, and `numerical_jacobian.hpp` implements (5.24) generically over $\boxplus/\boxminus$. The exit criteria test exactly the properties of this chapter: $\operatorname{Exp}(\phi) \in \mathrm{SO}(3)$, $\operatorname{Log} \circ \operatorname{Exp} = \mathrm{id}$ at zero, tiny, generic and near-π angles, agreement of Exp_q and Exp, $\operatorname{Log}_q(q) = \operatorname{Log}_q(-q)$, the defining relation (5.18) of J_r, and $J_r^{-1} J_r = I$, all without allocation.

Key takeaways

- Attitude lives on the curved manifold SO(3); its errors live in the flat tangent space $\mathbb{R}^3$. Exp and Log connect them.
- Compute angles with atan2, never with arccos. Guard every coefficient function against 0/0, and use Taylor branches wherever cancellation is amplified (notably

in J_r).

- Local (right) perturbation: $R = \hat{R}\,\mathrm{Exp}(\delta\theta)$, error in body axes. Global errors are $\hat{R}\,\delta\theta$.
- The adjoint identity $R\,\mathrm{Exp}(\phi)R^{\mathsf{T}} = \mathrm{Exp}(R\phi)$ moves perturbations through rotations.
- $\mathrm{Exp}(\phi+\delta) \approx \mathrm{Exp}(\phi)\,\mathrm{Exp}(J_r(\phi)\delta)$ and $\mathrm{Log}(\mathrm{Exp}(\phi)\,\mathrm{Exp}(\delta)) \approx \phi + J_r^{-1}(\phi)\delta$.
- Every project Jacobian is a chain-rule product of the entries of Table 5.1, and is verified by central differences at a non-trivial point.

Exercises

5.1 Show that $\mathfrak{so}(3)$ is closed under the commutator $[X, Y] = XY - YX$ and that $[[a]_\times, [b]_\times] = [a \times b]_\times$.

5.2 Prove $\mathrm{Exp}(-\phi) = \mathrm{Exp}(\phi)^{\mathsf{T}}$ and $\mathrm{Exp}(s\phi)\,\mathrm{Exp}(t\phi) = \mathrm{Exp}((s+t)\phi)$. Why does the second identity not contradict non-commutativity?

5.3 Implement Log with both arccos and atan2 and reproduce Figure 5.2(b). Then construct a rotation within 10^{-6} rad of π and test the near-π branch.

5.4 Using the coefficient series of (5.7), choose a threshold and a number of terms for J_r so that its absolute error stays below 10^{-14} for all $\theta \in [0, 0.1]$. Justify your choice with the error model of Section 5.4.1.

5.5 Derive the Jacobian of $f(R) = \mathrm{Log}(\hat{R}^{\mathsf{T}}R)$ with respect to a local perturbation of R, and of $g(\hat{R}) = \mathrm{Log}(\hat{R}^{\mathsf{T}}R)$ with respect to a local perturbation of $\hat{R}$. Evaluate both at $R = \hat{R}$. (These are the building blocks of attitude residuals.)

5.6 Show that $J_l(\phi) = \mathrm{Exp}(\phi)J_r(\phi)$ and use it to prove (5.14) from (5.13).

5.7 A vehicle has attitude covariance $P_b = \mathrm{diag}(\sigma_r^2, \sigma_r^2, \sigma_y^2)$ in body axes while level. Compute P_b after it pitches by 90° with no new information, and interpret.

5.8 Verify every row of Table 5.1 with a 20-line Python script using central differences at a random point. Then flip one sign and confirm your script detects it.

Part II

Estimation Theory

Chapter 6

The Kalman Filter

In this chapter

- Derive the Kalman filter from Bayes' rule and Gaussian conditioning, in a few lines.
- Understand the gain, the innovation and its covariance, and the several equivalent forms of the covariance update.
- Know why the Joseph form is used and how to implement the update without explicit inverses.
- Discretize a continuous-time linear stochastic system exactly and approximately, including Van Loan's method.

Rudolf Kálmán's 1960 paper [29] gave a recursive solution to linear estimation that needs only the current estimate and covariance. It flew on Apollo and remains the backbone of navigation. This chapter derives it for linear systems. Chapter 7 extends it to the nonlinear, manifold-valued state of VIO.

6.1 Linear-Gaussian state-space models

A discrete-time linear system with Gaussian noise is described by

$$x_k = \Phi_k\, x_{k-1} + w_k, \qquad w_k \sim \mathcal{N}(0, Q_k), \tag{6.1}$$

$$z_k = H_k\, x_k + n_k, \qquad n_k \sim \mathcal{N}(0, R_k), \tag{6.2}$$

with $x_0 \sim \mathcal{N}(\hat{x}_0, P_0)$, and with x_0, $\{w_k\}$ and $\{n_k\}$ mutually independent and white. Φ_k is the state transition matrix, Q_k the process noise covariance, H_k the measurement matrix and R_k the measurement noise covariance. The goal is the posterior $p(x_k \mid z_{1:k})$ given all measurements up to time k.

6.2 The Bayes filter

Because the state is Markov (given x_{k-1}, the past does not influence x_k) and each measurement depends only on the current state, the posterior can be computed recursively in two steps:

$$\textbf{predict:}\quad p(x_k \mid z_{1:k-1}) = \int p(x_k \mid x_{k-1})\, p(x_{k-1} \mid z_{1:k-1})\, \mathrm{d}x_{k-1}, \tag{6.3}$$

$$\textbf{update:}\quad p(x_k \mid z_{1:k}) \propto p(z_k \mid x_k)\, p(x_k \mid z_{1:k-1}). \tag{6.4}$$

For a linear-Gaussian model all of these densities are Gaussian, so each step reduces to updating a mean and a covariance. We write $\hat{x}_k^-, P_k^-$ for the predicted (prior) moments and $\hat{x}_k^+, P_k^+$ for the updated (posterior) moments.

6.3 Derivation

6.3.1 Prediction

By Proposition 3.1 applied to (6.1), with x_{k-1} and w_k independent,

$$\hat{x}_k^- = \Phi_k \hat{x}_{k-1}^+, \qquad P_k^- = \Phi_k P_{k-1}^+ \Phi_k^\mathsf{T} + Q_k. \tag{6.5}$$

Prediction never decreases uncertainty in the directions excited by Q_k: it moves the error with the dynamics and adds process noise.

6.3.2 Update

Before the measurement arrives, the state and the measurement are jointly Gaussian. From (6.2), dropping the time index,

$$\begin{bmatrix} x \\ z \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \hat{x}^- \\ H\hat{x}^- \end{bmatrix}, \begin{bmatrix} P^- & P^- H^\mathsf{T} \\ HP^- & HP^- H^\mathsf{T} + R \end{bmatrix} \right). \tag{6.6}$$

Conditioning on the observed z with Theorem 3.3 gives the update directly.

Theorem 6.1 (Kalman update)**.** *Define the* innovation *(residual)* $r = z - H\hat{x}^-$*, its covariance* $S = HP^-H^\mathsf{T} + R$*, and the* Kalman gain $K = P^-H^\mathsf{T}S^{-1}$*. Then*

$$\hat{x}^+ = \hat{x}^- + Kr, \qquad P^+ = P^- - KSK^\mathsf{T} = (I - KH)\,P^-. \tag{6.7}$$

Proof. Equation (3.4) with $\Sigma_{ab} = P^-H^\mathsf{T}$ and $\Sigma_{bb} = S$ gives $\hat{x}^+ = \hat{x}^- + P^-H^\mathsf{T}S^{-1}(z - H\hat{x}^-)$ and $P^+ = P^- - P^-H^\mathsf{T}S^{-1}HP^-$. Since $KSK^\mathsf{T} = P^-H^\mathsf{T}S^{-1}HP^- = KHP^-$, both covariance forms agree. ∎

Key idea: the gain is a ratio of uncertainties

For a scalar state measured directly ($H = 1$), $K = \frac{P^-}{P^- + R}$ and $P^+ = \frac{P^-R}{P^- + R}$. If the prior is much more uncertain than the sensor ($P^- \gg R$), $K \to 1$ and the estimate jumps to the measurement. If the sensor is much worse ($R \gg P^-$), $K \to 0$ and the measurement is nearly ignored. The posterior variance is always smaller than both. In general, K maps the innovation, which lives in measurement space, into a correction in state space, including corrections to states that are not measured directly but are correlated with measured ones through P^-. This is how a pose measurement corrects velocity and biases.

6.3.3 The information form

Applying the Woodbury identity (Theorem 2.13) to $P^+ = P^- - P^-H^\mathsf{T}(HP^-H^\mathsf{T} + R)^{-1}HP^-$ gives

$$(P^+)^{-1} = (P^-)^{-1} + H^\mathsf{T}R^{-1}H, \qquad K = P^+H^\mathsf{T}R^{-1}. \tag{6.8}$$

Information adds: each independent measurement contributes $H^\mathsf{T}R^{-1}H$. This is the form in which the MAP problem (3.6) is solved, which confirms that the Kalman update is its closed-form solution. The covariance form (6.7) inverts an $m \times m$ matrix (m = measurement dimension); the information form inverts an $n \times n$ matrix (n = state dimension). Filters use the former when $m < n$.

6.4 Properties

Proposition 6.2 (Optimality). *For the linear-Gaussian model, $\hat{x}_k^+ = \mathbb{E}[x_k \mid z_{1:k}]$: the Kalman filter is the minimum mean-square error estimator among* all *estimators, linear or not. If the noises are not Gaussian but have the stated means and covariances, it remains the best* linear *unbiased estimator.*

Proposition 6.3 (Innovation properties). *If the model is correct, the innovations r_k are zero-mean, Gaussian, with covariance S_k, and uncorrelated across time. Consequently $r_k^\mathsf{T}S_k^{-1}r_k \sim \chi^2_{m_k}$.*

Sketch. $r_k = H_k(x_k - \hat{x}_k^-) + n_k$ is a linear combination of independent zero-mean Gaussians, with covariance $H_kP_k^-H_k^\mathsf{T} + R_k = S_k$. Whiteness follows from the orthogonality principle: the estimation error is uncorrelated with all past measurements, hence with all past innovations. The χ^2 statement is Theorem 3.6. ∎

Proposition 6.3 is the most useful diagnostic in filtering, because it can be checked on real data without ground truth. The *normalized innovation squared* (NIS) $r^\mathsf{T}S^{-1}r$ should look like a χ^2_m sample. Values that are systematically too large mean the filter is overconfident or the model is wrong; systematically too small means it is underconfident. Gating (Chapter 11) uses the same statistic one measurement at a time.

6.5 The Joseph form

Proposition 6.4 (Joseph form). *For* any *gain K (optimal or not), the error covariance after the update $\hat{x}^+ = \hat{x}^- + K(z - H\hat{x}^-)$ is*

$$P^+ = (I - KH)\,P^-\,(I - KH)^\mathsf{T} + KRK^\mathsf{T}. \tag{6.9}$$

For the optimal gain it equals $(I - KH)P^-$.

Proof. The posterior error is $e^+ = x - \hat{x}^+ = (I - KH)(x - \hat{x}^-) - Kn$. Its two terms are independent, so by Proposition 3.1 its covariance is (6.9). Expanding with $K = P^-H^\mathsf{T}S^{-1}$: $(I - KH)P^-(I - KH)^\mathsf{T} + KRK^\mathsf{T} = (I - KH)P^- - (I - KH)P^-H^\mathsf{T}K^\mathsf{T} + KRK^\mathsf{T} = (I - KH)P^- - P^-H^\mathsf{T}K^\mathsf{T} + KSK^\mathsf{T}$, and the last two terms cancel because $KSK^\mathsf{T} = P^-H^\mathsf{T}K^\mathsf{T}$. ∎

Algorithm 1 Numerically careful Kalman update

Input: prior $(\hat{x}, P)$, measurement z, model (H, R)

$r \leftarrow z - H\hat{x}$ ▷ innovation
$S \leftarrow HPH^\mathsf{T} + R; \quad S \leftarrow \frac{1}{2}(S + S^\mathsf{T})$
$L \leftarrow \mathrm{chol}(S)$ ▷ or LDL^T; failure means S is not PD
$\gamma \leftarrow \|L^{-1}r\|^2$ ▷ NIS, for gating and logging
if $\gamma > \chi^2_{0.95}(\dim r)$ **then**
    return reject (and count it)
solve $S\,K^\mathsf{T} = HP$ for K^T using L ▷ never form S^{-1}
$\hat{x} \leftarrow \hat{x} + Kr$
$P \leftarrow (I - KH)P(I - KH)^\mathsf{T} + KRK^\mathsf{T}; \quad P \leftarrow \frac{1}{2}(P + P^\mathsf{T})$

Why use the longer formula? In floating point, $(I - KH)P^-$ is the difference of two nearly equal matrices when the update is informative, and it is not symmetric. Rounding can push small eigenvalues negative, after which the filter is broken: variances become negative, Cholesky fails, gates misbehave. The Joseph form is a sum of a congruence of P^- and a congruence of R, so by Proposition 2.7 it is PSD whenever P^- and R are, regardless of rounding in K. It also remains correct when K is deliberately suboptimal (for example, a gain computed with a slightly stale P). AGENTS.md F3 mandates it.

Implementation note: cost of the Joseph form

With n states and m measurements, $(I - KH)P(I - KH)^\mathsf{T}$ costs $O(n^3)$ if done naively. For the MSCKF ($n = 75$) this is fine. If it ever matters, note that the update can be rearranged as $P^+ = P - KHP - (KHP)^\mathsf{T} + KSK^\mathsf{T}$, which costs $O(n^2 m)$ and is algebraically identical to the Joseph form for any gain. It no longer has the sum-of-congruences structure that guarantees positive semi-definiteness under rounding, though, so keep the symmetrization and monitor the smallest eigenvalue in tests. Profile before optimizing.

6.6 Implementing the update

Algorithm 1 collects the numerical rules of the previous sections. Two details deserve comment. The gain is obtained by solving $SK^\mathsf{T} = HP^\mathsf{T} = HP$ (since P is symmetric), so the Cholesky factor of S is reused for both the gate and the gain. And symmetrization after each step removes the asymmetry that rounding introduces, which otherwise slowly accumulates (AGENTS.md F2).

Sequential updates. If R is block-diagonal, the blocks of a stacked measurement are independent given x, and processing them one after another with intermediate updates gives exactly the same result as a single batch update. This can reduce cost (several small S inversions instead of one large one) and allows per-block gating. If the blocks share errors (correlated R), sequential processing without accounting for the correlation is wrong.

6.7 From continuous to discrete time

Physical systems, including the error dynamics of an inertial navigation system, are naturally written in continuous time:

$$\dot{x}(t) = F_c\, x(t) + G_c\, w(t), \qquad \mathbb{E}[w(t)w(s)^\mathsf{T}] = Q_c\, \delta(t-s), \tag{6.10}$$

where w is white noise with power spectral density Q_c (Section 3.7). Over a step of length Δt with F_c, G_c constant, the solution is

$$x(t+\Delta t) = \underbrace{\exp(F_c \Delta t)}_{\Phi}\, x(t) + \underbrace{\int_0^{\Delta t} \exp(F_c s)\, G_c\, w(t+\Delta t - s)\, \mathrm{d}s}_{w_k}, \tag{6.11}$$

which is exactly of the form (6.1). The discrete process noise is zero-mean Gaussian with covariance (by the same white-noise calculation as in Proposition 3.8)

$$Q_d = \int_0^{\Delta t} \exp(F_c s)\, G_c Q_c G_c^\mathsf{T} \exp(F_c s)^\mathsf{T}\, \mathrm{d}s. \tag{6.12}$$

6.7.1 Approximations

Expanding to first order in Δt gives the approximations used by most real-time filters and frozen in AGENTS.md F2:

$$\Phi \approx I + F_c \Delta t, \qquad Q_d \approx G_c Q_c G_c^\mathsf{T}\, \Delta t. \tag{6.13}$$

Both are accurate to $O(\Delta t^2)$ per step. The units confirm (6.13): a noise density σ_a in $\mathrm{m/s^2/\sqrt{Hz}}$ gives $\sigma_a^2 \Delta t$ in $\mathrm{m^2/s^2}$, a velocity variance. A second-order option adds $\frac{1}{2}F_c^2 \Delta t^2$ to Φ and $\frac{1}{2}(F_c G Q G^\mathsf{T} + G Q G^\mathsf{T} F_c^\mathsf{T})\Delta t^2$ to Q_d.

Example 6.5 (Double integrator). For position and velocity driven by white acceleration of density σ, $F_c = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$, $G_c = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$, $Q_c = \sigma^2$. Example 2.16 gives $\exp(F_c s) G_c = [s;\ 1]$, so (6.12) evaluates exactly to

$$Q_d = \sigma^2 \begin{bmatrix} \Delta t^3/3 & \Delta t^2/2 \\ \Delta t^2/2 & \Delta t \end{bmatrix}. \tag{6.14}$$

The first-order approximation $Q_d \approx \mathrm{diag}(0, \sigma^2 \Delta t)$ puts no noise on position within a single step. Yet after n steps of exact Φ, the accumulated position variance is $\sum_{k=1}^{n} (k\Delta t)^2 \sigma^2 \Delta t \to \sigma^2 t^3/3$ as $\Delta t \to 0$ (a Riemann sum), which is the exact answer of Proposition 3.8. The per-step error of the approximation does not accumulate into a wrong growth law; it only introduces an $O(\Delta t)$ relative error, negligible at 200 Hz.

6.7.2 Van Loan's method

When higher accuracy is wanted, (6.12) can be computed exactly with a single matrix exponential [51].

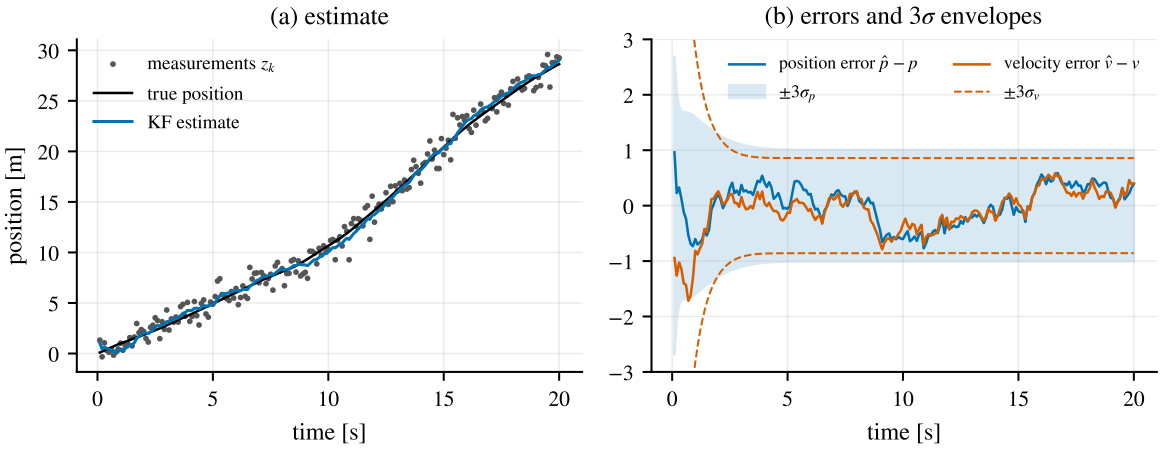


Figure 6.1: A constant-velocity Kalman filter tracking a 1D target from noisy position measurements. (a) The estimate smooths the measurements. (b) The errors stay inside the $\pm 3\sigma$ envelopes, which shrink quickly from the uninformative prior and settle at the steady-state values set by the balance between process noise and measurement noise. The velocity is never measured, yet it is estimated through its correlation with position.

Proposition 6.6 (Van Loan). *Let* $C = \exp\left(\begin{bmatrix} -F_c & G_c Q_c G_c^\mathsf{T} \\ 0 & F_c^\mathsf{T} \end{bmatrix} \Delta t\right) = \begin{bmatrix} C_{11} & C_{12} \\ 0 & C_{22} \end{bmatrix}$. *Then* $\Phi = C_{22}^\mathsf{T}$ *and* $Q_d = \Phi\, C_{12}$.

Proof. Block-triangular exponentials satisfy $C_{22} = \exp(F_c^\mathsf{T} \Delta t)$ and $C_{12} = \int_0^{\Delta t} \exp(-F_c(\Delta t - s))\, G Q G^\mathsf{T} \exp(F_c^\mathsf{T} s)\, \mathrm{d}s$ (verify by differentiating in Δt). Multiplying by $\Phi = \exp(F_c \Delta t)$ on the left and substituting $s \to \Delta t - s$ gives (6.12). ∎

For the 15-state INS this costs a 30×30 matrix exponential per step, too expensive at 200 Hz but excellent as a test oracle for the closed forms of Chapter 10.

6.8 A worked example

Figure 6.1 shows the filter on the double-integrator model of Example 6.5, with position measurements at 10 Hz. Three behaviors generalize to VIO. The covariance converges from its initial value to a steady state determined only by Q, R and the geometry, not by the data: the *discrete algebraic Riccati equation*. The unmeasured state (velocity) is estimated through the correlation that the dynamics build up between position and velocity. And the errors stay inside the 3σ envelope about 99.7% of the time, the visual signature of a consistent filter.

Key takeaways

- Predict: $\hat{x} \leftarrow \Phi \hat{x}$, $P \leftarrow \Phi P \Phi^\mathsf{T} + Q$. Update: condition the joint Gaussian of state and measurement.
- $K = P H^\mathsf{T} S^{-1}$ maps innovations into state corrections, including corrections to unmeasured but correlated states.

- Innovations of a correct filter are white with covariance S; their NIS is χ^2.
- Use the Joseph form, solve rather than invert, and symmetrize.
- Discretize with $\Phi \approx I + F_c \Delta t$ and $Q_d \approx G_c Q_c G_c^\top \Delta t$. Van Loan's method gives the exact values for testing.

Exercises

6.1 Derive (6.7) for a scalar state measured twice by independent sensors of variances R_1 and R_2, first sequentially, then as a stacked 2D measurement. Show the results agree.

6.2 Show that $P^+ \preceq P^-$ for the optimal gain. Is this still true for an arbitrary gain in the Joseph form?

6.3 Construct a 2×2 example in which $(I - KH)P^-$ computed in single precision is not positive semi-definite while the Joseph form is.

6.4 Verify Proposition 6.6 numerically for the double integrator, and compare Q_d with the first-order approximation for $\Delta t = 0.005$ and $\Delta t = 0.5$.

6.5 For the scalar random walk $x_k = x_{k-1} + w_k$ with $\mathrm{Var}(w) = q$ and direct measurements with variance r, solve the steady-state Riccati equation for P_∞^- and the steady-state gain.

6.6 Implement Algorithm 1 in Python, reproduce Figure 6.1, and compute the fraction of time-steps where the position error lies outside $\pm 3\sigma$.

Chapter 7

The Extended and Error-State Kalman Filters

In this chapter

- Extend the Kalman filter to nonlinear models by linearization, and understand what that costs.
- Understand the three-way split into true, nominal and error state, and why it is the right way to filter on a manifold.
- Define measurement residuals and Jacobians on the manifold with the correct sign convention.
- Derive the injection and reset steps, including the reset Jacobian G.
- Assemble the complete error-state Kalman filter loop.

7.1 The extended Kalman filter

Real systems are nonlinear:

$$x_k = f(x_{k-1}, u_k, w_k), \qquad z_k = h(x_k) + n_k, \tag{7.1}$$

where u_k is a known input such as an IMU reading. The *extended* Kalman filter (EKF) propagates the mean through the nonlinear functions and the covariance through their linearizations about the current estimate:

$$\hat{x}_k^- = f(\hat{x}_{k-1}^+, u_k, 0), \qquad P_k^- = F_k P_{k-1}^+ F_k^\mathsf{T} + L_k Q_k L_k^\mathsf{T}, \tag{7.2}$$

$$r_k = z_k - h(\hat{x}_k^-), \qquad H_k = \tfrac{\partial h}{\partial x}\big|_{\hat{x}_k^-}, \quad K_k = P_k^- H_k^\mathsf{T} (H_k P_k^- H_k^\mathsf{T} + R_k)^{-1}, \tag{7.3}$$

with $F_k = \partial f/\partial x$ and $L_k = \partial f/\partial w$ evaluated at the estimate. The update then follows Theorem 6.1.

The EKF is not optimal. It replaces each nonlinear transformation of a Gaussian by its linearization (Section 3.5), which is accurate only when the uncertainty is small compared with the curvature of f and h. Its Jacobians are evaluated at estimates that contain errors, which can make the linearized model claim information the real system does not provide (the consistency problem of Chapter 20). Despite this, the EKF and its variants dominate real-time navigation because they are cheap and, when errors are kept small, accurate. The *iterated* EKF relinearizes the measurement function at the updated estimate and repeats the update, which is one Gauss–Newton step per iteration; the unscented Kalman filter and particle filters avoid Jacobians altogether at a higher cost.

7.2 Filtering on a manifold

The EKF assumes x lives in $\mathbb{R}^n$, so that $\hat{x} + Kr$ makes sense. For VIO, x contains a rotation. Three quantities must therefore be distinguished.

Definition 7.1 (True, nominal and error state).

- The **true state** x is the unknown physical state, a point on the state manifold $\mathcal{M}$.
- The **nominal state** $\hat{x} \in \mathcal{M}$ is the current best estimate, propagated with the full nonlinear model and no noise.
- The **error state** $\delta x = x \boxminus \hat{x} \in \mathbb{R}^n$ is the small, minimal-dimension difference, defined with the operators of Section 5.8.

The filter's Gaussian belief is over the error state: $\delta x \sim \mathcal{N}(\delta\hat{x}, P)$, so that $x = \hat{x} \boxplus \delta x$.

For the IMU state this means 16 nominal numbers (quaternion included) but a 15-dimensional error and a 15×15 covariance, with no redundant directions and no constraints. The Kalman filter operates only on the error state, which is a vector, so all of Chapter 6 applies to it. This design is called the *error-state* (or *indirect*) Kalman filter, ESKF for short. In aerospace the same idea for attitude is the *multiplicative* EKF.

Key idea: why the error state

- **Minimal and unconstrained.** The error has exactly as many components as degrees of freedom, so its covariance is non-singular and needs no constraint handling.
- **Always near the origin.** After every update the error is reset to zero, so it never approaches the singularities of three-parameter rotations, and the small-angle approximations used to derive its dynamics remain valid.
- **Nearly linear.** The large, fast, nonlinear motion is carried by the nominal state, which is integrated exactly. The error evolves slowly and is small, so its dynamics are almost linear and linearization is accurate.
- **Slow.** Error dynamics are governed by sensor errors, not by vehicle motion, so the covariance could even be propagated at a lower rate than the nominal state if computation were scarce.

7.3 The ESKF cycle

Figure 7.1 shows the four steps.

1. **Nominal propagation.** At each IMU sample, integrate the nonlinear kinematics with the measured inputs, ignoring noise (Chapter 9).
2. **Error covariance propagation.** Propagate P with the linearized error dynamics (Chapter 10). The error-state mean does not need propagating: it is zero after every reset, and the linear error dynamics driven by zero-mean noise keep it zero.
3. **Update.** When a measurement arrives, compute the residual and its Jacobian on the manifold, and compute the Kalman estimate of the error, $\delta\hat{x} = Kr$, and the posterior covariance.

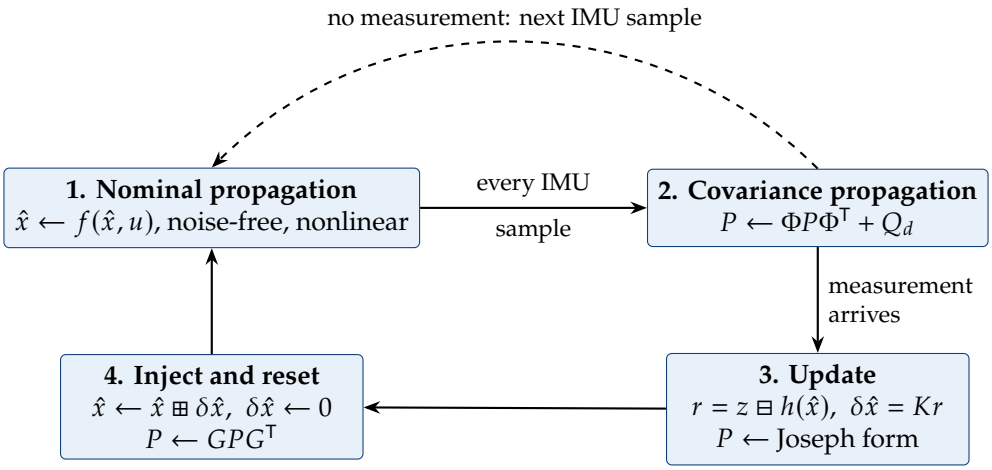


Figure 7.1: The error-state Kalman filter cycle. The error-state mean is zero throughout propagation, so only the covariance is propagated in step 2. It becomes non-zero in step 3 and is transferred to the nominal state and zeroed in step 4.

4. **Injection and reset.** Move the estimated error into the nominal state with $\boxplus$, set the error mean back to zero, and correct the covariance for the change of reference point.

7.4 Linearized error dynamics: the recipe

Suppose the true state obeys $\dot{x} = f(x, u, w)$ with noise w, and the nominal state obeys $\dot{\hat{x}} = f(\hat{x}, u, 0)$. The error dynamics are derived in four mechanical steps.

1. Write the true dynamics block by block.
2. Substitute $x = \hat{x} \boxplus \delta x$ in every block. For the attitude, $R = \hat{R}\,\mathrm{Exp}(\delta\theta)$.
3. Subtract the nominal dynamics.
4. Keep only terms of first order in the small quantities (δx and w), using $\mathrm{Exp}(\delta\theta) \approx I + [\delta\theta]_\times$ and dropping their products.

The result is a linear time-varying system

$$\dot{\delta x} = F_c(\hat{x}, u)\,\delta x + G_c(\hat{x})\,w, \tag{7.4}$$

which Section 6.7 turns into Φ and Q_d. Chapter 10 applies the recipe to the inertial navigation equations. For the attitude block, a short calculation is needed to express $\frac{\mathrm{d}}{\mathrm{d}t}\mathrm{Exp}(\delta\theta)$, and Chapter 10 works it out for the global perturbation as an example and leaves the local one to you.

7.5 Measurement updates on the manifold

Let a sensor produce a measurement z on a manifold $\mathcal{Z}$ (a vector space, SO(3), a pose, ...) according to the generative model

$$z = h(x) \boxplus n, \qquad n \sim \mathcal{N}(0, R_z). \tag{7.5}$$

The ESKF compares the measurement with its prediction from the nominal state,

$$r = z \boxminus h(\hat{x}), \tag{7.6}$$

and models how the residual depends on the unknown true error δx. Substituting $z = h(\hat{x} \boxplus \delta x) \boxplus n$ and linearizing,

$$r \approx H\,\delta x + n, \qquad H = \frac{\partial}{\partial \delta x}\Big[h(\hat{x} \boxplus \delta x) \boxminus h(\hat{x})\Big]_{\delta x = 0}. \tag{7.7}$$

H is the manifold Jacobian of h (Definition 5.7). With r and H, the update is the ordinary Kalman update of the error state with prior $\mathcal{N}(0, P)$:

$$S = HPH^\mathsf{T} + R_z, \qquad K = PH^\mathsf{T}S^{-1}, \qquad \delta\hat{x} = Kr, \qquad P^+ = (I - KH)P(I - KH)^\mathsf{T} + KR_zK^\mathsf{T}. \tag{7.8}$$

Pitfall: the sign of H

There are two ways to set up the linearization, and they differ by a sign. The generative view above differentiates the *prediction* with respect to the *true* error: $r \approx +H\delta x$. An optimization view instead writes the residual as a function of a correction δ to the estimate, $e(\delta) = z \boxminus h(\hat{x} \boxplus \delta)$, whose Jacobian is $-H$ (to first order). Both lead to the same correction if used consistently. Mixing them flips every correction. In this project, H is always (7.7), and its finite-difference test perturbs the nominal state: $r(\delta x) := h(\hat{x} \boxplus \delta x) \boxminus h(\hat{x})$ and $H \approx \partial r / \partial \delta x$.

For a vector-valued measurement, $\boxminus$ is subtraction and this reduces to the EKF. For a rotation-valued measurement such as the pose measurement of Milestone 6, the attitude part of the residual is $\mathrm{Log}_q(\hat{q}^{-1} \otimes q_{meas})$ (AGENTS.md F3).

7.6 Injection and reset

After the update, the error estimate $\delta\hat{x} = Kr$ is non-zero. Injection moves it into the nominal state:

$$\hat{x}^+ = \hat{x} \boxplus \delta\hat{x}: \quad \hat{p} \leftarrow \hat{p} + \delta\hat{p}, \quad \hat{v} \leftarrow \hat{v} + \delta\hat{v}, \quad \hat{q} \leftarrow \hat{q} \otimes \mathrm{Exp}_q(\delta\hat{\theta}), \quad \hat{b} \leftarrow \hat{b} + \delta\hat{b}, \tag{7.9}$$

followed by normalization of every quaternion that was touched, for the IMU and every clone (AGENTS.md F4).

The error is now defined relative to the new nominal state, $\delta x^+ = x \boxminus \hat{x}^+$, and its mean is zero. But its covariance is not exactly P^+, because the reference point of the tangent space has moved.

Proposition 7.2 (Reset Jacobian). *Let $\delta x \sim \mathcal{N}(\delta\hat{x}, P^+)$ be the error relative to $\hat{x}$ after the update, and $\delta x^+ = x \boxminus (\hat{x} \boxplus \delta\hat{x})$ the error relative to the injected nominal state. To first order, $\delta x^+ \sim \mathcal{N}(0, GP^+G^\mathsf{T})$ with $G = \partial\delta x^+/\partial\delta x$ evaluated at $\delta\hat{x}$. For the vector blocks $G = I$, and for each attitude block*

$$G_\theta = I - \tfrac{1}{2}[\delta\hat{\theta}]_\times. \tag{7.10}$$

Proof. For vector blocks $\delta x^+ = \delta x - \delta\hat{x}$, whose Jacobian is I. For the attitude, $q = \hat{q} \otimes \mathrm{Exp}_q(\delta\theta)$ and $q = \hat{q}^+ \otimes \mathrm{Exp}_q(\delta\theta^+)$ with $\hat{q}^+ = \hat{q} \otimes \mathrm{Exp}_q(\delta\hat{\theta})$, so

$$\mathrm{Exp}(\delta\theta^+) = \mathrm{Exp}(-\delta\hat{\theta})\ \mathrm{Exp}(\delta\theta).$$

By the BCH formula (5.12), $\delta\theta^+ = \delta\theta - \delta\hat{\theta} - \frac{1}{2}\,\delta\hat{\theta} \times \delta\theta + \cdots$, whose derivative with respect to $\delta\theta$ is $I - \frac{1}{2}[\delta\hat{\theta}]_\times$. At $\delta\theta = \delta\hat{\theta}$ the value is exactly zero, so the new error has zero mean. ∎

***Remark* 7.3** (relation to the right Jacobian). The derivative of $\mathrm{Log}(\mathrm{Exp}(-\delta\hat{\theta})\,\mathrm{Exp}(\delta\theta))$ at $\delta\theta = \delta\hat{\theta}$ can be computed exactly with (5.18): it is $J_r(\delta\hat{\theta}) = I - \frac{1}{2}[\delta\hat{\theta}]_\times + O(\|\delta\hat{\theta}\|^2)$. The frozen specification (AGENTS.md F4) uses the first-order form (7.10). For a typical correction of a few milliradians the two differ by about 10^{-6} and the difference is irrelevant. Changing it would be a logged convention change, not a silent substitution.

The covariance reset is therefore

$$P \leftarrow GPG^\mathsf{T}, \qquad G = \mathrm{blkdiag}\left(I_6,\ I - \tfrac{1}{2}[\delta\hat{\theta}]_\times,\ I_6,\ \ldots\right), \tag{7.11}$$

with one $I - \frac{1}{2}[\delta\hat{\theta}_i]_\times$ block for each clone's attitude and identity elsewhere.

Frozen convention: injection and reset, AGENTS.md F4

Inject with $\boxplus$ into the IMU block and every clone; normalize every touched quaternion; reset $\delta\hat{x} \leftarrow 0$ and $P \leftarrow GPG^\mathsf{T}$ with G implemented exactly as above. Using $G \approx I$ is a deviation that must be logged.

How much does G matter? It rotates the attitude covariance by half the injected correction, a first-order effect in $\|\delta\hat{\theta}\|$. With small corrections at high rate the effect is tiny, and many implementations use $G = I$. Large corrections (after a long outage, or a loop closure) make it noticeable. It costs almost nothing, and implementing it exactly removes one approximation from the consistency analysis of Milestone 7.

7.7 The complete error-state filter

Algorithm 2 assembles the pieces. It is generic: Chapters 9 and 10 supply f, F_c and G_c for the IMU, Chapter 11 supplies h and H for a pose sensor, and Chapter 18 supplies them for a camera. When measurements arrive between IMU samples, the filter first propagates to the measurement time by splitting the IMU interval (Chapter 9).

Algorithm 2 Error-state Kalman filter (one IMU step and an optional update)

Input: nominal $\hat{x}$, covariance P, IMU sample at t_k, optional measurement z at t_k

$\Delta t \leftarrow t_k - t_{k-1}$ ▷ from integer nanosecond timestamps

$\hat{x} \leftarrow f(\hat{x}, u_k, \Delta t)$; normalize $\hat{q}$ ▷ nominal propagation

$\Phi \leftarrow I + F_c \Delta t; \quad Q_d \leftarrow G_c Q_c G_c^\mathsf{T} \Delta t$

$P \leftarrow \Phi P \Phi^\mathsf{T} + Q_d; \quad P \leftarrow \frac{1}{2}(P + P^\mathsf{T})$

if measurement z available **then**

  $r \leftarrow z \boxminus h(\hat{x}); \quad H \leftarrow \partial[h(\hat{x} \boxplus \delta x) \boxminus h(\hat{x})]/\partial \delta x$

  $S \leftarrow HPH^\mathsf{T} + R_z$; factor S; gate on $r^\mathsf{T} S^{-1} r$

  $K \leftarrow PH^\mathsf{T} S^{-1}$ (by solving); $\quad \delta\hat{x} \leftarrow Kr$

  $P \leftarrow (I - KH)P(I - KH)^\mathsf{T} + KR_z K^\mathsf{T}$

  $\hat{x} \leftarrow \hat{x} \boxplus \delta\hat{x}$; normalize quaternions ▷ inject

  $P \leftarrow GPG^\mathsf{T}; \quad P \leftarrow \frac{1}{2}(P + P^\mathsf{T})$ ▷ reset

Project link: Milestones 4–6

M4 implements step 2 of Algorithm 2, M5 steps 3–4, and M6 the update, injection and reset with a pose measurement. Testing them separately localizes bugs: if propagation is exact on synthetic data, and the covariance matches Monte Carlo statistics, then any later inconsistency is in the update.

Key takeaways

- The EKF linearizes about the estimate; it is accurate when errors are small relative to curvature.
- The ESKF splits the state into a nonlinear nominal part and a small, minimal, nearly linear error part. The Kalman filter only ever sees the error.
- The error mean is zero during propagation; only P is propagated.
- Residual $r = z \boxminus h(\hat{x}) \approx H\delta x + n$ with H the manifold Jacobian of h at $\hat{x}$.
- After injection, reset $P \leftarrow GPG^\mathsf{T}$ with $G_\theta = I - \frac{1}{2}[\delta\hat{\theta}]_\times$ for every attitude block.

Exercises

7.1 Write the EKF for a planar robot with state (x, y, ψ) driven by speed and turn rate and measured by range to a known beacon. Identify where linearization error enters.

7.2 Show that if the error state is zero-mean after a reset and $\dot{\delta x} = F_c \delta x + G_c w$ with zero-mean w, then it stays zero-mean until the next update.

7.3 Derive the reset Jacobian for the *global* perturbation $R = \mathrm{Exp}(\delta\theta)\hat{R}$. How does it differ from (7.10)?

7.4 Suppose $\delta\hat{\theta} = (0, 0, 0.1)$ rad and $P_{\theta\theta} = \mathrm{diag}(10^{-4}, 10^{-4}, 10^{-2})$. Compute $GP_{\theta\theta}G^\mathsf{T}$ and compare with $J_r(\delta\hat{\theta})P_{\theta\theta}J_r(\delta\hat{\theta})^\mathsf{T}$. When is the difference noticeable?

7.5 An iterated ESKF repeats the update with H re-evaluated at $\hat{x} \boxplus \delta\hat{x}$. Show that one iteration is a Gauss–Newton step on the MAP cost $\|\delta x\|^2_{P^{-1}} + \|z \boxminus h(\hat{x} \boxplus \delta x)\|^2_{R_z^{-1}}$.

Part III

Inertial Navigation

Chapter 8

Inertial Sensors and Their Errors

In this chapter

- Understand what MEMS accelerometers and gyroscopes physically measure, especially specific force.
- State the IMU measurement model of the project and the role of each term.
- Know the main IMU error sources and which ones the filter models.
- Convert between continuous-time noise densities, discrete per-sample noise, and datasheet units.
- Read an Allan deviation plot and use it to identify noise parameters.

8.1 How MEMS inertial sensors work

A MEMS (micro-electro-mechanical system) **accelerometer** is a tiny proof mass suspended on silicon springs. When the case accelerates, the springs must deflect to drag the mass along. The deflection, measured capacitively, is proportional to the force the springs exert per unit mass. A MEMS **gyroscope** keeps a proof mass vibrating along one axis. When the sensor rotates, the Coriolis force $-2m\,\omega \times \dot{r}$ pushes the mass along a perpendicular axis with an amplitude proportional to the angular rate, and that secondary motion is measured. A six-axis IMU packages three of each, nominally orthogonal, with electronics that filter, sample and timestamp the signals.

8.2 Specific force: what an accelerometer measures

Newton's second law for the proof mass m in an inertial frame reads

$$m\,a = F_{\text{spring}} + m\,g, \tag{8.1}$$

where a is the acceleration of the mass (equal to that of the case once transients settle) and g is the gravitational acceleration. The sensor measures the spring force, i.e. the *specific force*

$$f = \frac{F_{\text{spring}}}{m} = a - g. \tag{8.2}$$

Gravity acts on the mass and the case equally and cannot be felt directly: this is Einstein's equivalence principle. Two cases build intuition.

- **At rest on a table**, $a = 0$, so $f = -g$. With $g_w = [0, 0, -9.81]^\mathsf{T}$, the accelerometer reads $+9.81$ m/s^2 *upward*: it senses the table pushing it up.

Error	Description	In the model?
White noise	sample-to-sample electronic and thermo-mechanical noise (angle/velocity random walk)	yes, n_g, n_a
Turn-on bias	constant offset that differs at each power-up	yes, initial b and P_0
In-run bias drift	slow wandering of the offset	yes, random walk
Bias instability	flicker ($1/f$) noise floor of the bias	approximated by the random walk
Scale factor	output = $(1+s)\times$ input	no
Axis misalignment	sensing axes not exactly orthogonal / aligned with the case	no
g-sensitivity	gyro output depends on acceleration	no
Temperature effects	bias and scale factor change with temperature	no (absorbed by bias walk)
Quantization	finite resolution of the ADC	no (below white noise)
Saturation, vibration	clipping on impacts; aliasing of high-frequency vibration	no
Earth rotation	gyros sense 7.3×10^{-5} rad/s	no (Chapter 4)

Table 8.1: IMU error sources and how the project's model treats them.

- **In free fall**, $a = g$, so $f = 0$. A falling IMU reads zero, whatever its velocity.

To recover the kinematic acceleration from the measurement, the estimator rotates the specific force into the world frame and *adds gravity back*: $a_w = R_{wb} f_b + g_w$. A sign error here produces a vertical acceleration error of $2g \approx 20$ m/s^2 and an obviously diverging altitude, which is fortunately easy to recognize.

8.3 The measurement model

Frozen convention: IMU model, AGENTS.md C6

$$\omega_m = \omega_b + b_g + n_g, \qquad a_m = R_{wb}^{\mathsf{T}}(a_w - g_w) + b_a + n_a, \qquad \dot{b}_g = n_{b_g}, \quad \dot{b}_a = n_{b_a}, \tag{8.3}$$

with ω_m, a_m the gyroscope and accelerometer readings in the body frame, ω_b the true angular rate, a_w the true acceleration of the body in the world frame, b_g, b_a the biases (body frame), and $n_g, n_a, n_{b_g}, n_{b_a}$ zero-mean white noises with densities $\sigma_g, \sigma_a, \sigma_{b_g}, \sigma_{b_a}$. The noise vector is ordered $w = [n_a, n_g, n_{b_a}, n_{b_g}] \in \mathbb{R}^{12}$.

The model has two kinds of error. *White noise* (n_g, n_a) is independent from sample to sample and averages out. *Bias* (b_g, b_a) is a slowly varying offset that does not average out and is therefore estimated as part of the state. Its slow drift is modeled as a random walk driven by n_{b_g}, n_{b_a}. Over the minutes of a EuRoC sequence the biases change little but measurably.

8.4 Error sources

Table 8.1 lists the main error sources. Unmodeled errors do not disappear: they appear as apparent extra noise or apparent bias drift. A filter that ignores them must compensate with larger noise parameters than the white-noise floor alone would suggest, which is why tuning is needed even with a well-calibrated sensor. High-accuracy systems estimate scale factors and misalignments too, and EuRoC's calibration with Kalibr [16] removes most static misalignment beforehand.

8.5 Noise parameters and their units

The noise densities are continuous-time quantities (Section 3.7). For EuRoC's ADIS16448, the `imu0/sensor.yaml` file gives the values in Table 8.2.

YAML key	Symbol	Value	Units
`gyroscope_noise_density`	σ_g	1.6968×10^{-4}	$\mathrm{rad/s}/\sqrt{\mathrm{Hz}}$
`accelerometer_noise_density`	σ_a	2.0×10^{-3}	$\mathrm{m/s^2}/\sqrt{\mathrm{Hz}}$
`gyroscope_random_walk`	σ_{b_g}	1.9393×10^{-5}	$\mathrm{rad/s^2}/\sqrt{\mathrm{Hz}}$
`accelerometer_random_walk`	σ_{b_a}	3.0×10^{-3}	$\mathrm{m/s^3}/\sqrt{\mathrm{Hz}}$
`rate_hz`	$1/\Delta t$	200	Hz

Table 8.2: EuRoC IMU noise parameters (`imu0/sensor.yaml`). Verify against your copy of the dataset.

The units follow from the definitions. A gyro rate in rad/s with a white noise density has units $\mathrm{rad/s} \cdot \sqrt{\mathrm{s}} = \mathrm{rad/s}/\sqrt{\mathrm{Hz}}$. A gyro bias random walk is the integral of white noise with units of $\mathrm{rad/s^2}$, so its density is in $\mathrm{rad/s^2}/\sqrt{\mathrm{Hz}}$, and after t seconds the bias has standard deviation $\sigma_{b_g}\sqrt{t}$ in rad/s.

Discrete equivalents. At sample period Δt:

$$\sigma_{g,d} = \frac{\sigma_g}{\sqrt{\Delta t}}, \qquad \sigma_{a,d} = \frac{\sigma_a}{\sqrt{\Delta t}}, \qquad \sigma_{b_g,d} = \sigma_{b_g}\sqrt{\Delta t}, \qquad \sigma_{b_a,d} = \sigma_{b_a}\sqrt{\Delta t}. \tag{8.4}$$

For EuRoC: $\sigma_{g,d} = 2.4 \times 10^{-3}$ rad/s per sample, $\sigma_{a,d} = 2.8 \times 10^{-2}$ m/s^2 per sample, and the gyro bias diffuses by 1.4×10^{-6} rad/s per sample. In the filter you never need these directly, because $Q_d = G_c Q_c G_c^{\mathsf{T}} \Delta t$ uses the densities, but they are what a synthetic data generator must add to each sample.

Datasheet units. Manufacturers often quote angle random walk (ARW) in $^\circ/\sqrt{\mathrm{h}}$ and velocity random walk (VRW) in $\mathrm{m/s}/\sqrt{\mathrm{h}}$. Since $1\ ^\circ/\sqrt{\mathrm{h}} = \frac{\pi/180}{60}\ \mathrm{rad}/\sqrt{\mathrm{s}} = 2.909 \times 10^{-4}\ \mathrm{rad/s}/\sqrt{\mathrm{Hz}}$, EuRoC's σ_g is an ARW of $0.58\ ^\circ/\sqrt{\mathrm{h}}$, and its σ_a is a VRW of $0.12\ \mathrm{m/s}/\sqrt{\mathrm{h}}$. Bias instability is quoted in $^\circ/\mathrm{h}$ and is *not* the same thing as the random walk density.

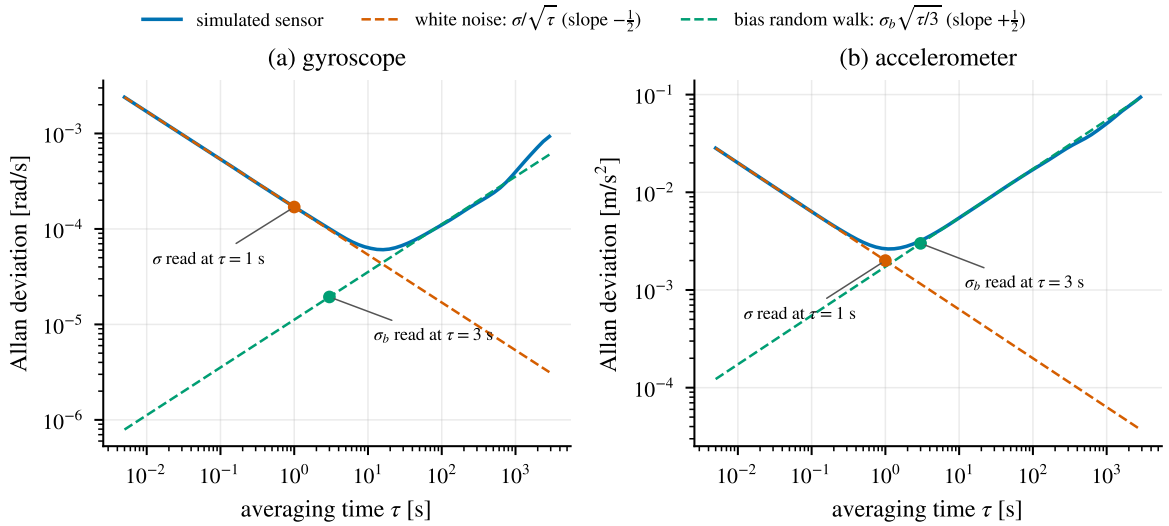


Figure 8.1: Allan deviation of a simulated 4-hour static recording with EuRoC's noise parameters. The $-\frac{1}{2}$ slope of white noise dominates short averaging times and the $+\frac{1}{2}$ slope of the bias random walk dominates long ones. Reading the asymptotes at $\tau = 1$ s and $\tau = 3$ s recovers the densities.

8.6 The Allan variance

How are these parameters measured? Record the sensor at rest for a few hours, then compute the *Allan variance*. Split the record of rate samples into clusters of duration τ, average each cluster to get $\bar{\omega}_k(\tau)$, and compute half the mean squared difference of consecutive cluster averages:

$$\sigma_A^2(\tau) = \tfrac{1}{2}\,\mathbb{E}\big[(\bar{\omega}_{k+1}(\tau) - \bar{\omega}_k(\tau))^2\big]. \tag{8.5}$$

Each noise process has a characteristic signature on a log-log plot of the Allan deviation $\sigma_A(\tau)$:

Process	$\sigma_A^2(\tau)$	slope of σ_A
quantization noise	$3Q^2/\tau^2$	-1
white noise (density σ)	σ^2/τ	$-\frac{1}{2}$
bias instability (B)	$\approx (0.664B)^2$	0
bias random walk (density σ_b)	$\sigma_b^2\tau/3$	$+\frac{1}{2}$
rate ramp (R)	$R^2\tau^2/2$	$+1$

White noise is read off the $-\frac{1}{2}$ line at $\tau = 1$ s, and the random walk density off the $+\frac{1}{2}$ line at $\tau = 3$ s. The minimum of the curve marks the averaging time beyond which a stationary sensor's bias starts to wander, and its height is related to the bias instability.

Figure 8.1 shows the procedure on simulated data. Real sensors add a flat bias-instability region near the minimum, which the random-walk model only approximates. The IEEE standard procedure [28] and El-Sheimy et al. [13] give details.

8.7 Tuning in practice

The densities from an Allan analysis describe a stationary sensor in a quiet lab. On a flying vehicle, vibration, temperature changes, scale-factor and misalignment errors, timing jitter and integration error all add to the effective noise. Common practice is to start from the Allan values (or the dataset's values) and inflate them by a modest factor until the filter is consistent, as judged by the statistical tests of Chapter 12. Two rules keep tuning honest.

1. **Change physically meaningful parameters only**, and keep them within a plausible range of their measured values.
2. **Judge on consistency statistics** (NEES on synthetic data, NIS and 3σ coverage on real data), not on trajectory error alone. Tuning for the lowest trajectory error on the test sequence is overfitting.

Pitfall: mixing up the two discretizations

The white-noise variance per sample is $\sigma^2/\Delta t$ (larger at higher rate), while the random-walk variance per step is $\sigma_b^2 \Delta t$ (smaller at higher rate). Swapping them is a common bug in synthetic data generators, and it produces noise that is wrong by a factor of $1/\Delta t^2 = 40\,000$ at 200 Hz.

8.8 The EuRoC IMU stream

EuRoC's `imu0/data.csv` contains one row per sample: `timestamp [ns], w_x, w_y, w_z [rad/s], a_x, a_y, a_z [m/s²]`, i.e. ω_m and a_m in B (AGENTS.md C7). Samples arrive at nominally 200 Hz, hardware-synchronized with the cameras. The filter must nevertheless take Δt from consecutive timestamps (AGENTS.md C5): real data has jitter and occasional gaps, which Milestone 3 logs. The ground-truth file starts later than the IMU file, so the filter is initialized at the first ground-truth sample at or after the stream start.

Project link: Milestones 3 and 4

M3 parses `sensor.yaml` into exactly the parameters of Table 8.2 and builds the event stream. M4's synthetic trajectory generator produces noise-free IMU data from (8.3); adding noise with the discrete equivalents above turns it into the Monte Carlo generator of M5.

Key takeaways

- An accelerometer measures specific force $f = a - g$: it reads $+g$ upward at rest and zero in free fall.
- $\omega_m = \omega_b + b_g + n_g$ and $a_m = R_{wb}^{\mathsf{T}}(a_w - g_w) + b_a + n_a$, with random-walk biases.
- Noise densities have units of $[u]/\sqrt{\text{Hz}}$; per-sample white noise is $\sigma/\sqrt{\Delta t}$, per-step bias diffusion is $\sigma_b\sqrt{\Delta t}$.
- The Allan deviation identifies white noise (slope $-\frac{1}{2}$, read at 1 s) and bias random walk (slope $+\frac{1}{2}$, read at 3 s).

- Unmodeled errors force noise inflation; tune on consistency, not on accuracy.

Exercises

8.1 An IMU lies on a table tilted by 10° about its x-axis. Compute the ideal accelerometer reading. Which component reveals the tilt, and with what sign?

8.2 A gyroscope datasheet gives ARW = $0.2\ ^\circ/\sqrt{\text{h}}$. Convert it to $\text{rad/s}/\sqrt{\text{Hz}}$ and to the per-sample standard deviation at 400 Hz.

8.3 Simulate 2 hours of a gyro with EuRoC parameters, compute the overlapping Allan deviation, and recover σ_g and σ_{b_g} from the asymptotes. How long a recording do you need to estimate σ_{b_g} to 10%?

8.4 Show that the Allan variance of white noise with density σ is σ^2/τ, and of a random walk with density σ_b is $\sigma_b^2\tau/3$.

8.5 A scale-factor error of $s = 10^{-3}$ affects a gyro on a vehicle spinning at 1 rad/s. What equivalent bias does it produce? Why can't a constant-bias model absorb it during maneuvers?

Chapter 9

Strapdown Propagation of the Nominal State

In this chapter

- State the continuous-time nominal kinematics and understand each term.
- Derive the exact solution of the kinematics for constant inputs over a step.
- Compare Euler, midpoint and Runge–Kutta integration, and predict their convergence orders.
- Propagate to arbitrary timestamps, handle gaps, and initialize the nominal state.
- Understand IMU preintegration as used by optimization-based estimators.

A *strapdown* inertial system has its sensors rigidly attached (strapped down) to the vehicle, as opposed to mounted on a stabilized gimbal platform. All the work of keeping track of orientation is done in software, by integrating the gyroscope. This chapter is about that integration.

9.1 Continuous-time kinematics

Taking the IMU model (8.3), replacing the unknown biases by their estimates and setting the noise to zero gives the nominal kinematics.

Frozen convention: nominal propagation, AGENTS.md F1

$$\hat{a} = a_m - \hat{b}_a, \quad \hat{\omega} = \omega_m - \hat{b}_g; \qquad \dot{\hat{p}} = \hat{v}, \quad \dot{\hat{v}} = \hat{R}_{wb}\,\hat{a} + g_w, \quad \dot{\hat{q}} = \tfrac{1}{2}\,\hat{q} \otimes [0,\, \hat{\omega}], \quad \dot{\hat{b}}_a = \dot{\hat{b}}_g = 0. \tag{9.1}$$

The attitude update is always multiplicative, $\hat{q} \leftarrow \hat{q} \otimes \mathrm{Exp}_q(\hat{\omega}\Delta t)$, followed by normalization. The integration scheme for p and v is chosen and justified in Milestone 4 and logged in `PROGRESS.md`.

The equations form a cascade: attitude depends on the gyroscope only; velocity depends on attitude and the accelerometer; position depends on velocity. Errors cascade the same way, which is the origin of the t^2 and t^3 growth laws of Chapter 1.

9.2 Exact solution for constant inputs

Suppose $\hat{\omega}$ and $\hat{a}$ are constant over $[t_k, t_k + \Delta t]$, which is what a zero-order hold of one IMU sample assumes. Then the kinematics can be solved in closed form.

Proposition 9.1 (Constant-input propagation). *With $\hat{\omega}$, $\hat{a}$ constant over the step and $\phi = \hat{\omega}\Delta t$,*

$$\hat{R}_{k+1} = \hat{R}_k \operatorname{Exp}(\phi), \tag{9.2}$$

$$\hat{v}_{k+1} = \hat{v}_k + g_w \Delta t + \hat{R}_k \, \Delta t \, J_l(\phi) \, \hat{a}, \tag{9.3}$$

$$\hat{p}_{k+1} = \hat{p}_k + \hat{v}_k \Delta t + \tfrac{1}{2} g_w \Delta t^2 + \hat{R}_k \, \Delta t^2 \, N(\phi) \, \hat{a}, \tag{9.4}$$

where $N(\phi) = \sum_{j \geq 0} [\phi]_\times^j / (j+2)! = \frac{1}{2} I + \frac{\theta - \sin\theta}{\theta^3} [\phi]_\times + \frac{\theta^2/2 + \cos\theta - 1}{\theta^4} [\phi]_\times^2$ and $\theta = \|\phi\|$.

Proof. The attitude is Proposition 4.5. During the step $\hat{R}(t_k + s) = \hat{R}_k \operatorname{Exp}(\hat{\omega} s)$, so $\hat{v}_{k+1} - \hat{v}_k = g_w \Delta t + \hat{R}_k \int_0^{\Delta t} \operatorname{Exp}(\hat{\omega} s) \, \mathrm{d}s \, \hat{a}$, and $\int_0^{\Delta t} \operatorname{Exp}(\hat{\omega} s) \, \mathrm{d}s = \Delta t \int_0^1 \operatorname{Exp}(u \phi) \, \mathrm{d}u = \Delta t \, J_l(\phi)$ by Theorem 5.9. Integrating once more gives the position, with $\int_0^{\Delta t} \int_0^s \operatorname{Exp}(\hat{\omega} u) \, \mathrm{d}u \, \mathrm{d}s = \Delta t^2 N(\phi)$ by term-wise integration of the exponential series, and the closed form follows from $[\phi]_\times^3 = -\theta^2 [\phi]_\times$. ∎

For small ϕ, $J_l(\phi) \approx I + \frac{1}{2}[\phi]_\times$ and $N(\phi) \approx \frac{1}{2} I + \frac{1}{6}[\phi]_\times$. The correction terms account for the rotation of the specific-force vector during the step. At 200 Hz and 1 rad/s, $\|\phi\| = 0.005$, so they change the velocity increment by about 0.25%: small, but systematic.

9.3 Numerical integration schemes

Real inputs are not constant over a step: the IMU provides samples at t_k and t_{k+1}, and the true signal varies between them. Three families of schemes are common. All of them update the attitude multiplicatively.

Euler with zero-order hold. Use the sample at the start of the step for the whole step:

$$\hat{R}_{k+1} = \hat{R}_k \operatorname{Exp}(\hat{\omega}_k \Delta t), \quad \hat{v}_{k+1} = \hat{v}_k + (\hat{R}_k \hat{a}_k + g_w)\Delta t, \quad \hat{p}_{k+1} = \hat{p}_k + \hat{v}_k \Delta t + \tfrac{1}{2}(\hat{R}_k \hat{a}_k + g_w)\Delta t^2. \tag{9.5}$$

Local error $O(\Delta t^2)$, global error $O(\Delta t)$. The dominant error is essentially a *half-sample time shift*: using the sample from the start of the interval for the whole interval delays the signal by $\Delta t/2$ on average.

Midpoint (trapezoidal). Use the average of the two bounding samples:

$$\hat{R}_{k+1} = \hat{R}_k \operatorname{Exp}\left(\tfrac{1}{2}(\hat{\omega}_k + \hat{\omega}_{k+1})\Delta t\right), \quad \bar{a}_w = \tfrac{1}{2}\left(\hat{R}_k \hat{a}_k + \hat{R}_{k+1} \hat{a}_{k+1}\right) + g_w, \tag{9.6}$$

$$\hat{v}_{k+1} = \hat{v}_k + \bar{a}_w \Delta t, \qquad \hat{p}_{k+1} = \hat{p}_k + \hat{v}_k \Delta t + \tfrac{1}{2} \bar{a}_w \Delta t^2.$$

Local error $O(\Delta t^3)$, global $O(\Delta t^2)$, at the cost of having to wait for the next sample before completing the step.

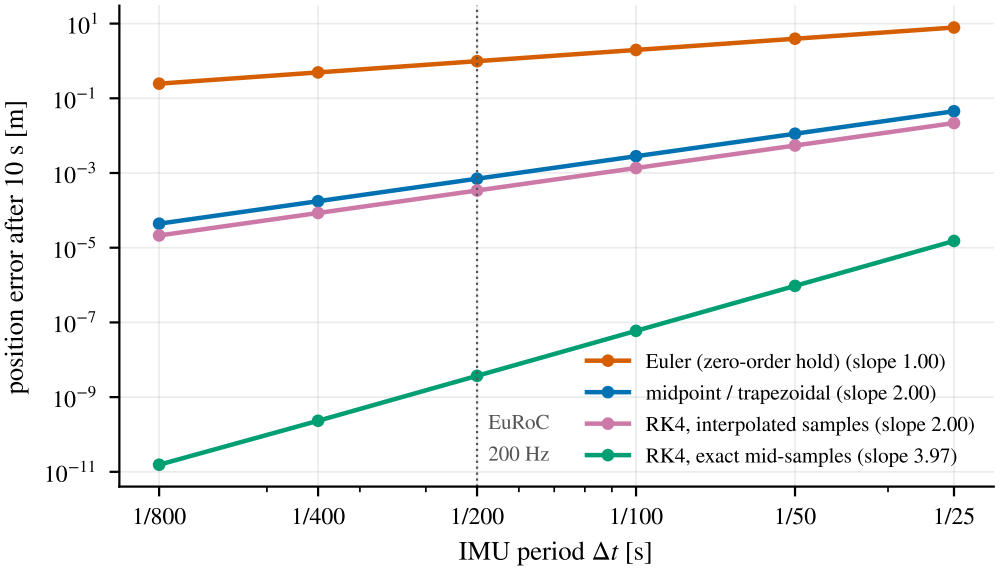


Figure 9.1: Position error after 10 s of noise-free integration on a synthetic trajectory (2 m circle at 0.8 rad/s with ±23° roll oscillations), as a function of the IMU period. The measured slopes match the predicted global orders. RK4 reaches fourth order only with exact mid-step inputs, which a real IMU does not provide; with interpolated inputs it is no better than the midpoint rule.

Runge–Kutta 4. RK4 evaluates the dynamics at the start, twice at the middle, and at the end of the step. Its global error is $O(\Delta t^4)$ *if* the inputs at mid-step are known exactly. With an IMU they are not, and must be interpolated from the bounding samples. Linear interpolation is only second-order accurate, which caps the whole scheme at $O(\Delta t^2)$.

Figure 9.1 confirms the predicted orders. Halving Δt divides the error by 2 for Euler and by 4 for the midpoint rule, which is exactly the convergence-order test of Milestone 4. The magnitudes matter as much as the slopes. At 200 Hz on this fairly dynamic trajectory, the Euler scheme accumulates about a meter of error in 10 s, far more than the sensor noise would cause (Figure 10.1), because the half-sample delay of the attitude creates a tilt error of order $\frac{1}{2}\omega\Delta t$, which leaks gravity. The midpoint rule is three orders of magnitude better. On gentler motion the difference shrinks, and frequent visual updates bound integration error in any case, but the integration error should be well below the noise-induced error, or it becomes an unmodeled error that the filter cannot account for.

Key idea: choosing a scheme

The integration error is a deterministic, unmodeled error. The scheme should make it small compared with the stochastic error the filter does model. Whichever scheme Milestone 4 adopts, document its order and verify it with the halving test on a trajectory with closed-form ground truth.

Coning and sculling. When the angular velocity vector itself rotates within a step (coning motion) or rotation and acceleration oscillate in phase (sculling), single-sample schemes rectify high-frequency motion into a spurious drift. High-end inertial systems with sample rates in the kHz range use multi-sample correction algorithms [19]. At 200 Hz on a flying

robot whose vibrations are mostly filtered out by the sensor, these effects are small compared with MEMS noise.

9.4 Timing

9.4.1 Propagating to an arbitrary time

Images arrive between IMU samples. To clone the state at an image time $t_c \in (t_k, t_{k+1})$ (AGENTS.md F5), the filter must propagate exactly to t_c: interpolate the IMU reading at t_c linearly between samples k and $k+1$, propagate from t_k to t_c with the interpolated input, clone, and then propagate from t_c to t_{k+1}. The covariance follows the same split, with Δt replaced by the two partial intervals.

9.4.2 Event ordering and gaps

Measurements must be processed in timestamp order, and the state must be propagated to a measurement's time before the measurement is applied (Chapter 19). Milestone 3 fixes the tie rule: an IMU sample precedes a measurement with an equal timestamp, so the state is propagated to that time first. Gaps in the IMU stream (intervals larger than about 1.5 nominal periods) must not be assumed away. Δt always comes from the timestamps, and a long gap simply produces a long step with a correspondingly large Q_d.

9.5 Initialization

9.5.1 From ground truth

The project initializes the nominal state from the first ground-truth sample at or after the start of the stream (Milestone 4): p, v, q, b_a and b_g all come from the ground-truth columns, mapped with the column order of AGENTS.md C7. This isolates propagation from the initialization problem. The use of ground truth for initialization is logged when the full VIO runs (Milestone 10b).

9.5.2 Static initialization

A real system initializes from its own sensors. If the vehicle is at rest for about a second,

$$\bar{a}_m = \mathrm{mean}(a_m) \approx R_{wb}^{\mathsf{T}}(-g_w) + b_a, \qquad \bar{\omega}_m = \mathrm{mean}(\omega_m) \approx b_g. \tag{9.7}$$

The gyro bias is the mean rate. Neglecting b_a, the unit vector $u_b = \bar{a}_m / \|\bar{a}_m\|$ is the world "up" direction expressed in the body frame, $u_b = R_{wb}^{\mathsf{T}} e_z$. Any rotation with $R_{wb} u_b = e_z$ is consistent with it: rotating about the axis $u_b \times e_z$ by the angle between the two vectors (computed with atan2) gives one, and composing with any rotation about e_z gives all the others. Roll and pitch are determined; yaw is arbitrary, and is set to zero (or to the ground-truth yaw for evaluation).

Pitfall: tilt and accelerometer bias are indistinguishable at rest

A horizontal accelerometer bias b_a produces the same reading as a tilt of $\|b_a\|/g$: a bias of 10 mm/s^2 looks exactly like a tilt of 1 mrad. At rest the filter cannot separate them, and the initial covariance must say so (large σ_{b_a}, correlated with the tilt). They become separable once the vehicle rotates, because the bias rotates with the body while gravity does not.

Initial covariances should reflect real uncertainty: a few milliradians in roll and pitch, a large or unknown yaw, the turn-on bias range from the datasheet, and a small but non-zero velocity uncertainty at rest. *Dynamic* initialization from a moving start, as in VINS-Mono [39], first solves a vision-only structure-from-motion problem and then aligns it with preintegrated IMU data to recover scale, gravity and velocity.

9.6 Sidebar: IMU preintegration

An optimization-based estimator may relinearize a window of states many times. Re-integrating every IMU sample between keyframes i and j each time the estimate of state i changes would be wasteful. *Preintegration* [34, 15] integrates the measurements in the body frame of keyframe i, so that the result does not depend on the state at i:

$$\Delta R_{ij} = \prod_{k=i}^{j-1} \operatorname{Exp}(\hat{\omega}_k \Delta t), \quad \Delta v_{ij} = \sum_{k=i}^{j-1} \Delta R_{ik}\, \hat{a}_k \Delta t, \quad \Delta p_{ij} = \sum_{k=i}^{j-1} \left(\Delta v_{ik} \Delta t + \tfrac{1}{2} \Delta R_{ik}\, \hat{a}_k \Delta t^2 \right). \tag{9.8}$$

These relate the two keyframe states through $R_j = R_i \Delta R_{ij}$, $v_j = v_i + g\Delta t_{ij} + R_i \Delta v_{ij}$ and $p_j = p_i + v_i \Delta t_{ij} + \frac{1}{2} g \Delta t_{ij}^2 + R_i \Delta p_{ij}$. The preintegrated quantities depend on the bias estimate, and first-order bias Jacobians avoid re-integration when the bias estimate changes. A filter like the MSCKF does not need preintegration, because it integrates each sample exactly once.

Project link: Milestone 4

Exit criteria: a stationary input keeps the state constant to 10^{-9} for 60 s; a constant rate reproduces $q_0 \otimes \operatorname{Exp}_q(\omega t)$; a circular trajectory converges at the predicted order; the quaternion norm stays within 10^{-12} of one after 10^6 steps; propagation does not allocate; and 10 s of dead reckoning on MH_01 produce no NaN.

Key takeaways

- $\dot{p} = v$, $\dot{v} = R\hat{a} + g$, $\dot{q} = \frac{1}{2} q \otimes [0, \hat{\omega}]$; attitude updates are multiplicative.
- For constant inputs the solution is exact: $R \operatorname{Exp}(\phi)$, $v + g\Delta t + R\Delta t J_l(\phi)\hat{a}$, and a similar position formula.
- Euler is first order (a half-sample delay), midpoint second order. RK4 is limited to second order by input interpolation.
- Integration error is unmodeled error; keep it well below sensor-noise error and verify the order.

- Propagate exactly to measurement times by splitting the IMU interval.
- At rest, gravity gives roll and pitch, the mean gyro gives b_g, and tilt and horizontal b_a are indistinguishable.

Exercises

9.1 Verify Proposition 9.1 numerically by comparing with an ODE solver at tight tolerance.

9.2 Show that the Euler scheme applied to a constant rotation rate is exact for the attitude, but not for the velocity, and quantify the velocity error per step in terms of $\|\hat{\omega}\|\Delta t$.

9.3 Reproduce Figure 9.1. Then add gyro noise and plot the noise-induced position error on the same axes. At what IMU rate does integration error fall below noise error for each scheme?

9.4 Implement the static initializer and test it on synthetic data with a known tilt. Add a horizontal accelerometer bias and show that it biases the estimated tilt by $\|b_a\|/g$.

9.5 Why must the propagation to an image timestamp interpolate the IMU sample instead of using the nearest sample? Estimate the attitude error from using the nearest sample at 1 rad/s and 200 Hz.

Chapter 10

Error-State Dynamics and Covariance Propagation

In this chapter

- Set up the 15-dimensional IMU error state with the frozen index layout.
- Derive linearized error dynamics with the four-step recipe, including the attitude error under a perturbation convention.
- Discretize into Φ and Q_d and propagate the covariance of the IMU state and its clones.
- Predict analytically how fast each noise source inflates the position uncertainty.
- Validate the propagation by finite differences and by Monte Carlo simulation.

10.1 True and nominal systems

The true state evolves under the IMU model (8.3) solved for the physical quantities. With $\omega = \omega_m - b_g - n_g$ and $a = a_m - b_a - n_a$ the true body rate and specific force,

$$\dot{p} = v, \qquad \dot{v} = R\,(a_m - b_a - n_a) + g_w, \qquad \dot{R} = R\,[\omega_m - b_g - n_g]_\times, \qquad \dot{b}_a = n_{b_a}, \qquad \dot{b}_g = n_{b_g}, \tag{10.1}$$

where $R = R_{wb}$. The nominal state follows (9.1), which is (10.1) with the biases replaced by their estimates and the noise set to zero. The error state measures their difference.

Frozen convention: error-state layout, AGENTS.md C3

Indices	Block	Meaning
0–2	δp	position error, world frame
3–5	δv	velocity error, world frame
6–8	$\delta\theta$	attitude error, **body frame (local)**: $R = \hat{R}\,\mathrm{Exp}(\delta\theta)$
9–11	δb_a	accelerometer bias error
12–14	δb_g	gyroscope bias error
$15 + 6i \ldots +2$	δp_i	clone i position error
$18 + 6i \ldots +2$	$\delta\theta_i$	clone i attitude error (local)

Noise ordering: $w = [n_a, n_g, n_{b_a}, n_{b_g}] \in \mathbb{R}^{12}$.

Index constants belong in one place in the code (`constexpr` offsets in `types.hpp`), and every block access goes through them. Hard-coded literals such as `P.block<3,3>(6,6)` scattered through the code are how layout bugs are born.

10.2 Deriving the error dynamics

The goal is the linear system

$$\dot{\delta x}_I = F_c\,\delta x_I + G_c\,w, \qquad F_c \in \mathbb{R}^{15\times 15}, \quad G_c \in \mathbb{R}^{15\times 12}, \tag{10.2}$$

obtained with the recipe of Section 7.4. Two of the five blocks are immediate.

Position. Substituting $p = \hat{p} + \delta p$ and $v = \hat{v} + \delta v$ into $\dot{p} = v$ and subtracting $\dot{\hat{p}} = \hat{v}$ gives $\dot{\delta p} = \delta v$ exactly. No linearization is needed.

Biases. $\dot{b}_a = n_{b_a}$ and $\dot{\hat{b}}_a = 0$ give $\dot{\delta b}_a = n_{b_a}$, and likewise $\dot{\delta b}_g = n_{b_g}$.

The velocity and attitude blocks involve the rotation and require the Lie tools of Chapter 5. One lemma handles the time derivative of the perturbation itself.

> **Lemma 10.1** (Time derivative of a perturbation). *For a differentiable $\delta\theta(t)$, $\frac{\mathrm{d}}{\mathrm{d}t}\operatorname{Exp}(\delta\theta) = \operatorname{Exp}(\delta\theta)[J_r(\delta\theta)\,\dot{\delta\theta}]_\times = [J_l(\delta\theta)\,\dot{\delta\theta}]_\times \operatorname{Exp}(\delta\theta)$. To first order in $\delta\theta$, both reduce to $[\dot{\delta\theta}]_\times$ multiplying $\operatorname{Exp}(\delta\theta)$ on the corresponding side.*

Proof. Apply the definition (5.18) of J_r and J_l with $\delta = \dot{\delta\theta}\,\mathrm{d}t$. ∎

10.2.1 Worked example: attitude error under the global perturbation

To demonstrate the method without solving the project's exercise, we first derive the attitude error dynamics for the *global* perturbation $R = \operatorname{Exp}(\delta\theta_g)\hat{R}$. This is **not** the project's convention; it is here for contrast.

> **Example 10.2** (Global attitude error dynamics). Differentiate both sides of $R = \operatorname{Exp}(\delta\theta_g)\hat{R}$ using Lemma 10.1 to first order and the kinematics $\dot{R} = R[\omega]_\times$, $\dot{\hat{R}} = \hat{R}[\hat{\omega}]_\times$:
>
> $$\underbrace{\operatorname{Exp}(\delta\theta_g)\hat{R}[\omega]_\times}_{\dot{R}} \approx [\dot{\delta\theta}_g]_\times \operatorname{Exp}(\delta\theta_g)\hat{R} + \operatorname{Exp}(\delta\theta_g)\hat{R}[\hat{\omega}]_\times.$$
>
> Move the last term to the left and use $\omega - \hat{\omega} = -\delta b_g - n_g$:
>
> $$[\dot{\delta\theta}_g]_\times \operatorname{Exp}(\delta\theta_g)\hat{R} \approx \operatorname{Exp}(\delta\theta_g)\hat{R}[-\delta b_g - n_g]_\times.$$
>
> Multiply on the right by $(\operatorname{Exp}(\delta\theta_g)\hat{R})^{\mathsf{T}}$ and apply (2.8) with the rotation $\operatorname{Exp}(\delta\theta_g)\hat{R}$:
>
> $$[\dot{\delta\theta}_g]_\times \approx [\operatorname{Exp}(\delta\theta_g)\hat{R}(-\delta b_g - n_g)]_\times \approx [-\hat{R}\,\delta b_g - \hat{R}\,n_g]_\times,$$
>
> where the last step drops the product of $\delta\theta_g$ with δb_g and n_g, both small. Hence
>
> $$\dot{\delta\theta}_g = -\hat{R}\,\delta b_g - \hat{R}\,n_g. \tag{10.3}$$
>
> The global attitude error does not depend on itself at all: the world-frame error is constant unless driven by gyro errors, which are rotated into the world by $\hat{R}$.

10.2.2 Your turn: the local (right) perturbation

Derivation exercise (project Jacobian): attitude and velocity rows of F_c and G_c

Derive $\dot{\delta\theta}$ and $\dot{\delta v}$ under the project's local perturbation $R = \hat{R}\,\mathrm{Exp}(\delta\theta)$.

1. **Attitude.** Differentiate $R = \hat{R}\,\mathrm{Exp}(\delta\theta)$ with Lemma 10.1 (right form). Cancel the nominal term, then use $\mathrm{Exp}(-\delta\theta)[\hat{\omega}]_\times \mathrm{Exp}(\delta\theta) = [\mathrm{Exp}(-\delta\theta)\hat{\omega}]_\times$ and $\mathrm{Exp}(-\delta\theta)\hat{\omega} \approx \hat{\omega} - \delta\theta \times \hat{\omega}$. You should find a term proportional to $\delta\theta$ that the global version lacks. Explain it physically: what happens to a body-frame error when the body rotates?
2. **Velocity.** Substitute $R = \hat{R}\,\mathrm{Exp}(\delta\theta) \approx \hat{R}(I + [\delta\theta]_\times)$ and $a_m - b_a - n_a = \hat{a} - \delta b_a - n_a$ into $\dot{v} = R(a_m - b_a - n_a) + g_w$, subtract $\dot{\hat{v}} = \hat{R}\hat{a} + g_w$, drop second-order products, and use (2.2) to move $\delta\theta$ to the right.
3. **Assemble** F_c and G_c in the frozen order of AGENTS.md C3. Before filling any block, predict from the cascade structure of the kinematics which blocks must be zero.

Check: build $\Phi \approx I + F_c\Delta t$ and compare each column with $\big(f(\hat{x} \boxplus \epsilon e_i) \boxminus f(\hat{x})\big)/\epsilon$ for one step f of your nominal integrator, at a state with non-identity attitude, non-zero rates, specific force and biases (Milestone 5). Agreement must be at the level of the $O(\Delta t^2)$ terms neglected by $I + F_c\Delta t$. Then record the result in `DOCS/derivations.md`.

The worked solution is omitted from this edition; build without `NoSolutions` *to include it.*

Physically, the velocity row you will find encodes the most important coupling in inertial navigation: an attitude error tilts the computed specific force, so a fraction of the large vector $\hat{a}$ (dominated by the reaction to gravity, 9.81 m/s^2) leaks into the velocity error. That single term is behind the t^3 law of Chapter 1.

10.3 Discretization

Over one IMU step the error dynamics are discretized as in Section 6.7 (AGENTS.md F2):

$$\Phi \approx I_{15} + F_c\Delta t, \qquad Q_c = \mathrm{diag}\left(\sigma_a^2 I_3,\ \sigma_g^2 I_3,\ \sigma_{b_a}^2 I_3,\ \sigma_{b_g}^2 I_3\right), \qquad Q_d \approx G_c Q_c G_c^{\mathsf{T}} \Delta t, \tag{10.4}$$

with F_c and G_c evaluated at the nominal state and the bias-corrected IMU inputs at the beginning of the step (or at the midpoint, consistently with the chosen integration scheme). The densities are the continuous-time values of Table 8.2; the Δt in Q_d converts them into variances accumulated during the step.

***Remark* 10.3** (higher-order options)**.** F_c is block upper-triangular with a large nilpotent part, so $\exp(F_c\Delta t)$ has a closed form. In particular the attitude block of the exact Φ is a rotation matrix, while the first-order block is not exactly orthogonal. At 200 Hz and 1 rad/s the difference per step is about $(\|\hat{\omega}\|\Delta t)^2/2 \approx 10^{-5}$, below the noise. Adopting a higher-order Φ is allowed if it is logged (AGENTS.md F2). Van Loan's method (Proposition 6.6) gives exact reference values for testing whichever form is used.

10.4 Propagating the full covariance

With N clones the state has dimension $n = 15 + 6N$, and the covariance is partitioned as

$$P = \begin{bmatrix} P_{II} & P_{IC} \\ P_{CI} & P_{CC} \end{bmatrix}, \qquad P_{II} \in \mathbb{R}^{15\times 15},\ P_{IC} \in \mathbb{R}^{15\times 6N},\ P_{CC} \in \mathbb{R}^{6N\times 6N}. \tag{10.5}$$

Clones are frozen copies of past poses, so their error dynamics are zero: $\dot{\delta x}_C = 0$. The full transition matrix is blkdiag(Φ, I_{6N}), and the full process noise is blkdiag$(Q_d, 0)$. Multiplying out:

Frozen convention: covariance propagation, AGENTS.md F2

$$P_{II} \leftarrow \Phi P_{II}\Phi^{\mathsf{T}} + Q_d, \qquad P_{IC} \leftarrow \Phi P_{IC}, \qquad P_{CI} = P_{IC}^{\mathsf{T}}, \qquad P_{CC} \text{ unchanged}, \tag{10.6}$$

followed by $P \leftarrow \frac{1}{2}(P + P^{\mathsf{T}})$.

The block form is not only a convenience. Updating only the affected blocks costs $O(15^2 \cdot 6N)$ per step instead of $O(n^3)$, and it never touches the large P_{CC} block between images. The cross-covariance P_{IC} is what remembers that the clones and the current state share a history. It is what lets a visual constraint between two past poses correct the current velocity and biases.

Implementation note: allocation-free propagation

P is a fixed-*max*-size Eigen matrix (Chapter 21). Write the block updates with `.noalias()` into preallocated member workspaces, e.g. a 15×15 temporary for ΦP_{II} and a $15\times 6N_{max}$ temporary for ΦP_{IC}, because the product reads the block it overwrites. Avoid `auto` with Eigen expressions.

10.5 How uncertainty grows

The structure of F_c explains how quickly an IMU-only solution degrades. Consider a level, stationary IMU, and one horizontal axis. Three couplings act in cascade.

- A gyro error (white noise or bias error) integrates into a tilt error $\delta\theta$.
- A tilt of $\delta\theta$ rotates the measured reaction to gravity, producing a horizontal specific-force error of magnitude $g\,\delta\theta$, which integrates into velocity error.
- Velocity error integrates into position error.

Each noise source therefore reaches position after a known number of integrations, and Proposition 3.8 gives its contribution to the position variance directly.

Proposition 10.4 (Dead-reckoning growth laws). *For a level, stationary IMU with zero initial*

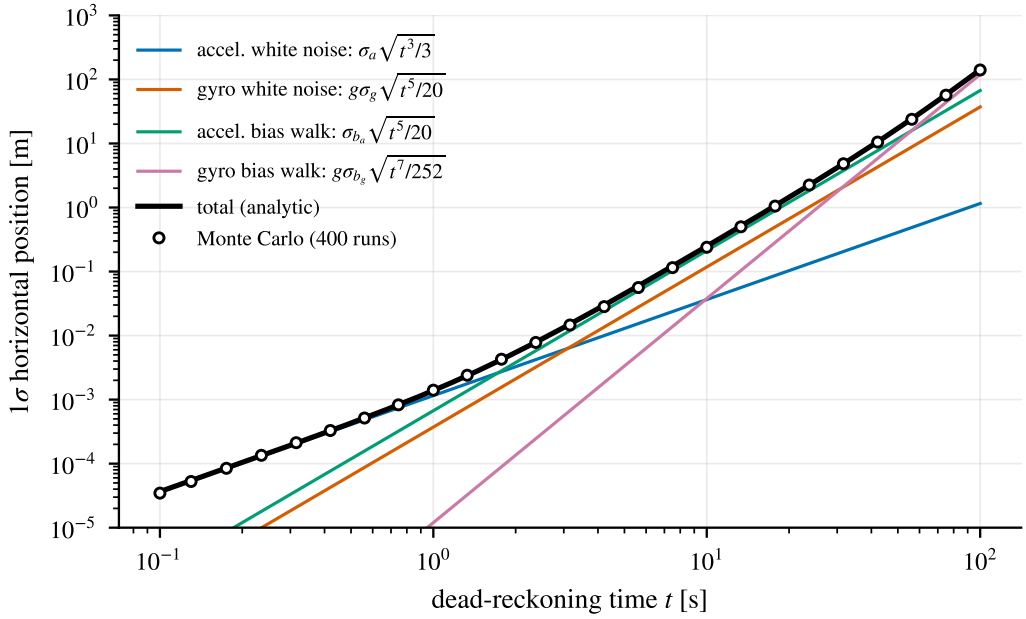


Figure 10.1: Horizontal position standard deviation of a dead-reckoning IMU with EuRoC noise parameters, from (10.7), with a Monte Carlo check (400 runs of the discretized error equations at 200 Hz). Accelerometer white noise dominates for the first two seconds, the accelerometer bias walk from there to about a minute, and then the gyro bias walk takes over with its $t^{7/2}$ growth.

uncertainty, the horizontal position variance grows as

$$\sigma_p^2(t) = \underbrace{\frac{\sigma_a^2 t^3}{3}}_{\text{accel. white}} + \underbrace{\frac{g^2\sigma_g^2 t^5}{20}}_{\text{gyro white}} + \underbrace{\frac{\sigma_{b_a}^2 t^5}{20}}_{\text{accel. bias walk}} + \underbrace{\frac{g^2\sigma_{b_g}^2 t^7}{252}}_{\text{gyro bias walk}}. \tag{10.7}$$

Proof. Accelerometer white noise is integrated twice ($m = 2$ in Proposition 3.8). Gyro white noise is integrated once into tilt, scaled by g, and integrated twice more ($m = 3$). The accelerometer bias random walk is itself integrated white noise, then integrated twice ($m = 3$). The gyro bias random walk adds one more integration ($m = 4$). The sources are independent, so their variances add. ∎

Figure 10.1 evaluates (10.7) with EuRoC's parameters. After one second of blind flight the position uncertainty is about 1 mm, after ten seconds about 25 cm, and after a minute more than 10 m. The camera's job is to keep the filter permanently in the left part of this plot.

Project link: Milestone 5

The analytic check of M5 isolates the first term of (10.7): with $P_0 = 0$, only accelerometer noise and a fixed orientation, the horizontal position variance must equal $\sigma_a^2 t^3/3$. Because Φ and Q_d are first-order approximations, the agreement improves as $\Delta t \to 0$ (Example 6.5), and the test tolerance should reflect that.

Why diagonal entries need not grow monotonically. The attitude covariance is expressed in body axes (Remark 5.14). When the vehicle rotates, the attitude block of Φ rotates the covariance between axes, so an individual diagonal entry can shrink during pure propagation even though no information is gained. Monotonicity holds for invariant quantities, such as the eigenvalues of $P_{\theta\theta}$ when the vehicle is not rotating, but not for individual entries.

10.6 Validating covariance propagation

Two independent tests catch nearly all propagation bugs.

Finite differences on the discrete propagation. The Jacobian of one step of the nominal integrator, computed numerically with $\boxplus/\boxminus$, must match Φ column by column (the derivation exercise above). This catches sign errors, transposes, and blocks placed at wrong indices. The test must use a non-trivial state.

Monte Carlo. Simulate M noisy IMU sequences from a known trajectory, integrate each with the nominal equations from a perturbed initial state drawn from P_0, and compute the errors $e^{(j)} = x_{\text{true}} \boxminus \hat{x}^{(j)}$. Their sample covariance must agree with the propagated P. If the filter is correct, the ratio of a sample variance to the predicted variance is distributed as $\chi^2_{M-1}/(M-1)$, so for $M = 500$ it should lie in $[0.88, 1.13]$ with 95% probability for each diagonal entry. Larger deviations, or many entries near the bounds in the same direction, indicate an error in G_c, Q_c or the units. This catches errors that finite differences cannot, such as a wrong noise density or a missing Δt.

Key takeaways

- The error state is 15-dimensional with the frozen layout $[\delta p, \delta v, \delta\theta, \delta b_a, \delta b_g]$.
- Error dynamics follow from substituting $x = \hat{x} \boxplus \delta x$, subtracting the nominal dynamics, and keeping first-order terms. The perturbation convention changes the result.
- $\Phi \approx I + F_c\Delta t$, $Q_d \approx G_c Q_c G_c^{\mathsf{T}} \Delta t$ with continuous densities; clones have identity dynamics.
- Position variance grows like t^3 (accel. noise), t^5 (gyro noise, accel. bias walk) and t^7 (gyro bias walk).
- Validate Φ by finite differences and P by Monte Carlo with statistically justified bounds.

Exercises

10.1 Derive the velocity error dynamics for the global perturbation, and compare its structure with your local result. Which one depends on the vehicle's heading?

10.2 Using your F_c, show that $\delta\theta$ is decoupled from δp and δv, and explain this from the cascade structure.

10.3 Starting from (10.4), write Q_d block by block and check its units.

10.4 Implement covariance propagation for the 15-state system in Python and reproduce Figure 10.1, including the Monte Carlo points.

10.5 For a vehicle rotating about its z-axis at 1 rad/s with $P_{\theta\theta} = \mathrm{diag}(10^{-4}, 10^{-6}, 10^{-6})$, propagate $P_{\theta\theta}$ for a quarter turn with no noise and observe the diagonal entries.

10.6 How large must M be for a Monte Carlo variance check to detect a 10% error in σ_a with high probability?

Part IV

The Error-State Filter in Practice

Chapter 11

Measurement Updates, Gating and the Pose Sensor

In this chapter

- Apply the generic manifold update to concrete sensors, deriving H by the elementary Jacobians.
- Understand which state directions a measurement observes, from the nullspace of H.
- Implement the mock pose sensor of Milestone 6 and derive its Jacobian.
- Gate measurements with the χ^2 test, and understand how gating fails in an overconfident filter.
- Understand how a pose sensor makes velocity and biases observable.

11.1 The generic update

Frozen convention: measurement update, AGENTS.md F3

1. Residual $r = z \boxminus h(\hat{x})$: plain subtraction for vector measurements; for the pose measurement $r = [\, p_{meas} - \hat{p};\ \mathrm{Log}_q(\hat{q}^{-1} \otimes q_{meas}) \,]$.
2. $H = \partial r / \partial \delta x$ at $\delta x = 0$ using C4 (Section 7.5), derived by hand and verified by finite differences.
3. $S = HPH^\mathsf{T} + R_z$; $K = PH^\mathsf{T}S^{-1}$ by a Cholesky/LDL$^\mathsf{T}$ solve, never an explicit inverse.
4. Mahalanobis gate $r^\mathsf{T}S^{-1}r < \chi^2_{0.95}(\dim r)$ from a `constexpr` table. Rejections are counted and logged, never silently dropped.
5. Joseph form $P \leftarrow (I-KH)P(I-KH)^\mathsf{T} + KR_zK^\mathsf{T}$, then injection and reset (Section 7.6).

Everything specific to a sensor is in h and H. The rest of this section derives them for two sensors that the project does not use, to demonstrate the technique, before turning to the pose sensor it does use.

11.2 Worked example: position with a lever arm

A GNSS antenna mounted at a known offset ℓ_b from the IMU, expressed in the body frame, measures the antenna position:

$$z = h(x) + n = p_{wb} + R_{wb}\,\ell_b + n, \qquad n \sim \mathcal{N}(0, \sigma^2 I_3). \tag{11.1}$$

The residual is a plain difference, $r = z - (\hat{p} + \hat{R}\ell_b)$. To find H, perturb the nominal state with $\boxplus$ and expand:

$$h(\hat{x} \boxplus \delta x) = \hat{p} + \delta p + \hat{R}\,\mathrm{Exp}(\delta\theta)\ell_b \approx \hat{p} + \delta p + \hat{R}\ell_b + \hat{R}[\delta\theta]_\times \ell_b = h(\hat{x}) + \delta p - \hat{R}[\ell_b]_\times \delta\theta.$$

Reading off the coefficients (the rotation-action row of Table 5.1),

$$H = \begin{bmatrix} I_3 & 0_3 & -\hat{R}[\ell_b]_\times & 0_3 & 0_3 \end{bmatrix}. \tag{11.2}$$

The attitude column is non-zero because the antenna swings around the IMU when the body rotates. A lever-arm position measurement therefore carries attitude information, but only about rotations that move the antenna: the nullspace of $-\hat{R}[\ell_b]_\times$ is the direction ℓ_b, since rotating about the lever arm itself does not move the antenna.

11.3 Worked example: a direction sensor

A magnetometer, or an accelerometer used as an inclinometer at rest, measures a known world vector m_w in body coordinates:

$$z = R_{wb}^\mathsf{T} m_w + n. \tag{11.3}$$

With the inverse-action row of Table 5.1, $h(\hat{x} \boxplus \delta x) = \mathrm{Exp}(-\delta\theta)\hat{R}^\mathsf{T} m_w \approx \hat{R}^\mathsf{T} m_w + [\hat{R}^\mathsf{T} m_w]_\times \delta\theta$, so

$$H = \begin{bmatrix} 0_3 & 0_3 & [\hat{R}^\mathsf{T} m_w]_\times & 0_3 & 0_3 \end{bmatrix}. \tag{11.4}$$

$[v]_\times$ has rank two and nullspace $\mathrm{span}(v)$, so a direction measurement observes two of the three attitude degrees of freedom. The rotation about the measured direction is unobservable. For gravity this is the familiar statement that an accelerometer at rest gives roll and pitch but not yaw.

Key idea: the nullspace of H tells you what a sensor cannot see

Any error direction $\delta x \in \mathrm{null}(H)$ produces no change in the predicted measurement, so the update cannot correct it. Reading the nullspace of each new H is a quick sanity check of both the derivation and your intuition about the sensor.

11.4 The pose sensor of Milestone 6

Before introducing a camera, the project tests the whole ESKF with a "mock sensor" that measures the pose directly. Ground-truth poses, decimated to about 10 Hz, are corrupted with synthetic noise of known statistics.

Frozen convention: pose measurement, AGENTS.md C6

$$p_{meas} = p_{wb} + n_p, \qquad q_{meas} = q_{wb} \otimes \mathrm{Exp}_q(n_\theta), \qquad R_z = \mathrm{diag}(\sigma_p^2 I_3,\ \sigma_\theta^2 I_3). \tag{11.5}$$

The attitude noise is applied on the right, in the body frame, which is exactly the local perturbation. That choice makes the residual (7.6) statistically clean: if the nominal state were exact, the attitude residual would be exactly n_θ. To generate the noise, draw n_p and n_θ from seeded `std::mt19937_64` generators (AGENTS.md E5) with the same σ_p, σ_θ that appear in R_z.

Derivation exercise (project Jacobian): the pose measurement Jacobian $H \in \mathbb{R}^{6\times 15}$

Using (7.7), derive H for $h(x) = (p_{wb}, q_{wb})$ with the residual of AGENTS.md F3.

1. The position rows are a plain difference. Which state blocks does $\hat{p} + \delta p$ depend on?
2. For the attitude rows, evaluate $\mathrm{Log}_q\left(\hat{q}^{-1} \otimes (\hat{q} \otimes \mathrm{Exp}_q(\delta\theta))\right)$ and simplify. Which elementary Jacobian applies if you instead linearize around a non-zero residual?
3. Write H in the frozen column order and state its nullspace. Which states are *not* directly measured, and how can they still be corrected?

Check: the finite-difference test of $r(\delta x) = h(\hat{x} \boxplus \delta x) \boxminus h(\hat{x})$ at a non-trivial state, as required by Milestone 6.

The worked solution is omitted from this edition; build without `NoSolutions` *to include it.*

How unmeasured states get corrected. The pose sensor measures neither velocity nor biases, yet the update corrects both. The mechanism is the cross-covariance built up by propagation. Between updates, a velocity error produces a growing position error, so P_{pv} becomes large; an accelerometer bias produces a velocity error and then a position error, so P_{pb_a} grows; a gyro bias produces an attitude error, so $P_{\theta b_g}$ grows. The gain $K = PH^\mathsf{T}S^{-1}$ has non-zero rows for every state correlated with the measured ones, so each pose update corrects them in proportion to those correlations.

Why the biases converge. Observability here has an intuitive reading. The gyro bias is the slope of the difference between the integrated gyro and the measured attitude, which a sequence of attitude measurements reveals. The velocity is the slope of the measured position. The accelerometer bias is the difference between the acceleration implied by successive velocities and the specific force rotated with the (measured) attitude, so it too becomes observable. Even at rest it is separable from tilt, because the attitude is measured directly. With pose measurements every state of the IMU error state is observable, and their covariances converge.

11.5 Gating

Real measurements include outliers: a misassociated feature, a reflection, a moving object. A single outlier with a large residual can drag the estimate far away and corrupt the covariance. The χ^2 test of Proposition 6.3 rejects measurements that are statistically implausible given the filter's own uncertainty:

$$\gamma = r^\mathsf{T} S^{-1} r \overset{?}{<} \chi^2_{1-\alpha}(\dim r). \tag{11.6}$$

dim r	1	2	3	4	5	6
$\chi^2_{0.95}$	3.841	5.991	7.815	9.488	11.070	12.592
$\chi^2_{0.99}$	6.635	9.210	11.345	13.277	15.086	16.812

Table 11.1: Gate thresholds. A longer table, up to 600 degrees of freedom, is in Appendix C.

With a 95% gate, a consistent filter rejects 5% of perfectly good measurements. That is the price of rejecting most outliers, and it is why rejections are counted and logged: a rejection rate close to 5% is a sign of health, a rate near zero suggests an underconfident filter (or a gate that is too loose to be useful), and a rate far above 5% indicates overconfidence or a model error.

Pitfall: the death spiral

Gating assumes the filter is consistent. If P is too small, good measurements produce large γ and are rejected; with no corrections P should grow, but if the model's process noise is also too small it grows slowly while the true error grows quickly, and more measurements are rejected. The filter then dead-reckons with false confidence until it diverges. When the rejection rate jumps, suspect overconfidence before suspecting outliers.

The threshold depends on the dimension of r. A pose measurement uses $\chi^2_{0.95}(6)$, a single image observation $\chi^2_{0.95}(2)$, and a projected MSCKF feature residual $\chi^2_{0.95}(2M-3)$, where M is the number of observations (Chapter 18). Because the quantiles grow with the dimension, the table is generated offline and stored as a `constexpr` array, which avoids depending on a statistics library in the core.

11.6 Testing the update

Project link: Milestone 6

The exit criteria test properties that hold for any correct implementation:

- H passes the finite-difference test at a non-trivial state;
- $P_{prior} - P_{post}$ is positive semi-definite (an update never adds uncertainty);
- after injection and reset all quaternions are unit, and a second update with the same z produces a residual consistent with the first correction;
- as $R_z \to 0$ the posterior pose converges to z;
- with constant synthetic biases, the bias estimates converge to within their own 3σ;
- update, injection and reset do not allocate;
- a full MH_01 run with noisy ground-truth updates neither diverges nor produces NaN, and the χ^2 rejection count is logged.

The "second update" test is subtle and worth understanding. After the first update, the nominal pose has moved toward z by Kr, so the second residual should be approximately

$(I - HK)r$, which is smaller. If it is instead larger, or has the opposite sign, the injection used the wrong side of the quaternion product or the wrong sign of H.

Key takeaways

- Derive H by perturbing $h(\hat{x} \boxplus \delta x)$, expanding to first order and reading off coefficients.
- The nullspace of H lists the error directions a sensor cannot see.
- The pose sensor's attitude noise is a local perturbation, matching the residual $\mathrm{Log}_q(\hat{q}^{-1} \otimes q_{meas})$.
- Unmeasured states are corrected through cross-covariances built up by propagation.
- Gate with $\chi^2_{0.95}(\dim r)$, log the rejections, and treat a high rejection rate as a symptom of overconfidence.

Exercises

11.1 For the lever-arm example, compute S and K for a state with

$$P = \mathrm{diag}(\sigma_p^2 I,\ \sigma_v^2 I,\ \sigma_\theta^2 I,\ \sigma_{b_a}^2 I,\ \sigma_{b_g}^2 I).$$

Which components of $\delta\theta$ does the update correct?

11.2 Derive H for a range measurement to a known beacon b_w: $z = \|p_{wb} - b_w\| + n$. What is its nullspace?

11.3 Simulate the death spiral: run a 1D constant-velocity filter with Q underestimated by a factor of 100 and a 95% gate. Plot the rejection rate and the error over time.

11.4 Show that with a pose measurement of a static vehicle, the horizontal accelerometer bias is observable. (Hint: what does the attitude measurement remove from the tilt/bias ambiguity of Chapter 9?)

11.5 Prove that $P_{prior} - P_{post} = KSK^\mathsf{T} \succeq 0$ for the optimal gain, and explain why this test can fail in floating point if the simple covariance update is used.

Chapter 12

Consistency and Evaluation

In this chapter

- Test whether a filter's covariance tells the truth, with NEES, NIS and 3σ coverage.
- Compute exact statistical bounds for single-run and Monte Carlo tests.
- Diagnose over- and under-confidence and tune noise parameters without cheating.
- Measure trajectory accuracy with ATE and RPE, using the alignment appropriate for VIO.

A filter produces two things: an estimate and a covariance. Most evaluations look only at the first. But a filter with good average accuracy and a covariance that is too small is dangerous: it rejects valid measurements, it reports false confidence to whatever consumes its output, and it eventually diverges. This chapter is about checking the second output.

12.1 What consistency means

Recall from Chapter 3 that a filter reporting $(\hat{x}_k, P_k)$ is consistent if its error $e_k = x_k \boxminus \hat{x}_k$ has zero mean and covariance P_k. Together with Proposition 6.3, a consistent, correctly modeled filter satisfies three testable properties:

1. estimation errors are compatible with P_k (tested with NEES, needs ground truth);
2. innovations are compatible with S_k (tested with NIS, needs no ground truth);
3. innovations are white (tested with autocorrelation, needs no ground truth).

The error must be computed with the manifold difference $\boxminus$, using exactly the same convention as the filter's covariance. Comparing a local-perturbation covariance with a global attitude error, or with an Euler angle difference, produces meaningless statistics.

12.2 Normalized estimation error squared

Definition 12.1 (NEES). The normalized estimation error squared at time k is

$$\epsilon_k = e_k^\mathsf{T} P_k^{-1} e_k, \qquad e_k = x_k \boxminus \hat{x}_k. \tag{12.1}$$

If the filter is consistent and its errors are Gaussian, $\epsilon_k \sim \chi^2_n$ with $n = \dim e$ by Theorem 3.6: its expected value is n. A single run gives one sample per time step, which is noisy. Monte Carlo runs average it out. With N independent runs, the average NEES $\bar{\epsilon}_k = \frac{1}{N}\sum_{j=1}^{N} \epsilon_k^{(j)}$

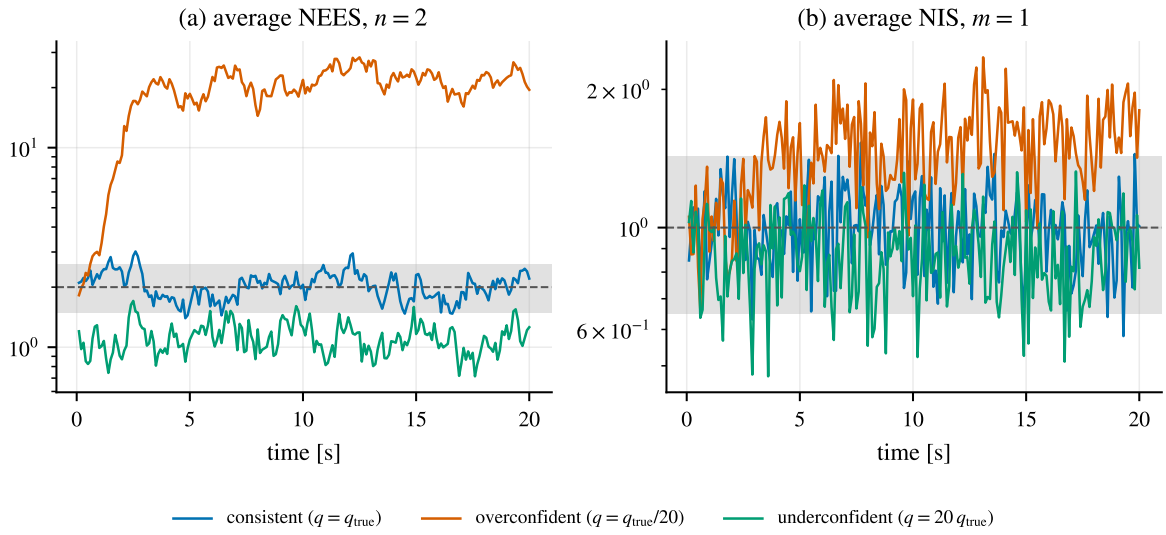


Figure 12.1: Consistency statistics of a constant-velocity filter over 50 Monte Carlo runs, with the 95% acceptance region (gray) of (12.2). Underestimating the process noise by a factor of 20 makes the filter overconfident: the NEES leaves the region by an order of magnitude, and the NIS rises above it. Overestimating it makes the filter underconfident: the NEES falls below the region. Note that the NIS of the underconfident filter is less clearly abnormal: NIS alone is a weak detector of underconfidence.

satisfies $N\bar{\epsilon}_k \sim \chi^2_{Nn}$, so the two-sided 95% interval is

$$\left[\frac{\chi^2_{0.025}(Nn)}{N}, \frac{\chi^2_{0.975}(Nn)}{N} \right]. \tag{12.2}$$

For the 15-state IMU filter with $N = 50$ runs this is $[13.52, 16.56]$; for $N = 100$, $[13.95, 16.09]$. Appendix C lists more. Average NEES above the interval means overconfidence (errors larger than claimed); below means underconfidence.

Block NEES. The NEES of the full state mixes well-estimated and poorly estimated blocks. Computing it separately for position, velocity, attitude and biases, each with its own 3×3 block of P and χ^2_3 bounds, points directly at the misbehaving part.

12.3 Normalized innovation squared

Definition 12.2 (NIS). $\nu_k = r_k^\mathsf{T} S_k^{-1} r_k$, the same quantity the gate computes.

Under consistency $\nu_k \sim \chi^2_m$ with $m = \dim r_k$. NIS needs no ground truth, so it is the main consistency monitor on real data. Two summaries are useful: the fraction of updates with ν_k below its 95% quantile, which should be close to 95% (Milestone 7), and the time-average of ν_k, which should be close to m. If measurements have different dimensions, normalize by m_k or compare each against its own quantile.

NEES	NIS	Likely causes
high	high	process noise too small; wrong noise units (missing Δt or $\sqrt{\Delta t}$); unmodeled IMU errors; integration error
high	normal	errors in directions the measurement does not see (unobservable states, e.g. yaw in VIO); wrong $\boxminus$ convention in the evaluation
normal	high	measurement noise too small; outliers passing the gate; wrong H
low	low	process or measurement noise too large
high, growing	normal	spurious information along unobservable directions (EKF inconsistency, Chapter 20)
spikes	spikes	time-synchronization errors; wrong extrinsics; bad initialization

Table 12.1: Reading consistency statistics. Combinations narrow the search for the bug.

3σ **coverage.** For each scalar component i, a consistent Gaussian error lies within $\pm 3\sqrt{P_{ii}}$ 99.7% of the time. Plotting errors against their 3σ envelopes is the most readable diagnostic, and the coverage fraction is a simple scalar summary. On EuRoC, Milestone 7 requires at least 95% coverage for velocity and biases. These states are not measured by the pose sensor, so their consistency is not circular.

12.4 Diagnosing inconsistency

Table 12.1 is a starting point for debugging. Two additional checks are valuable. First, plot each residual component over time: a non-zero mean reveals a bias (wrong extrinsics, a wrong sign in h); slow oscillation reveals correlated errors (a coloured noise the filter assumes white). Second, compare the covariance with Monte Carlo statistics on synthetic data, where the truth is known exactly and the noise is exactly as modeled. If the filter is inconsistent on synthetic data, the bug is in the filter. If it is consistent there but not on real data, the problem is the model.

12.4.1 Tuning without cheating

1. **Synthetic first.** With synthetic data generated with exactly the filter's noise parameters, there is nothing to tune: a consistent implementation must pass. Any failure is a bug.
2. **Physically meaningful knobs only.** On real data, adjust the IMU noise densities (start from Table 8.2, inflate modestly for unmodeled effects) and the measurement noise (start from the pixel noise of the detector). Do not add arbitrary noise to arbitrary states.
3. **Judge on consistency, not accuracy.** Choose parameters that make NIS and coverage statistics correct. Lower trajectory error obtained by making the filter inconsistent will not generalize.
4. **Separate tuning from evaluation.** Tune on some sequences and report results on others.
5. **Ground truth never enters the filter** except where explicitly allowed and logged (initialization).

12.5 Trajectory accuracy

Consistency says whether the covariance is honest; accuracy says how small the errors are. VIO accuracy is reported with two metrics [47, 53].

12.5.1 Absolute trajectory error

The estimated trajectory lives in a world frame that is only defined up to the unobservable directions, so it must be aligned with the ground truth before comparing. Given time-associated positions p_i (ground truth) and $\hat{p}_i$ (estimate), find the transformation that best maps the estimate onto the truth, then compute

$$\mathrm{ATE}_{\mathrm{RMSE}} = \left(\frac{1}{N} \sum_{i=1}^{N} \left\| p_i - (R^\star \hat{p}_i + t^\star) \right\|^2 \right)^{1/2}. \tag{12.3}$$

For rigid alignment, the optimal transformation has a closed form (Kabsch/Umeyama [50]). Center both point sets on their means μ and $\hat{\mu}$, form $\Sigma = \frac{1}{N} \sum_i (p_i - \mu)(\hat{p}_i - \hat{\mu})^\mathsf{T} = U D V^\mathsf{T}$ (SVD), and set

$$R^\star = U \operatorname{diag}\left(1, 1, \det(U V^\mathsf{T})\right) V^\mathsf{T}, \qquad t^\star = \mu - R^\star \hat{\mu}. \tag{12.4}$$

Key idea: align only what is unobservable

Which transformation to allow depends on the estimator. Monocular vision alone has a free scale, so it uses a similarity transform (7 DOF). VIO has metric scale and observable roll and pitch: its unobservable directions are exactly translation and yaw. The appropriate alignment is therefore 4-DOF (translation plus a rotation about gravity) [53]. A full 6-DOF alignment would also remove roll and pitch errors that the estimator was responsible for. For centered positions, the optimal yaw is $\psi^\star = \operatorname{atan2}\left(\sum_i (y_i \hat{x}_i - x_i \hat{y}_i),\ \sum_i (x_i \hat{x}_i + y_i \hat{y}_i) \right)$.

Milestone 10b reports ATE after SE(3) alignment, as in most published benchmarks; reporting the 4-DOF result alongside it is more faithful to what VIO can know.

12.5.2 Relative pose error

ATE is dominated by the largest drift and depends on trajectory length and shape. The relative pose error (RPE) compares relative motions over a fixed distance or time Δ:

$$E_i = \left(T_i^{-1} T_{i+\Delta} \right)^{-1} \left(\hat{T}_i^{-1} \hat{T}_{i+\Delta} \right), \tag{12.5}$$

and reports statistics of the translation and rotation parts of E_i, often as a percentage of distance traveled. It needs no global alignment and measures local drift, which is what odometry is about. The `evo` package [20] computes both metrics with all the alignment variants.

Project link: Milestone 7

M7 logs $\hat{x}$, diag(P), ground truth, NIS and NEES to a preallocated buffer, flushes it outside the hot loop, and plots 3σ envelopes and NEES/NIS against their χ^2 bounds from a single script. Exit criteria: average NEES inside the 95% interval for 15 DOF over most of a ≥ 50-run synthetic Monte Carlo; NIS within its per-update bound for about 95% of updates; at least 95% 3σ coverage of velocity and biases on MH_01; zero allocations while logging.

Key takeaways

- A filter must be judged on the honesty of its covariance as well as its accuracy.
- NEES needs ground truth and tests P; NIS needs none and tests S. Both are χ^2 under consistency.
- Monte Carlo averages have exact bounds $[\chi^2_{0.025}(Nn), \chi^2_{0.975}(Nn)]/N$.
- Overconfidence (NEES too high) is the dangerous failure; NIS alone detects underconfidence poorly.
- Tune physically meaningful parameters for consistency, on synthetic data first.
- Report ATE after an alignment matched to the unobservable directions, and RPE for local drift.

Exercises

12.1 Show that the single-run NEES of a consistent 15-state filter exceeds its 95% quantile 5% of the time. Why is the Monte Carlo average a much sharper test?

12.2 Reproduce Figure 12.1. Then make only the measurement noise wrong instead of the process noise and compare the signatures.

12.3 Derive the closed-form 4-DOF alignment above by maximizing $\sum_i p_i^\mathsf{T} R_z(\psi)\hat{p}_i$ over ψ.

12.4 Explain why the NEES of yaw in a VIO run on real data tends to grow over time even when the NIS looks healthy. (Revisit after Chapter 20.)

12.5 Compute the RPE over 10 m segments for a trajectory with a constant yaw drift of 1° per minute flying at 1 m/s. How does it compare with the ATE after a 10-minute flight?

Part V

Geometric Computer Vision

Chapter 13

Camera Models

In this chapter

- Derive the pinhole projection and the roles of the intrinsic parameters.
- Model and invert the radial-tangential lens distortion used by EuRoC.
- State the project's camera measurement model in normalized coordinates, with its noise.
- Derive the projection Jacobian and understand what a single view cannot observe.
- Understand camera-IMU extrinsics and time offsets.

13.1 The pinhole model

An ideal pinhole camera lets light through a single point, the optical center. A scene point with camera-frame coordinates $p_c = (X, Y, Z)$, with Z along the optical axis, projects through the center onto an image plane. By similar triangles, its image on a virtual plane at unit distance in front of the center is

$$\pi(p_c) = \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} X/Z \\ Y/Z \end{bmatrix}. \tag{13.1}$$

These are the *normalized image coordinates*: dimensionless, independent of the lens, and equal to the tangents of the ray's angles from the optical axis. The vector $\bar{x} = (x, y, 1)$ is the *bearing* of the point, defined up to scale.

Pixels are obtained by scaling and shifting the normalized coordinates with the *intrinsic matrix*:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \qquad K = \begin{bmatrix} f_u & s & c_u \\ 0 & f_v & c_v \\ 0 & 0 & 1 \end{bmatrix}, \tag{13.2}$$

with focal lengths f_u, f_v in pixels, principal point (c_u, c_v), and skew s (zero for modern sensors). The focal length in pixels is the physical focal length divided by the pixel pitch. EuRoC's `cam0` has $f_u = 458.654$, $f_v = 457.296$, $c_u = 367.215$, $c_v = 248.375$ on a 752×480 sensor, a horizontal field of view of about 79°.

13.2 Lens distortion

Real lenses bend rays, most strongly near the image edges. The *radial-tangential* (Brown–Conrady) model used by EuRoC maps ideal normalized coordinates (x, y) to distorted ones

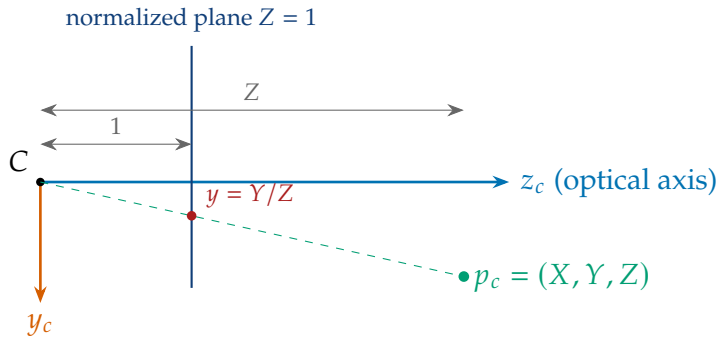


Figure 13.1: Pinhole projection in the y-z plane. The ray from the optical center C to the point intersects the plane $Z = 1$ at height Y/Z. All points along the ray project to the same image point, so depth is lost.

(x_d, y_d):

$$x_d = x\,(1 + k_1 r^2 + k_2 r^4) + 2p_1 xy + p_2(r^2 + 2x^2), \tag{13.3}$$

$$y_d = y\,(1 + k_1 r^2 + k_2 r^4) + p_1(r^2 + 2y^2) + 2p_2 xy, \qquad r^2 = x^2 + y^2, \tag{13.4}$$

after which K is applied to (x_d, y_d). For `cam0`, $(k_1, k_2, p_1, p_2) = (-0.2834, 0.07396, 1.94 \times 10^{-4}, 1.76 \times 10^{-5})$. The negative k_1 produces *barrel* distortion (Figure 13.2). At the middle of the left and right image edges the radial factor $1 + k_1 r^2 + k_2 r^4$ is about 0.79, which moves points by about 100 pixels; at the corners it is about 0.73, a shift of about 160 pixels. Ignoring distortion is therefore not an option.

Undistortion. The front end measures distorted pixels and needs ideal normalized coordinates. The model has no closed-form inverse, but a fixed-point iteration converges quickly for moderate distortion: starting from $x^{(0)} = x_d$,

$$x^{(j+1)} = \frac{x_d - \Delta_x(x^{(j)}, y^{(j)})}{1 + k_1 r_j^2 + k_2 r_j^4}, \qquad y^{(j+1)} = \frac{y_d - \Delta_y(x^{(j)}, y^{(j)})}{1 + k_1 r_j^2 + k_2 r_j^4}, \tag{13.5}$$

where Δ denotes the tangential terms. A Gauss–Newton iteration with the 2×2 Jacobian of the distortion converges faster and more reliably near the edges. OpenCV's `cv::undistortPoints` implements the iterative scheme. Milestone 8 checks undistortion by distorting and undistorting a grid of points.

Key idea: undistort points, not images

The estimator needs a few hundred feature locations, not a rectified image. Undistorting only the tracked points costs microseconds; remapping a full image costs milliseconds, resamples the pixels (blurring them), and changes the noise of the feature detector in a position-dependent way. Track in the raw image and undistort the tracks.

Wide-angle and fisheye lenses need different models, such as the equidistant (Kannala–Brandt) model, but the structure of the pipeline is the same: everything downstream of undistortion works with bearings.

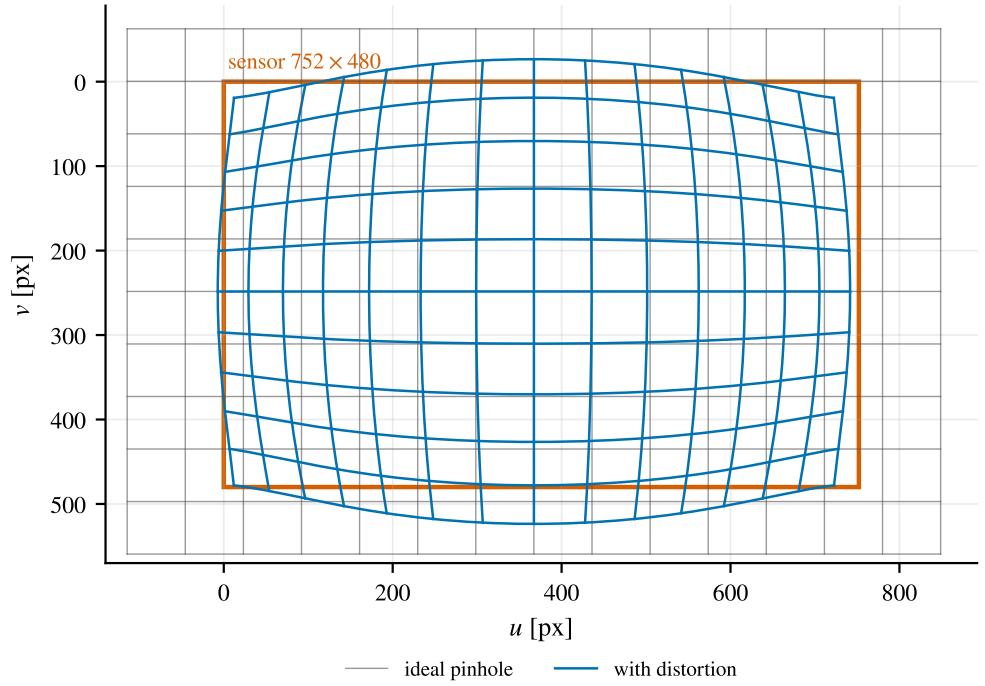


Figure 13.2: EuRoC `cam0` lens distortion. A regular grid in normalized coordinates (gray) is imaged by the real lens as the curved blue grid. Distortion pulls points toward the center, increasingly toward the edges, so the sensor (orange) sees a wider field than an ideal pinhole with the same focal length would.

13.3 The measurement model

Frozen convention: camera model, AGENTS.md C6

In normalized, undistorted image coordinates,

$$p_c = R_{bc}^{\mathsf{T}}\left(R_{wb}^{\mathsf{T}}(p_f - p_{wb}) - p_{bc}\right), \qquad z = \begin{bmatrix} p_{c,x}/p_{c,z} \\ p_{c,y}/p_{c,z} \end{bmatrix} + n_{uv}, \qquad \sigma_{uv} \approx \sigma_{px}/f. \tag{13.6}$$

The first equation chains the world-to-body and body-to-camera transformations of (4.16). The noise model follows from $u = f_u x + c_u$: a pixel error δu corresponds to a normalized error $\delta u / f_u$. With $\sigma_{px} = 1$ px and $f \approx 458$ px, $\sigma_{uv} \approx 2.2 \times 10^{-3}$. Strictly, distortion makes the mapping from pixel noise to normalized noise position-dependent, by the inverse of the distortion Jacobian, and $f_u \neq f_v$. Both effects are small for EuRoC and conventionally ignored, but they are a legitimate refinement.

13.4 The projection Jacobian

Proposition 13.1 (Jacobian of the pinhole projection)**.**

$$\frac{\partial \pi}{\partial p_c} = \frac{1}{Z}\begin{bmatrix} 1 & 0 & -X/Z \\ 0 & 1 & -Y/Z \end{bmatrix} = \frac{1}{Z}\begin{bmatrix} I_2 & -\pi(p_c) \end{bmatrix}. \tag{13.7}$$

Proof. Differentiate X/Z and Y/Z with the quotient rule. ∎

The Jacobian has rank two, and its nullspace is spanned by p_c itself: moving the point along its viewing ray does not change its image. This is the single-view loss of depth in differential form, and it is why a feature must be observed from at least two sufficiently different viewpoints before it constrains anything (Chapter 14). The factor $1/Z$ says that distant points have small image motion for a given change in position. They constrain rotation well and translation poorly.

The full measurement Jacobian with respect to the filter state is the chain-rule product of this matrix with the Jacobian of p_c with respect to the clone pose and the feature position. Deriving those is part of Chapter 18.

Pitfall: points behind the camera

(13.1) happily projects points with $Z < 0$, flipping their image through the center. A triangulation or linearization that produces a negative depth must be rejected (the *cheirality* check), not projected. Points with very small positive Z produce huge Jacobians and should be rejected too.

13.5 Extrinsics and time offset

Extrinsic calibration. The transformation T_{bc} between the IMU and the camera is estimated offline, for EuRoC with Kalibr [16], and stored as `T_BS` in `cam0/sensor.yaml`. Its rotation is close to a permutation of axes, and its translation is a few centimeters (Appendix D). An error of $\delta\phi$ in the extrinsic rotation biases every predicted bearing by about $\delta\phi$, which looks to the filter like a systematic residual. A translation error matters mostly for near points. The project treats T_{bc} as exact (AGENTS.md C3). Estimating it online is a stretch goal.

Time offset. If the camera timestamp differs from the IMU clock by t_d, a feature moving across the image at velocity $\dot{z}$ is measured with an error of about $\dot{z}\, t_d$. A 1 ms offset at an image-plane velocity of 0.5 normalized units per second (fast rotation) is an error of 5×10^{-4}, a quarter of a pixel. EuRoC is hardware synchronized, so $t_d \approx 0$; consumer devices can have offsets of tens of milliseconds and must estimate it online [32].

Shutter. EuRoC's cameras have global shutters: every pixel is exposed at the same instant. Rolling-shutter cameras expose rows sequentially, so each row has its own timestamp and pose, which requires an extended measurement model.

Key takeaways

- Normalized coordinates $(X/Z, Y/Z)$ are lens-independent bearings; K maps them to pixels.
- EuRoC's lens has strong barrel distortion; undistort the tracked points, not the images.
- The measurement model works in normalized coordinates with $\sigma_{uv} \approx \sigma_{px}/f$.

- $\partial\pi/\partial p_c = \frac{1}{Z}[I_2 \;\; -\pi(p_c)]$; its nullspace is the viewing ray.
- Reject points with negative or tiny depth. Extrinsic and timing errors appear as systematic residuals.

Exercises

13.1 Compute the horizontal and vertical fields of view of `cam0` with and without distortion.

13.2 Implement distortion and fixed-point undistortion for the EuRoC model. How many iterations are needed for 10^{-6} accuracy at the image corners? Compare with Gauss–Newton.

13.3 Show that the normalized noise of a feature at the image corner, mapped through the inverse distortion Jacobian, differs from σ_{px}/f, and by how much.

13.4 A point at depth 10 m is observed by a camera translating sideways at 1 m/s and rotating at 1 rad/s. Which motion dominates its image velocity? What does this imply for the observability of translation from distant points?

13.5 Show that a point at infinity (direction d) has image $\pi(R^\mathsf{T} d)$ independent of camera translation.

Chapter 14

Multi-View Geometry

In this chapter

- Derive the epipolar constraint and the essential matrix.
- Estimate relative motion from correspondences with the 8-, 5- and 2-point methods.
- Use RANSAC to reject outliers, size its iteration count, and set thresholds in the right units.
- Understand the pure-rotation degeneracy and how to handle it.
- Triangulate points linearly and by Gauss–Newton with an inverse-depth parameterization, and quantify triangulation accuracy as a function of parallax.

14.1 Epipolar geometry

Two cameras observe the same 3D point. Let the coordinates of the point in the two camera frames be X_1 and X_2, related by the rigid motion

$$X_2 = R\,X_1 + t, \qquad R = R_{c_2c_1}, \quad t = p_{c_2c_1}, \tag{14.1}$$

and let $\bar{x}_1 = X_1/Z_1$ and $\bar{x}_2 = X_2/Z_2$ be the bearings (homogeneous normalized coordinates).

Theorem 14.1 (Epipolar constraint). *Every correspondence of a rigid point satisfies*

$$\bar{x}_2^{\mathsf{T}}\,E\,\bar{x}_1 = 0, \qquad E = [t]_\times R. \tag{14.2}$$

Proof. Cross both sides of $X_2 = RX_1 + t$ with t: $[t]_\times X_2 = [t]_\times RX_1$. The left side is orthogonal to X_2, so $X_2^{\mathsf{T}}[t]_\times RX_1 = 0$. Dividing by $Z_1Z_2 > 0$ gives (14.2). ∎

Geometrically, the two camera centers and the 3D point span a plane, the *epipolar plane*, and the constraint states that the two viewing rays and the baseline are coplanar (Figure 14.1). Given $\bar{x}_1$, the corresponding point in the second image must lie on the *epipolar line* $\ell_2 = E\bar{x}_1$. The constraint involves no depth: it is a statement about bearings only, which is why it is the natural tool for checking tracks before any 3D reconstruction exists.

Proposition 14.2 (Properties of the essential matrix). *$E = [t]_\times R$ has rank two (for $t \neq 0$), two equal non-zero singular values, and five degrees of freedom: three for the rotation and two for the direction of t. Its overall scale, like the scale of t, is unobservable.*

In pixel coordinates the same constraint reads $\tilde{u}_2^{\mathsf{T}} F \tilde{u}_1 = 0$ with the *fundamental matrix* $F = K_2^{-\mathsf{T}} E K_1^{-1}$. The project works with undistorted normalized coordinates throughout, so it uses E directly (Milestone 9).

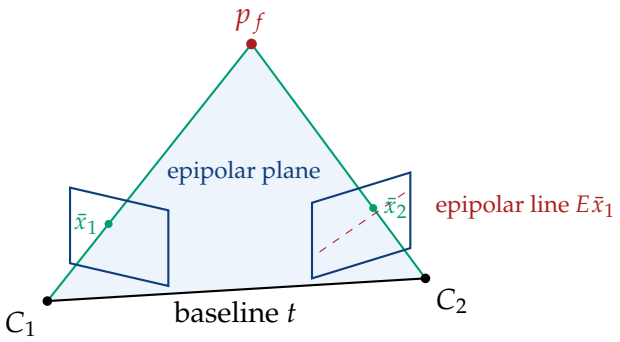


Figure 14.1: Epipolar geometry. The camera centers, the point and both viewing rays lie in one plane. The image of the point in the second camera must therefore lie on the line where that plane cuts the second image.

14.2 Estimating relative motion

The eight-point algorithm. Equation (14.2) is linear in the nine entries of E. Each correspondence gives one equation $a_i^\top \operatorname{vec}(E) = 0$, where a_i contains the products of the coordinates of $\bar{x}_1$ and $\bar{x}_2$. With eight or more correspondences, stack them into $A \operatorname{vec}(E) = 0$ and take the right singular vector of A with the smallest singular value. The result is then projected onto the set of valid essential matrices by setting its singular values to $(1, 1, 0)$. Finally R and the direction of t are extracted by a second SVD; of the four candidate solutions, the one that places triangulated points in front of both cameras is kept.

The five-point algorithm. Five correspondences are the minimum, since E has five degrees of freedom. Nistér's algorithm [38] solves the resulting polynomial system and returns up to ten candidates. Fewer points per hypothesis means fewer RANSAC iterations, which is why `cv::findEssentialMat` uses it.

The two-point algorithm with known rotation. In VIO the gyroscope supplies the relative rotation R between consecutive frames with high accuracy. With R known, (14.2) becomes linear in t:

$$\bar{x}_2^\top [t]_\times R \bar{x}_1 = t^\top \left(R\bar{x}_1 \times \bar{x}_2 \right) = 0. \tag{14.3}$$

Each correspondence says that t is orthogonal to the vector $c_i = R\bar{x}_1^{(i)} \times \bar{x}_2^{(i)}$, so two correspondences determine the direction of t as $c_1 \times c_2$. A two-point hypothesis needs far fewer RANSAC iterations than a five-point one, and it is the outlier-rejection strategy of MSCKF-VIO [48].

Inlier distance. The algebraic residual $\bar{x}_2^\top E \bar{x}_1$ has no geometric meaning. The *Sampson distance*

$$d_S = \frac{(\bar{x}_2^\top E \bar{x}_1)^2}{(E\bar{x}_1)_1^2 + (E\bar{x}_1)_2^2 + (E^\top \bar{x}_2)_1^2 + (E^\top \bar{x}_2)_2^2} \tag{14.4}$$

is a first-order approximation of the squared reprojection distance, in the units of the coordinates used. With normalized coordinates, a pixel threshold τ_{px} must be converted to τ_{px}/f (Milestone 9).

Pitfall: thresholds in the wrong units

Calling `cv::findEssentialMat` on normalized coordinates with an identity camera matrix and a threshold of "1.0" accepts residuals of 1 normalized unit, about 450 pixels: every track becomes an inlier. The threshold must be τ_{px}/f, e.g. $1/458 \approx 2.2 \times 10^{-3}$.

14.3 RANSAC

Random Sample Consensus [14] estimates a model from data containing outliers by hypothesis and verification.

Algorithm 3 RANSAC for relative motion

Input: correspondences, minimal sample size s, threshold τ, confidence p

best $\leftarrow \emptyset$
for $k = 1, \ldots, N$ **do**
    draw s correspondences at random; fit candidate models
    count inliers with distance $< \tau$; keep the model with the most inliers
    update N from the current inlier ratio w with (14.5)
refit the model on all inliers of the best hypothesis

If a fraction w of the data are inliers, a random minimal sample is outlier-free with probability w^s. To draw at least one clean sample with probability p, the number of iterations must be

$$N = \frac{\log(1-p)}{\log(1-w^s)}. \tag{14.5}$$

$p = 0.99$	$w = 0.9$	$w = 0.7$	$w = 0.5$
8-point ($s = 8$)	9	78	1177
5-point ($s = 5$)	6	26	146
2-point ($s = 2$)	3	7	17

The exponential dependence on s is why minimal solvers matter, and why using the gyroscope to remove three unknowns pays off.

14.4 Degeneracies

14.4.1 Pure rotation

If the camera only rotates, $t = 0$ and $E = [0]_\times R = 0$: the epipolar constraint is satisfied by every pair of points, inlier or not. Under small translation the problem is not degenerate but badly conditioned: t is poorly determined, inlier sets fluctuate, and RANSAC may reject good tracks or accept bad ones. Hovering drones rotate in place often, so this case must be handled (Milestone 9).

Under pure rotation the correspondences satisfy a stronger constraint, $\bar{x}_2 \propto R\bar{x}_1$ (the *infinite homography*), which does not involve t at all. A robust front end measures the parallax

remaining after rotation compensation, for example the median of $\|\pi(R\bar{x}_1) - \pi(\bar{x}_2)\|$ over the tracks:

- if it is large, run essential-matrix (or two-point) RANSAC;
- if it is tiny, skip epipolar RANSAC and reject only tracks that violate the rotation-only prediction $\bar{x}_2 \propto R\bar{x}_1$, with R from the gyroscope.

Either way the system keeps tracking: low parallax means the features carry little information about translation, not that they are wrong. The MSCKF update handles them gracefully, because their nullspace-projected residuals simply contribute little.

14.4.2 Other degeneracies

The eight-point algorithm is degenerate when all points lie on a plane, a common situation (floors, walls); the five-point algorithm is not. Very distant points behave like points at infinity and carry only rotational information. Moving objects form a second, consistent rigid motion that RANSAC can mistake for the dominant one if they fill the image.

14.5 Triangulation

Given observations z_i of a point in cameras with known poses (R_{wc_i}, p_{wc_i}), triangulation estimates its world position p_f.

14.5.1 Linear triangulation

In camera i the point has coordinates $X_i = R_{wc_i}^{\mathsf{T}}(p_f - p_{wc_i})$, which must be parallel to the bearing $\bar{x}_i = (z_i, 1)$: $[\bar{x}_i]_\times X_i = 0$. Substituting,

$$[\bar{x}_i]_\times R_{wc_i}^{\mathsf{T}}\, p_f = [\bar{x}_i]_\times R_{wc_i}^{\mathsf{T}}\, p_{wc_i}, \qquad i = 1, \dots, M. \tag{14.6}$$

Each view contributes three equations of rank two. Stacking all views gives an overdetermined linear system $Ap_f = b$ that is solved in the least-squares sense with QR or SVD. With two views and noise-free data the solution is exact. This is the linear initialization of AGENTS.md F5. It minimizes an algebraic error, not the reprojection error, so it is refined next.

14.5.2 Nonlinear refinement with inverse depth

The maximum-likelihood estimate minimizes the reprojection error

$$\min_{p_f} \sum_{i=1}^{M} \left\| z_i - \pi(X_i(p_f)) \right\|^2. \tag{14.7}$$

The MSCKF [37] parameterizes the point in an *anchor* camera A (for example the first observation) by inverse depth [9]:

$$X_A = \frac{1}{\rho}\begin{bmatrix}\alpha\\ \beta\\ 1\end{bmatrix}, \qquad X_i = R_{iA}X_A + p_{iA} \quad \Longrightarrow \quad h_i := \rho X_i = R_{iA}\begin{bmatrix}\alpha\\ \beta\\ 1\end{bmatrix} + \rho\, p_{iA}, \quad z_i = \pi(h_i), \tag{14.8}$$

where (R_{iA}, p_{iA}) is the transformation from the anchor camera to camera i, computed from the clone poses, and $\pi(h_i) = \pi(X_i)$ because projection ignores scale. The parameters (α, β) are the normalized coordinates in the anchor view and $\rho = 1/Z_A$ is the inverse depth. This parameterization behaves well for distant points ($\rho \to 0$ is a regular point), and the measurement function is nearly linear in (α, β, ρ). Its Jacobian follows from the chain rule:

$$\frac{\partial z_i}{\partial(\alpha, \beta, \rho)} = \left.\frac{\partial \pi}{\partial h}\right|_{h_i} \begin{bmatrix} r_1 & r_2 & p_{iA}\end{bmatrix}, \tag{14.9}$$

where r_1, r_2 are the first two columns of R_{iA} and $\partial\pi/\partial h$ is Proposition 13.1. Gauss–Newton iterates $\delta y = -(J^\mathsf{T}J)^{-1}J^\mathsf{T}e$ with the stacked Jacobian J and residual e, usually converging in a few iterations from the linear initialization. Levenberg–Marquardt damping adds robustness.

14.5.3 How accurate is triangulation?

For two views separated by a baseline b perpendicular to the viewing direction, the depth is $Z = b/d$ with d the disparity in normalized coordinates. With independent noise σ_{uv} in each view, $\sigma_d = \sqrt{2}\sigma_{uv}$ and to first order

$$\sigma_Z \approx \frac{Z^2}{b}\sqrt{2}\,\sigma_{uv}. \tag{14.10}$$

Depth error grows quadratically with depth and inversely with baseline. Equivalently, the relative depth error is inversely proportional to the parallax angle $\approx b/Z$.

Figure 14.2 shows that 10% depth accuracy needs about 2° of parallax, a baseline of 17 cm at 5 m. Below about 1° the linearization behind (14.10) breaks down entirely. A triangulation module must therefore check its own result before handing it to the filter:

- positive depth in every view (cheirality);
- sufficient parallax (or baseline relative to depth), or equivalently a well-conditioned Gauss–Newton Hessian;
- small reprojection error in every view after refinement;
- a plausible depth range.

Features that fail are discarded, not fed to the update (AGENTS.md F5, Milestone 10b).

Key takeaways

- Rigid correspondences satisfy $\bar{x}_2^\mathsf{T}[t]_\times R\bar{x}_1 = 0$. E has 5 DOF and unknown scale.
- With the rotation from the gyroscope, two correspondences determine the translation direction.

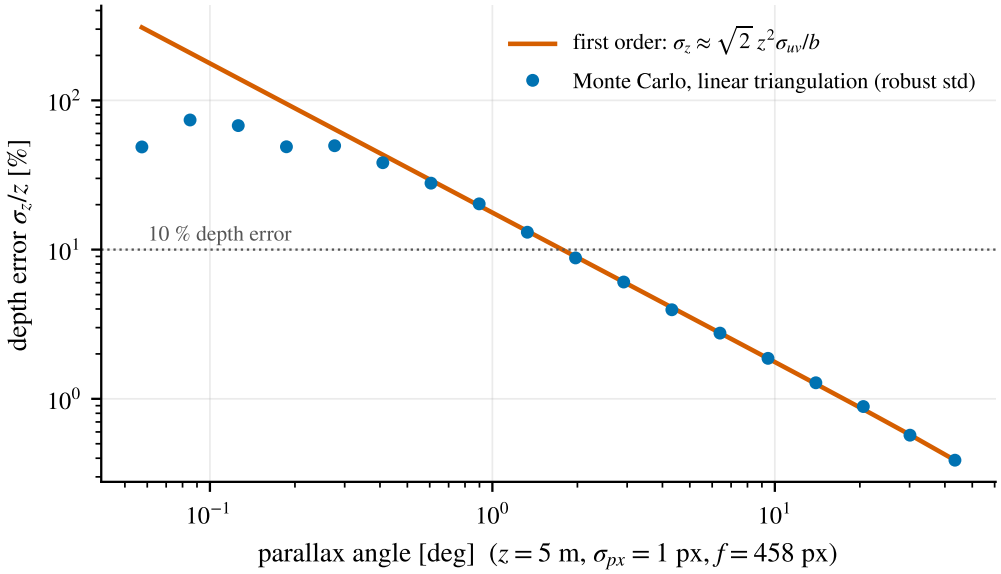


Figure 14.2: Relative depth error of two-view triangulation at $Z = 5$ m with 1-pixel noise, versus parallax angle. Above about 2° the first-order formula (14.10) is accurate. Below about 1° the error distribution becomes heavy-tailed (depths can even become negative), and the estimate is unusable.

- RANSAC iterations grow as w^{-s}: prefer minimal solvers. Thresholds in normalized units are τ_{px}/f.
- Pure rotation makes E vanish. Detect low parallax and fall back to rotation-only consistency checks.
- Triangulate linearly, refine by Gauss–Newton with inverse depth, and reject low-parallax or negative-depth points.
- Depth error grows as Z^2/b.

Exercises

14.1 Verify Theorem 14.1 numerically for random points and a random relative pose, then show that adding an outlier breaks it.

14.2 Show that $[t]_\times R$ has singular values $(\|t\|, \|t\|, 0)$.

14.3 Implement the two-point method with known rotation inside RANSAC on synthetic data with 30% outliers. Compare the number of iterations needed with the five-point method.

14.4 Generate a pure-rotation sequence, observe what `cv::findEssentialMat` returns, and implement the parallax test.

14.5 Show that (14.6) has rank two per view and that two views with non-parallel rays give a unique solution.

14.6 Derive (14.10), reproduce Figure 14.2, and repeat with three views over the same

baseline.

Chapter 15

The Visual Front End

In this chapter

- Understand the structure tensor and why it defines both good corners and solvable tracking.
- Derive Lucas–Kanade optical flow from brightness constancy, including the aperture problem and pyramids.
- Build a robust tracking pipeline: bucketing, forward-backward checks, IMU-aided prediction and RANSAC.
- Manage feature tracks through their life cycle with bounded memory.

The front end turns a stream of images into *feature tracks*: for each tracked scene point, the list of its undistorted normalized coordinates in the images where it was seen, with the image timestamps. Everything the MSCKF knows about the scene comes from these tracks, so their quality bounds the quality of the estimator.

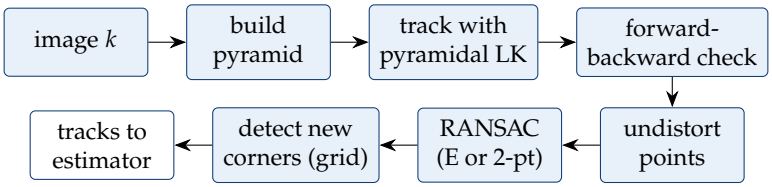


Figure 15.1: A KLT-based front end, as used by MSCKF-style estimators.

15.1 What makes a good feature

Consider shifting a small window W of the image I by a displacement d. The sum of squared differences is, to second order,

$$E(d) = \sum_{x \in W} \big(I(x+d) - I(x)\big)^2 \approx \sum_{x \in W} \big(\nabla I(x)^\mathsf{T} d\big)^2 = d^\mathsf{T} M\, d, \qquad M = \sum_{x \in W} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}, \tag{15.1}$$

where I_x, I_y are the image gradients. M is the *structure tensor* (second-moment matrix) of the window. Its eigenvalues $\lambda_1 \geq \lambda_2 \geq 0$ describe how distinctive the window is.

- Both small: the window is *flat*; it looks the same after any small shift.
- One large, one small: an *edge*; shifting along the edge changes nothing.
- Both large: a *corner*; every shift changes the window, so its position is well defined in 2D.

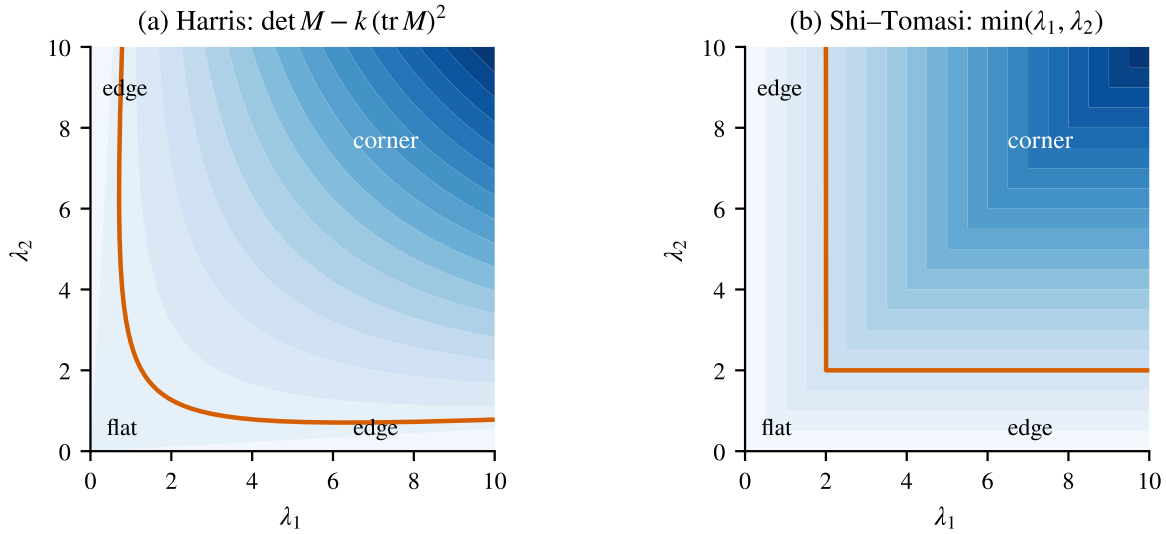


Figure 15.2: Corner scores over the plane of structure-tensor eigenvalues, with a threshold contour (orange). Both scores accept only windows with two large eigenvalues. Shi–Tomasi's $\min(\lambda_1, \lambda_2)$ is exactly the quantity that makes the Lucas–Kanade system well conditioned.

The Harris detector [21] scores $\det M - k(\operatorname{tr} M)^2$ to avoid computing eigenvalues; the Shi–Tomasi detector [42] scores $\lambda_2 = \min(\lambda_1, \lambda_2)$ directly (Figure 15.2). The two behave similarly; Shi–Tomasi has a deeper justification, which the next section reveals.

15.2 Lucas–Kanade optical flow

15.2.1 Brightness constancy and the aperture problem

Assume that a scene point keeps its brightness between consecutive frames: $I_{k+1}(x+d) = I_k(x)$. Linearizing in d,

$$I_{k+1}(x) + \nabla I^{\mathsf{T}} d \approx I_k(x) \quad \Longleftrightarrow \quad I_x\, d_x + I_y\, d_y + I_t = 0, \qquad I_t = I_{k+1}(x) - I_k(x). \tag{15.2}$$

This optical-flow constraint is one equation in two unknowns. It determines only the component of d along the gradient: motion along an edge is invisible. This is the *aperture problem*, the same phenomenon seen through a small hole when a long line slides past.

15.2.2 The Lucas–Kanade solution

Lucas and Kanade [33] assume the displacement is constant over a small window and solve (15.2) for all its pixels in the least-squares sense:

$$\min_{d} \sum_{x \in W} \big(\nabla I(x)^{\mathsf{T}} d + I_t(x)\big)^2 \quad \Longrightarrow \quad M\, d = -\sum_{x \in W} I_t(x) \nabla I(x), \tag{15.3}$$

with the structure tensor M of (15.1) as the normal matrix. The system is solvable and well conditioned exactly when both eigenvalues of M are large compared with the image

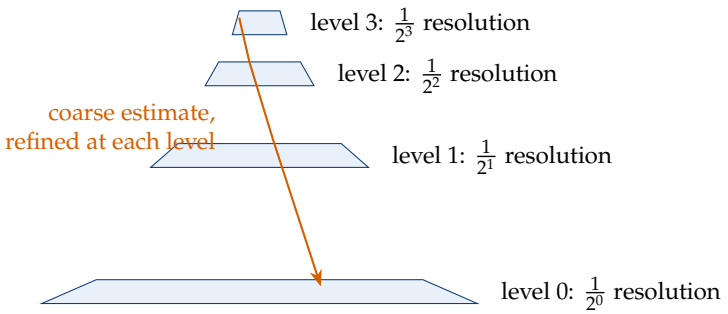


Figure 15.3: An image pyramid. A displacement that is too large to linearize at full resolution is small at the top level; each level doubles the estimate and refines it.

noise. Shi and Tomasi's insight was to select features by the very criterion that makes them trackable: a "good feature to track" is a window with a large $\lambda_{\min}(M)$.

Because (15.3) rests on a linearization, it is iterated. Warp the second image by the current estimate, recompute I_t, solve for an increment, and repeat until the increment is a small fraction of a pixel. This is Gauss–Newton on the sum of squared differences [1]. Sub-pixel accuracy comes from bilinear interpolation of the warped image.

15.2.3 Pyramids

The linearization is valid only for displacements smaller than the scale over which the image gradient is roughly constant, about the window radius. Fast motion violates this. The remedy is a Gaussian *pyramid*: the image is repeatedly blurred and down-sampled by two (Figure 15.3). Tracking starts at the coarsest level, where a 40-pixel motion becomes a 5-pixel motion, and the estimate is doubled and refined at each finer level [6]. With L levels above the base, the trackable displacement grows by about 2^L.

15.3 Making tracking robust

Optical flow optimizes photometric similarity, not physical correctness. It will track the shadow of a moving object, a specular highlight, or a T-junction formed by two objects at different depths. Several layers of checks remove such tracks.

1. **Forward-backward consistency.** Track each point from frame k to $k+1$, then back to k. A correct track returns to its starting point; reject it if the round-trip error exceeds a fraction of a pixel.
2. **IMU-aided prediction.** The gyroscope predicts the rotation between frames. Predicting each feature's new position as $\pi(R_{c_{k+1}c_k}\bar{x}_k)$ (exact for distant points) gives LK a much better starting point and allows smaller windows and fewer pyramid levels.
3. **Geometric verification.** Reject tracks that violate the epipolar constraint, with essential-matrix or two-point RANSAC, or the rotation-only constraint under low parallax (Chapter 14).

4. **Statistical gating in the filter.** Tracks that pass all of the above are still gated by the χ^2 test of the MSCKF update (Chapter 18).

15.3.1 Feature distribution

Asking a detector for the 200 strongest corners typically returns clusters on the most textured object in view. Clustered features constrain the motion poorly: features in a small patch of the image make rotation and translation hard to separate. The standard remedy is *grid bucketing*: divide the image into cells (for example 8×6) and cap the number of features per cell, plus a minimum distance between features. New features are detected only in cells that have fallen below their quota, using a mask around existing tracks. A uniform distribution of bearings makes the stacked measurement Jacobian better conditioned, which is exactly what the estimator needs.

15.4 The feature life cycle

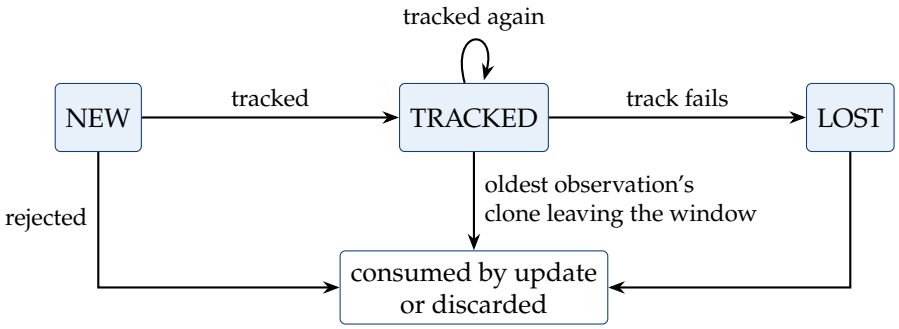


Figure 15.4: Feature life cycle. A track is handed to the MSCKF update when it is lost, or when its oldest observation is about to leave the sliding window (AGENTS.md F5). Its observations are used once and then discarded.

Each feature has a persistent ID and a history of observations: (clone index or image timestamp, normalized coordinates). Since the estimator can only use observations made at times that still have a clone in the window, the history never needs to exceed `kMaxClones` entries. That makes it possible to store all tracks in a fixed-capacity pool with fixed-capacity histories, so the steady state of the front end performs no allocation for its bookkeeping (Milestone 9).

Pitfall: using a measurement twice

Once a track's observations have been used in an MSCKF update, they must never be used again, not even partially. Re-using them double counts the same information and makes the filter overconfident. If a track continues after being used (because its oldest clone was marginalized), its subsequent observations start a fresh track.

15.5 Engineering the front end

The front end is the one component allowed to allocate (AGENTS.md E4), because OpenCV allocates internally, but it must reuse its buffers. In practice: keep the previous pyramid and swap it with the new one instead of rebuilding storage; preallocate point vectors with `reserve`; keep the `cv::Mat` objects as members so that their memory is reused when sizes do not change; and *measure* the remaining allocations with the allocation guard rather than claiming they are zero. Feature tracking is typically the most expensive part of the pipeline in time, too, so bucketing parameters, window size and pyramid depth are the main performance knobs.

Descriptor-based front ends (ORB, BRIEF) match features across frames by appearance instead of tracking them. They handle large viewpoint changes and re-detection after occlusion, which matters for loop closure, but for frame-to-frame VIO at 20 Hz KLT tracking is cheaper and more accurate. Direct methods skip features altogether and use photometric error in the estimator itself.

Project link: Milestones 8 and 9

M8 builds detection, bucketing, pyramidal LK with forward-backward checks, persistent IDs and point undistortion, and tests them on synthetic images with known sub-pixel motion. M9 adds the life-cycle manager with a fixed-capacity pool and essential-matrix RANSAC in normalized coordinates, including the documented pure-rotation behavior.

Key takeaways

- The structure tensor M measures how well a window's position is defined; corners have two large eigenvalues.
- Lucas–Kanade solves $Md = -\sum I_t \nabla I$: good corners are exactly the trackable windows.
- The aperture problem: a single pixel constrains only the flow along the gradient.
- Pyramids extend the trackable displacement; gyro prediction makes tracking cheaper and more robust.
- Bucketing spreads features for good geometry; forward-backward checks and RANSAC remove bad tracks.
- Tracks are consumed exactly once and their histories are bounded by the window size.

Exercises

15.1 Compute the structure tensor of an ideal step edge and of an ideal corner, and verify the eigenvalue picture.

15.2 Implement single-level LK for a synthetic textured image translated by $(0.3, -0.7)$ pixels. How many iterations until convergence to 0.01 px? What happens with a translation of 8 pixels, and how many pyramid levels fix it?

15.3 Show that the forward-backward error of a correct track is due only to noise and interpolation, and estimate a sensible threshold.

15.4 Implement grid bucketing and plot the feature distribution with and without it on a EuRoC image.

15.5 Design the data structure for the feature pool with fixed capacity and per-feature histories bounded by `kMaxClones`. What happens when the pool is full?

Part VI

The Multi-State Constraint Kalman Filter

Chapter 16

From EKF-SLAM to the MSCKF

In this chapter

- Understand how EKF-SLAM uses visual landmarks, and why it scales poorly.
- Understand the central idea of the MSCKF: constraints between multiple past poses instead of landmarks.
- See why past poses must be kept in the state, with their correlations.
- Relate the MSCKF to sliding-window optimization.

16.1 EKF-SLAM

The most direct way to use a camera in an EKF is to put the landmarks in the state. With L landmarks,

$$x = \left(x_I,\ p_{f_1},\ \ldots,\ p_{f_L}\right), \qquad \dim \delta x = 15 + 3L, \tag{16.1}$$

and each image observation $z = \pi(p_c(x_I, p_{f_j})) + n$ is an ordinary EKF measurement of the IMU state and one landmark. This is EKF-SLAM, and MonoSLAM [10] showed it can run in real time with a single camera. It has three structural problems.

1. **Cost.** The covariance has $(15 + 3L)^2$ entries, and every update touches all of them. Tracking 200 landmarks gives a 615×615 covariance; updating it at 20 Hz with hundreds of observations per frame is expensive, and the cost grows quadratically with the map.
2. **Initialization.** A single monocular observation says nothing about depth. A new landmark must be initialized with a huge depth uncertainty, which is highly non-Gaussian in Euclidean coordinates, or its initialization must be delayed until it has been seen from several viewpoints. Inverse-depth parameterizations [9] help, but add complexity.
3. **Housekeeping.** Landmarks that leave the field of view must either stay in the state, growing it, or be marginalized, which in covariance form is cheap but discards their future value.

16.2 The multi-state constraint

Consider a feature observed in M images taken at times $t_1, \ldots, t_M$. Each observation constrains the pose at its time and the feature position. Taken together, after the unknown feature position is eliminated, the $2M$ observations impose $2M - 3$ constraints *on the M poses alone*. This is the *multi-state constraint*: geometric information about how the camera moved between the images, with the scene point serving only as an intermediary.

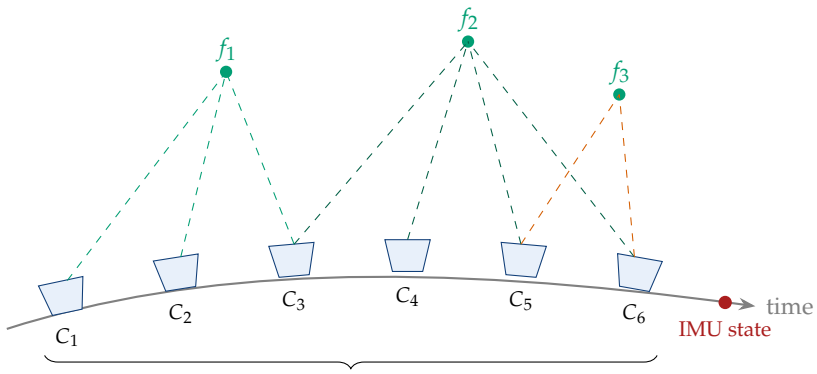


Figure 16.1: The MSCKF state: the current IMU state plus clones of past camera poses. Feature f_1 is seen by clones 1–3 and constrains them; f_2 by clones 3–6. Neither feature is in the state. Feature f_3 has only two observations so far and keeps being tracked.

Key idea: the MSCKF in one sentence

Keep a sliding window of past poses in the state instead of landmarks; when a feature track is complete, triangulate it, linearize its observations, and eliminate the feature by projecting onto the left nullspace of its Jacobian, leaving a residual that depends only on the poses; then update.

Mourikis and Roumeliotis [37] showed that this uses all the information that the observations carry about the poses (under linearization), at a cost linear in the number of features. Feature positions are never part of the state, so there is no landmark initialization problem inside the filter, and features leaving the view simply trigger an update.

16.3 Why past poses must be in the state

Why not keep only the current IMU state and use the constraint between the current pose and the past ones? Because the past poses are *estimated* quantities, with errors correlated with the current state's error: they were all produced by the same propagation from the same measurements. A measurement relating the current pose to a past pose estimate, which treats that past estimate as fixed or independent, double counts information and makes the filter overconfident.

The cure is *stochastic cloning* [40]. When an image is captured, a copy of the current pose (a clone) is appended to the state, together with its exact correlations to everything else. From then on the clone is a state variable whose estimate can be corrected by later measurements, and whose correlation with the evolving IMU state is propagated exactly. Chapter 17 develops this in detail.

16.4 The MSCKF state

The project's state (AGENTS.md C3) is the IMU state plus $N \le$ `kMaxClones` clones, each holding the IMU pose (p_{wb}, q_{wb}) at an image timestamp:

$$x = \left(p,\ v,\ q,\ b_a,\ b_g,\ \ p_1, q_1,\ \ldots,\ p_N, q_N\right), \qquad \dim \delta x = 15 + 6N. \tag{16.2}$$

The camera pose of clone i is derived through the fixed extrinsics, $R_{wc_i} = R_{wb_i} R_{bc}$ and $p_{wc_i} = p_{wb_i} + R_{wb_i} p_{bc}$. With `kMaxClones` = 10, the error state has 75 dimensions, independent of how many features are tracked.

16.5 Relation to sliding-window optimization

Optimization-based VIO (OKVIS [30], VINS-Mono [39]) keeps a similar window of poses and minimizes the sum of squared reprojection and IMU residuals over poses *and* landmarks, relinearizing at every iteration, then marginalizes old states into a Gaussian prior. The MSCKF update is equivalent to a single Gauss–Newton iteration of such a problem, with the landmarks eliminated by a Schur complement and the prior given by the filter's covariance. Eliminating a landmark by nullspace projection and by Schur complement give the same information about the poses (Theorem 18.1). The differences are that the MSCKF never relinearizes past measurements, and processes each feature track exactly once, which makes it cheaper and more predictable, at some cost in accuracy under strong nonlinearity.

16.6 Variants

The basic MSCKF has been refined in many directions. MSCKF 2.0 [31] added First-Estimates Jacobians for consistency and online calibration of extrinsics and time offset. Hybrid filters keep long-lived features in the state as SLAM landmarks while processing short tracks MSCKF-style. MSCKF-VIO [48] uses a stereo camera and two-point RANSAC for fast flight, and OpenVINS [17] is a modular research platform that implements most of these options and documents them carefully.

Key takeaways

- EKF-SLAM keeps landmarks in the state; its cost grows quadratically with the map.
- The MSCKF keeps past poses (clones) instead; each feature track is turned into constraints on those poses.
- Past poses must be in the state, with their correlations, or information is double counted.
- The MSCKF update is equivalent to one Gauss–Newton step of sliding-window bundle adjustment with landmarks marginalized.

Exercises

16.1 Estimate the memory and the per-update flop count of EKF-SLAM with 200 landmarks and of the MSCKF with 10 clones.

16.2 A feature is observed in M images. How many scalar constraints does it impose on the poses? Why does the count start being useful only at $M = 2$, and why does the project require $M \geq 3$?

16.3 Explain in your own words why using a past pose estimate as if it were known exactly makes the filter overconfident. Construct a scalar example.

Chapter 17

Stochastic Cloning and the Sliding Window

In this chapter

- See, on a scalar example, why relative measurements require cloned states with correlations.
- Derive the general state-augmentation formula $P \leftarrow JPJ^{\mathsf{T}}$.
- Derive the project's cloning Jacobian and the invariance property of cloning.
- Marginalize the oldest clone in place without allocation, and choose a window policy and size.

17.1 A scalar example

Let x be a scalar position that performs a random walk, $x_{k+1} = x_k + w_k$ with $\mathrm{Var}(w_k) = Q$, and suppose its current estimate has variance P. At time 1 we *clone* it, $x_c := x_1$; between times 1 and 2 the state moves; then a sensor measures the displacement:

$$z = x_2 - x_c + n, \qquad \mathrm{Var}(n) = R. \tag{17.1}$$

Augmenting the state with the clone gives the vector (x, x_c). At the moment of cloning both entries are the *same* random variable, so their joint covariance is $\begin{bmatrix} P & P \\ P & P \end{bmatrix}$, which is singular. Propagating x but not x_c gives

$$P_{\text{aug}} = \begin{bmatrix} P+Q & P \\ P & P \end{bmatrix}, \qquad H = \begin{bmatrix} 1 & -1 \end{bmatrix}. \tag{17.2}$$

The innovation covariance and gain are

$$S = (P+Q) - 2P + P + R = Q + R, \qquad K = \frac{1}{Q+R}\begin{bmatrix} Q \\ 0 \end{bmatrix}. \tag{17.3}$$

The result is exactly right: a displacement measurement is informative only about what happened *between* the two times, so S involves only Q and R. The update corrects the current state by the part of its uncertainty that arose after the clone, and leaves the clone untouched. The posterior variance of x_2 is $P + Q - Q^2/(Q+R)$, which stays large if P is large, as it must: a relative measurement cannot reveal absolute position.

Now suppose we had treated the past estimate as known (no clone, zero variance). Then $S = P + Q + R$ and the posterior variance of x_2 would be $(P+Q)R/(P+Q+R)$, which tends

to R when P is large. The filter would believe it had learned the absolute position to within the sensor noise from a displacement measurement: a severe overconfidence. Treating the past estimate as an independent random variable is wrong in a different way. Only the exact cross-covariance gives the right answer, and stochastic cloning [40] is the mechanism that keeps it.

17.2 State augmentation

Proposition 17.1 (Augmentation with a function of the state)**.** *Let x have error covariance P, and append a new variable $y = g(x)$ whose error satisfies $\delta y \approx J_g\,\delta x$. The augmented error $(\delta x, \delta y) = J\,\delta x$ with $J = \begin{bmatrix} I \\ J_g \end{bmatrix}$ has covariance*

$$P \leftarrow JPJ^\mathsf{T} = \begin{bmatrix} P & PJ_g^\mathsf{T} \\ J_gP & J_gPJ_g^\mathsf{T} \end{bmatrix}. \tag{17.4}$$

Proof. Proposition 3.1 with the linear map J. ∎

The augmented covariance has rank $\dim x$: it is singular, because y is a deterministic function of x at the moment of augmentation. This is correct and harmless. Propagation adds process noise to the IMU block only, after which the clone and the current state are no longer perfectly correlated. The Joseph form and symmetrization keep the matrix positive semi-definite throughout.

17.3 The project's clone

At an image timestamp, after propagating the IMU state to exactly that time, the filter appends a clone holding the current pose (p_{wb}, q_{wb}), and augments the covariance with the cloning Jacobian (AGENTS.md F5).

Derivation exercise (project Jacobian): the cloning Jacobian J

The clone's error is defined with the same retraction as the IMU pose (AGENTS.md C4): $p_c = \hat{p}_c + \delta p_c$ and $q_c = \hat{q}_c \otimes \mathrm{Exp}_q(\delta\theta_c)$, with $\hat{p}_c = \hat{p}$ and $\hat{q}_c = \hat{q}$ at cloning time.

1. At the moment of cloning the true clone equals the true IMU pose. Express δp_c and $\delta\theta_c$ in terms of the IMU error state. Is any linearization needed?
2. Write $J_g \in \mathbb{R}^{6\times(15+6N)}$ and then $J \in \mathbb{R}^{(21+6N)\times(15+6N)}$, with the new clone appended at the end of the state.
3. What would change if the clone stored the *camera* pose (p_{wc}, q_{wc}) instead of the IMU pose? (The project deliberately does not do this.)

Check: Milestone 10a requires that after cloning, the new clone's covariance block and its cross-covariances equal the corresponding IMU pose blocks *exactly*, and that a run with cloning enabled but only pose updates reproduces the IMU estimate and covariance of a run without cloning to 10^{-10}.

The worked solution is omitted from this edition; build without `NoSolutions` *to include it.*

Proposition 17.2 (Cloning does not change the IMU marginal)**.** *Augmentation, propagation with static clones, and marginalization of clones leave the marginal distribution of the IMU state unchanged, provided no measurement involves the clones.*

Proof. Augmentation leaves the P_{II} block unchanged by (17.4). Propagation updates P_{II} by $\Phi P_{II}\Phi^\mathsf{T} + Q_d$ regardless of the clones. Marginalization deletes clone rows and columns. A measurement that involves only the IMU state has a Jacobian with zero clone columns; its gain for the IMU block, $P_{II}H_I^\mathsf{T}S^{-1}$, and $S = H_I P_{II} H_I^\mathsf{T} + R_z$ do not involve the clones either. ■

This is the invariance test of Milestone 10a. It is a powerful end-to-end check of the augmentation, propagation and marginalization code, because any indexing error in the clone blocks leaks into the IMU block and breaks the equality.

17.4 Clones in propagation and updates

Clones are static (Chapter 10): propagation multiplies their cross-covariance with the IMU state by Φ and leaves the clone-clone block alone. Visual measurements depend only on clone states, so their Jacobians are zero in the IMU columns. The update nevertheless corrects the IMU state, through the gain rows $P_{IC}H_C^\mathsf{T}S^{-1}$: information about relative motion between past poses corrects the current velocity, attitude and biases through the correlations built up by propagation. Injection and reset (Section 7.6) apply to every clone, each with its own $I - \frac{1}{2}[\delta\hat{\theta}_i]_\times$ block.

17.5 Marginalizing the oldest clone

When the window holds `kMaxClones` clones and a new image arrives, the oldest clone must go. In covariance form, marginalization is deletion of its six rows and columns (Theorem 3.3). Before deleting it, the filter must process every feature that has an observation at that clone (AGENTS.md F5), because those observations become unusable once the clone is gone.

Implementation note: in-place deletion without allocation

P is stored in a fixed-max-size matrix whose active part is the top-left $n \times n$ corner. Deleting the block at offset o means shifting every row below it up by six and every column to its right left by six, then reducing n by six. The source and destination regions overlap. Copying toward lower indices in ascending order is safe, because each element is read before it is overwritten (`std::copy` permits overlapping ranges when the destination begins before the source), but an Eigen block assignment between overlapping blocks of the same matrix is an aliasing hazard whose traversal order is not guaranteed. Write the shift explicitly, or through a preallocated member workspace, and test it against a reference built with a dynamic matrix (Milestone 10a).

The clone states themselves are stored in a fixed-capacity container, such as a ring buffer of N_{max} poses with timestamps, so that removing the oldest is a pointer move. If the covariance blocks are kept in the same order as the container, a ring-buffer layout could even avoid the shift, at the cost of more complex index arithmetic. Simplicity is usually the better choice here: the shift costs $O(n^2)$ once per image.

17.6 Window policy and size

The project uses the simplest policy: first in, first out. Each image creates a clone, and the oldest is marginalized when the window is full. More elaborate policies keep clones that are geometrically useful: if the vehicle barely moved since the previous image, marginalize the second-newest clone instead of the oldest, so that the window spans a longer baseline.

The window size trades accuracy for cost.

- **Longer windows** let longer tracks be used in full, with more parallax, and give each update more constraints.
- **Cost** grows as $O(n^3)$ per update with $n = 15 + 6N$, and as $O(n^2)$ per IMU step for the cross-covariance propagation.
- **Memory**: a fixed-max covariance of doubles occupies $8(15 + 6N_{max})^2$ bytes: 45 kB for $N_{max} = 10$, 146 kB for 20, 304 kB for 30. Workspaces for H, S and K add similar amounts.

Pitfall: the stack budget

Fixed-max-size Eigen matrices store their maximum-size buffer *inline*, with no heap allocation. That is what makes them allocation-free, but it also means a filter object constructed on the stack carries all of its matrices on the stack. With the default 1 MB main-thread stack on Windows, a filter with a few hundred kilobytes of workspaces can overflow it. Construct the filter once, during the load phase, as a heap object (for example with `std::make_unique`), which the memory tiers allow, and keep the large matrices as members rather than function-local temporaries (AGENTS.md E4, Milestone 10a).

Key takeaways

- A relative measurement between a past and a present state needs the past state in the filter, with its exact correlations. Treating the past estimate as known makes the filter overconfident.
- Augmentation with $y = g(x)$ uses $P \leftarrow JPJ^\mathsf{T}$, $J = [I; J_g]$; the result is singular at first and becomes regular after propagation.
- Cloning, static propagation and marginalization leave the IMU marginal unchanged; this is a strong test.
- Marginalizing a clone deletes its rows and columns, after first processing its features. Shift in place carefully.
- Fixed-max matrices live inline: keep them as members of a heap-allocated filter.

Exercises

17.1 In the scalar example, compute the posterior covariance of (x_2, x_c) and verify that the clone's variance is unchanged. When would a relative measurement change the clone's estimate?

17.2 Extend the scalar example to two clones and two displacement measurements, and show that processing them gives the same result as one measurement of $x_2 - x_{c_1}$ followed by one of $x_{c_2} - x_{c_1}$, if correlations are kept.

17.3 Implement in-place deletion of a 6×6 block from a 75×75 symmetric matrix stored in a larger buffer, and test it against a reference.

17.4 For $N_{max} = 10, 20, 30$, estimate the flop count of one MSCKF update with 50 features of 6 observations each, with and without QR compression (Chapter 18).

Chapter 18

The MSCKF Visual Update

In this chapter

- Build the linearized measurement model of a feature track over the clones that observed it.
- Eliminate the feature by left-nullspace projection, and prove that no information about the poses is lost.
- Compute the projection efficiently with Householder reflections.
- Gate features, stack them, compress the stack with QR, and apply the update to the IMU state and all clones.

18.1 A feature track

A feature j has been observed at the clones $\mathcal{S}_j = \{i_1, \dots, i_M\}$, with undistorted normalized observations $z_{j,i}$. The update is triggered when the track is lost or when its oldest observation's clone is about to be marginalized, and requires $M \geq 3$ (AGENTS.md F5). The first step is to estimate the feature position $\hat{p}_f$ in the world frame by triangulation from the clone camera poses

$$\hat{R}_{wc_i} = \hat{R}_{wb_i} R_{bc}, \qquad \hat{p}_{wc_i} = \hat{p}_{wb_i} + \hat{R}_{wb_i} p_{bc}, \tag{18.1}$$

using the linear initialization and Gauss–Newton refinement of Section 14.5, with the positive-depth, parallax and reprojection checks. Features that fail are discarded.

18.2 Residuals and Jacobians

For each observation, the predicted measurement uses the clone estimate and the triangulated point,

$$\hat{p}_{c_i} = R_{bc}^{\mathsf{T}} \big(\hat{R}_{wb_i}^{\mathsf{T}} (\hat{p}_f - \hat{p}_{wb_i}) - p_{bc} \big), \qquad r_{j,i} = z_{j,i} - \pi(\hat{p}_{c_i}). \tag{18.2}$$

Linearizing with respect to the error state δx (IMU and clones) and the feature position error $\delta p_f = p_f - \hat{p}_f$,

$$r_{j,i} \approx H_{x,ji}\, \delta x + H_{f,ji}\, \delta p_f + n_{j,i}, \qquad n_{j,i} \sim \mathcal{N}(0, \sigma_{uv}^2 I_2). \tag{18.3}$$

The structure of these Jacobians follows from the chain rule

$$H_{x,ji} = \frac{\partial \pi}{\partial p_c}\Big|_{\hat{p}_{c_i}} \frac{\partial p_{c_i}}{\partial \delta x}, \qquad H_{f,ji} = \frac{\partial \pi}{\partial p_c}\Big|_{\hat{p}_{c_i}} \frac{\partial p_{c_i}}{\partial p_f}, \tag{18.4}$$

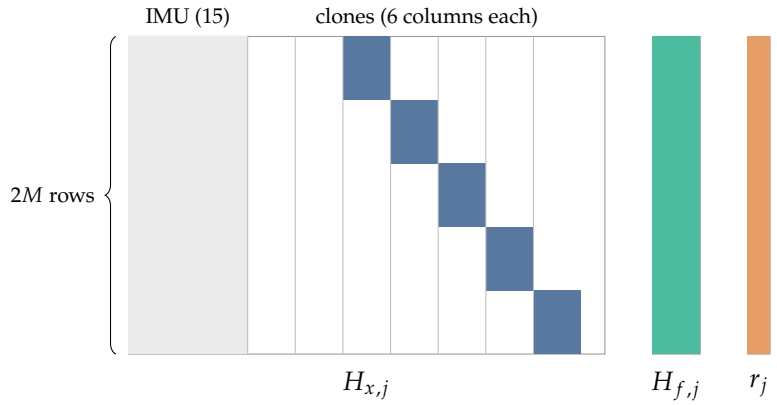


Figure 18.1: Sparsity of the stacked feature Jacobian for a feature seen by five consecutive clones. Each pair of rows depends on one clone; the IMU columns are zero. $H_{f,j}$ is dense.

with the projection Jacobian of Proposition 13.1. The observation at clone i depends only on that clone's pose, so $H_{x,ji}$ is zero except in the six columns of $(\delta p_i, \delta\theta_i)$. In particular it is zero in all IMU columns.

Derivation exercise (project Jacobian): the visual measurement Jacobians $H_{x,ji}$ and $H_{f,ji}$

With $p_{c_i} = R_{bc}^{\mathsf{T}}\big(R_{wb_i}^{\mathsf{T}}(p_f - p_{wb_i}) - p_{bc}\big)$, the local perturbation $R_{wb_i} = \hat{R}_{wb_i}\,\mathrm{Exp}(\delta\theta_i)$ and $p_{wb_i} = \hat{p}_{wb_i} + \delta p_i$:

1. Derive $\partial p_{c_i}/\partial\delta\theta_i$ with the inverse-action row of Table 5.1. Be careful about which vector the rotation acts on.
2. Derive $\partial p_{c_i}/\partial\delta p_i$ and $\partial p_{c_i}/\partial p_f$. How are they related, and why does that make physical sense?
3. Multiply by $\partial\pi/\partial p_c$ and place the blocks in the frozen column layout. State the sparsity pattern of the stacked $H_{x,j}$.

Check: Milestone 10b requires finite-difference tests of the projection Jacobians with respect to the clone δp, the clone $\delta\theta$ and p_f, at a non-trivial clone pose with non-trivial extrinsics.

The worked solution is omitted from this edition; build without `NoSolutions` *to include it.*

Stacking the M observations of feature j,

$$r_j \approx H_{x,j}\,\delta x + H_{f,j}\,\delta p_f + n_j, \qquad r_j, n_j \in \mathbb{R}^{2M}, \quad H_{x,j} \in \mathbb{R}^{2M\times(15+6N)}, \quad H_{f,j} \in \mathbb{R}^{2M\times 3}. \tag{18.5}$$

18.3 The problem with the feature

Equation (18.5) cannot be used directly in an update. The feature error δp_f is not part of the state, so there is nothing to correct. Worse, $\hat{p}_f$ was computed from the same measurements $z_{j,i}$ and the same clone estimates, so its error is correlated with both n_j and δx in a way the filter's covariance does not describe. Treating $\hat{p}_f$ as known would double count the measurements. EKF-SLAM resolves this by putting p_f in the state. The MSCKF eliminates it.

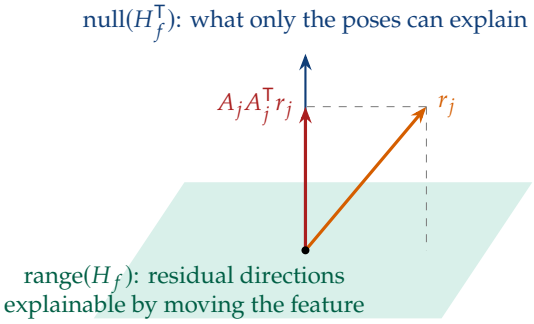


Figure 18.2: Geometry of the nullspace projection. Residual components inside the column space of H_f can be absorbed by an adjustment of the unknown feature position, so they say nothing about the poses. The component orthogonal to it, in the left nullspace, cannot be explained by any feature position and must be explained by the poses.

18.4 Left-nullspace projection

$H_{f,j}$ is a tall $2M \times 3$ matrix. If the feature is well triangulated it has full column rank 3, and its left nullspace has dimension $2M - 3$ (Section 2.2). Let $A_j \in \mathbb{R}^{2M\times(2M-3)}$ have orthonormal columns spanning null$(H_{f,j}^\top)$, so that $A_j^\top H_{f,j} = 0$ and $A_j^\top A_j = I$. Multiplying (18.5) by $A_j^\top$,

$$r_{o,j} := A_j^\top r_j \approx \underbrace{A_j^\top H_{x,j}}_{H_{o,j}}\,\delta x + \underbrace{A_j^\top H_{f,j}}_{=0}\,\delta p_f + A_j^\top n_j = H_{o,j}\,\delta x + n_{o,j}. \tag{18.6}$$

The feature has disappeared, and the projected noise is still white with the same variance: $\mathrm{Cov}(n_{o,j}) = \sigma_{uv}^2 A_j^\top A_j = \sigma_{uv}^2 I_{2M-3}$. Equation (18.6) is a standard EKF measurement of the state alone, of dimension $2M - 3$.

Theorem 18.1 (Nullspace projection loses no information)**.** *Suppose δp_f has no prior information and $H_{f,j}$ has full column rank. Then the information about δx in the projected measurement* (18.6) *equals the information about δx in* (18.5) *after δp_f is marginalized out:*

$$H_{o,j}^\top H_{o,j} = H_{x,j}^\top \Big(I - H_{f,j}(H_{f,j}^\top H_{f,j})^{-1} H_{f,j}^\top\Big) H_{x,j}, \tag{18.7}$$

and likewise for the information vectors $H_{o,j}^\top r_{o,j}$. The two formulations therefore produce identical updates.

Proof. The joint information matrix of $(\delta x, \delta p_f)$ contributed by (18.5) is $\sigma^{-2}\begin{bmatrix} H_x^\top H_x & H_x^\top H_f \\ H_f^\top H_x & H_f^\top H_f \end{bmatrix}$. Marginalizing δp_f in information form is the Schur complement (Theorem 3.3), $\sigma^{-2}\big(H_x^\top H_x - H_x^\top H_f (H_f^\top H_f)^{-1} H_f^\top H_x\big)$, and by the projector identity (2.11), $I - H_f(H_f^\top H_f)^{-1}H_f^\top = AA^\top$. The same algebra applies to the information vector $\sigma^{-2}H_x^\top(I - H_f(H_f^\top H_f)^{-1}H_f^\top)r$. ∎

An equivalent way to see this: complete A_j to an orthogonal matrix $Q = [Q_1 \;\; A_j]$. Multiplying by $Q^\top$ splits the equations into three that involve δp_f (through the invertible 3×3 matrix

$Q_1^\top H_f$) and $2M-3$ that do not. Since δp_f is free, the first three can always be satisfied by choosing δp_f, so they carry no information about δx. The rest carry all of it.

***Remark* 18.2** (how many observations). Two observations ($M=2$) leave $2M-3=1$ constraint, essentially the epipolar constraint between two clones. It is weak and sensitive to triangulation error, so the project requires $M \geq 3$ (AGENTS.md F5), which gives at least three constraints and a better-conditioned triangulation. If the triangulation is poor (low parallax), $H_{f,j}$ is close to rank-deficient, the projection is ill-defined, and the feature should have been rejected by the triangulation checks.

18.4.1 Computing the projection

A_j is never formed. By Proposition 2.10, the last $2M-3$ columns of the Q factor of a full QR decomposition of $H_{f,j}$ span its left nullspace. Householder QR of the $2M \times 3$ matrix $H_{f,j}$ uses exactly three reflections. Applying the same three reflections to $H_{x,j}$ and r_j, and keeping the last $2M-3$ rows of the results, gives $H_{o,j}$ and $r_{o,j}$ directly. The cost is $\mathcal{O}(M \cdot k)$ for k columns, and only the $6M$ non-zero clone columns of $H_{x,j}$ need processing. Givens rotations achieve the same with finer control over sparsity and were suggested in the original paper. Milestone 10b checks $\|A^\top H_f\| < 10^{-10}$ and the orthonormality of A in a test, where forming A explicitly is acceptable.

18.5 Gating, stacking and compression

Per-feature gating. Each projected residual is gated before it is accepted (AGENTS.md F5):

$$\gamma_j = r_{o,j}^\top \left(H_{o,j} P H_{o,j}^\top + \sigma_{uv}^2 I\right)^{-1} r_{o,j} < \chi^2_{0.95}(2M-3). \tag{18.8}$$

This rejects tracks that passed the front end's checks but are inconsistent with the filter's current belief: tracks on moving objects, slowly drifting tracks, and features with wrong associations.

Stacking. The accepted features are stacked into one measurement,

$$r_o = H_o\,\delta x + n_o, \qquad r_o \in \mathbb{R}^d, \quad d = \sum_j (2M_j - 3), \tag{18.9}$$

with the number of rows capped at a compile-time maximum so that the stacked workspace is fixed-size. With 50 features of 6 observations each, $d = 450$, much larger than the state dimension $n = 75$.

QR compression. When $d > n$, the update can be made cheaper without approximation.

> **Proposition 18.3** (Measurement compression). *Let $H_o = Q_1 T_H$ be a thin QR decomposition, with $T_H \in \mathbb{R}^{n\times n}$ upper triangular, and define $r_n = Q_1^\top r_o$. Then the measurement $r_n = T_H\,\delta x + n_n$ with $n_n \sim \mathcal{N}(0, \sigma_{uv}^2 I_n)$ yields exactly the same update as (r_o, H_o).*

Algorithm 4 MSCKF visual update

Input: filter state with clones, set of feature tracks ready for update

$d \leftarrow 0$
for each ready feature j with $M_j \geq 3$ observations **do**
  triangulate $\hat{p}_f$ (linear + Gauss–Newton); **if** checks fail **then** discard, **continue**
  build r_j, $H_{x,j}$ (clone columns only), $H_{f,j}$
  apply Householder QR of $H_{f,j}$ to $[H_{x,j} \;\; r_j]$; keep rows $4 \ldots 2M_j \rightarrow H_{o,j}, r_{o,j}$
  if $\gamma_j < \chi^2_{0.95}(2M_j - 3)$ **and** $d + 2M_j - 3 \leq d_{max}$ **then**
    append $H_{o,j}$, $r_{o,j}$ to the stacked workspace; $d \leftarrow d + 2M_j - 3$
  else
    count the rejection
  mark the track's observations as consumed
if $d > 0$ **then**
  optionally compress with QR if $d > n$
  EKF update (Joseph), inject into IMU and all clones, normalize, reset with G

Proof. The update depends on the measurement only through the information $H_o^\top H_o/\sigma^2$ and the information vector $H_o^\top r_o/\sigma^2$ (Equation (6.8)). Since $Q_1^\top Q_1 = I$, $T_H^\top T_H = H_o^\top Q_1 Q_1^\top H_o = H_o^\top H_o$ (as $Q_1 Q_1^\top$ projects onto range(H_o), which contains the columns of H_o), and $T_H^\top r_n = H_o^\top Q_1 Q_1^\top r_o = H_o^\top r_o$. The discarded components $Q_2^\top r_o$ are pure noise, independent of δx. ∎

Compression reduces the innovation covariance from $d \times d$ to at most $n \times n$, and the Joseph update from $O(dn^2 + d^3)$ to $O(n^3)$ after an $O(dn^2)$ QR. Only the non-zero columns of H_o (the clone columns) need to take part in the decomposition. Compression is optional in the project (AGENTS.md F5); it is worthwhile when $d \gg n$.

18.6 The update

The stacked (and possibly compressed) measurement is applied with the generic update of Chapter 11: $S = HPH^\top + \sigma_{uv}^2 I$, gain by Cholesky solve, Joseph form, then injection into the IMU state and *every* clone, normalization of every touched quaternion, and the reset with one $I - \frac{1}{2}[\delta\hat{\theta}_i]_\times$ block per attitude. Corrections to the clones are real: they improve the poses that future features will be triangulated from.

Project link: Milestone 10b

The exit criteria test every link of Algorithm 4: projection Jacobians by finite differences; triangulation exact on noise-free data and bounded under noise, rejecting negative-depth and low-parallax features; $\|A^\top H_f\| < 10^{-10}$ with orthonormal A; a full synthetic VIO run without divergence, with NEES reported (yaw inconsistency expected and documented); zero allocations with fixed-max workspaces; and MH_01 end to end with ATE reported.

Key takeaways

- Each observation depends on one clone; $H_{x,j}$ is zero in the IMU columns and dense in the feature's $H_{f,j}$.

- Projecting onto the left nullspace of $H_{f,j}$ removes the feature, keeps the noise white, and loses no information about the poses (it equals Schur-complement marginalization).
- Three Householder reflections applied to $[H_{x,j}\ r_j]$ compute the projection without forming A_j.
- Gate each feature with $\chi^2_{0.95}(2M-3)$, stack, compress with QR if $d > n$, and update IMU and clones together.

Exercises

18.1 Verify Theorem 18.1 numerically: compare the posterior covariance of δx from the projected update with that from an update of the augmented state $(\delta x, \delta p_f)$ with a very large prior on δp_f.

18.2 Show that the three discarded rows $Q_1^\top r_j$ determine the correction to $\hat{p}_f$ given δx. How would you recover an updated feature estimate if you wanted one?

18.3 For $M = 2$, compute A_j explicitly and interpret the single projected residual geometrically.

18.4 Implement the Householder application of Algorithm 4 and compare its cost and result with forming A_j from a full QR.

18.5 Prove that the projected noise vectors $n_{o,j}$ of two different features are independent if their raw noises are, so that stacking them with a block-diagonal (here scalar) noise covariance is correct. Why are the projected *residuals* themselves correlated?

Chapter 19

The Complete Algorithm

In this chapter

- Assemble every component into one event-driven estimator, with the correct order of operations.
- Estimate the computational budget and map the algorithm onto the project's software architecture.
- Diagnose the most common failures from their symptoms.

19.1 The event loop

The estimator consumes a single, time-ordered stream of sensor events. Milestone 3 builds it by merging the presorted IMU, image (and, for testing, pose) arrays with a two-pointer merge, with the rule that an IMU sample precedes any other event with the same timestamp. Algorithm 5 shows the loop.

Three ordering rules are essential.

1. **Propagate before you update.** A measurement at time t must be applied to the state at time t, never to the state at the previous IMU sample.
2. **Use a clone's observations before you delete it.** Every feature with an observation at the oldest clone is processed before that clone is marginalized; otherwise its information is lost.
3. **Use each observation once.** After a track has been used, its observations are discarded. If the feature keeps being tracked, later observations form a new track.

With the order of Algorithm 5, the window never holds more than `kMaxClones` clones, which keeps the covariance within its fixed maximum size. A feature processed because its oldest clone is leaving cannot use the current image's observation in that update, because the new clone does not exist yet. Implementations that clone first need room for one extra clone. Either choice is valid; the storage bound must match it.

19.2 Initialization and the first seconds

The project initializes from ground truth, which removes the hardest failure mode from early testing and is logged as a use of ground truth (Milestone 10b). With static initialization (Chapter 9), the first seconds are delicate: the yaw is arbitrary, the accelerometer bias is entangled with tilt, and the window contains few clones with little parallax. Generous initial covariances, and a check that enough features with sufficient parallax exist before trusting

Algorithm 5 Monocular MSCKF

Input: event stream, calibration, noise parameters

▷ *load phase: allocation allowed*
parse data and calibration; build the event arrays; construct the filter and front end with fixed-max workspaces
initialize $(\hat{x}, P)$ (ground truth in the project, logged; static initialization otherwise)
▷ *streaming phase: no allocation in the core*
for each event e in timestamp order **do**
    if e is an IMU sample at t **then**
        propagate nominal state and covariance from t_{prev} to t ▷ Chapters 9 and 10
    else if e is an image at t_c **then**
        propagate to t_c (interpolated IMU) ▷ exact image time
        front end: track, verify, undistort; update track histories ▷ Chapter 15
        process tracks that were lost in this image ▷ Algorithm 4
        if $N =$ `kMaxClones` **then**
            process tracks with an observation at the oldest clone
            marginalize the oldest clone in place ▷ Chapter 17
        clone $(\hat{p}, \hat{q})$; $P \leftarrow JPJ^{\mathsf{T}}$; attach this image's observations to the new clone
    log state, diag(P), NIS and gate statistics to the preallocated buffer

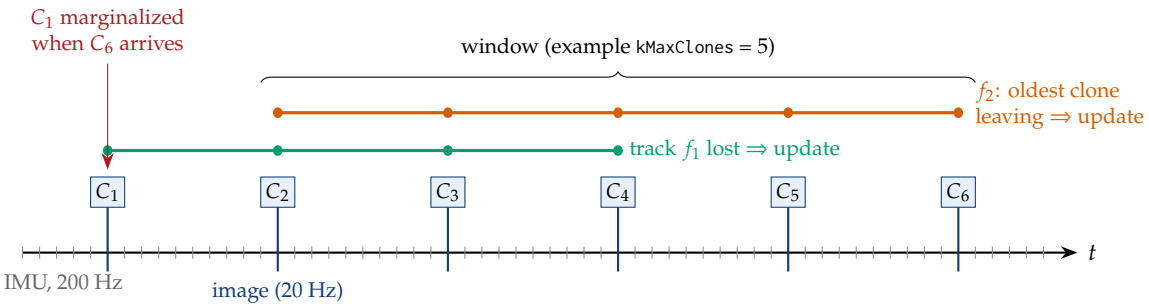


Figure 19.1: Timeline of the MSCKF. The IMU propagates at 200 Hz; each image creates a clone; feature tracks are processed when they end or when the clone holding their oldest observation is about to leave the window.

visual updates, avoid early divergence. Inspect the IMU data of each EuRoC sequence before running on it: a static start followed by vigorous motion is the easiest case for initialization, and a fast start the hardest.

19.3 Computational budget

With $N = 10$ clones the error state has $n = 75$ dimensions. Rough flop counts per operation:

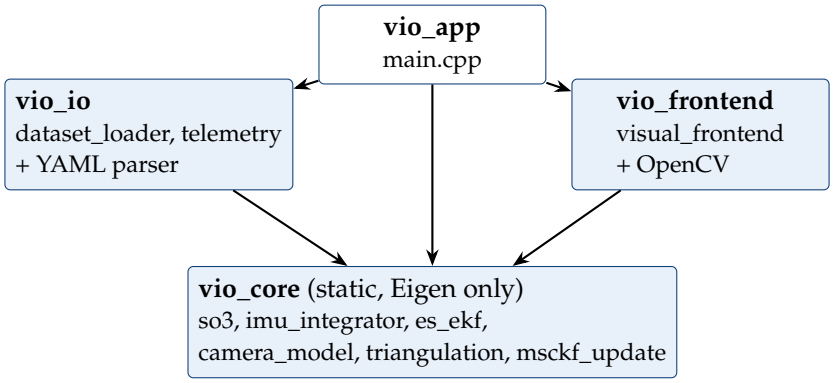


Figure 19.2: Library targets of the project (AGENTS.md). Dependencies point one way, so the estimator core can be built, tested and reasoned about without the I/O and vision libraries.

Operation	Frequency	Cost (flops)
Nominal propagation	200 Hz	$\sim 10^2$
P_{II}, P_{IC} propagation	200 Hz	$\sim 2 \times 10^4$
Cloning ($P \leftarrow JPJ^\mathsf{T}$ as block copies)	20 Hz	$\sim 10^4$
Per feature: triangulation, projection, gating ($M = 6$)	per track	$\sim 10^5$
QR compression of $d = 450$ rows	per update	$\sim 5 \times 10^6$
Update (S, gain, Joseph) with $n = 75$	per update	$\sim 3 \times 10^6$
Front end (KLT on 150 features)	20 Hz	dominant in time

The filter core needs on the order of 10^7 flops per image, a few milliseconds on an embedded processor and well under a millisecond on a desktop. The front end is usually the bottleneck in wall-clock time. The cubic growth with n is why the window is short and why QR compression matters for large stacks.

19.4 Software architecture

Figure 19.2 shows how the algorithm maps to the project's targets. The zero-allocation discipline applies to `vio_core`, which contains everything executed per IMU sample and per update. The front end may allocate but must reuse its buffers, and the I/O layer allocates freely during loading.

19.5 Debugging guide

Table 19.1 collects the failures that experience shows are most common. The milestone structure of the project is itself the most effective debugging tool: each layer is verified against known truth before the next is built on top of it, so a new failure is almost always in the newest layer.

Symptom	Likely causes
Altitude diverges at about $2g$ in IMU-only propagation	gravity or specific-force sign; g_w added twice
Position diverges within seconds even with ground-truth init	R_{wb} used as R_{bw}; quaternion w/z swapped at load; gyro and accel columns or biases swapped; degrees vs radians
Rotation matrices slowly scale vectors	missing quaternion normalization
NaN after a while	missing Taylor branch; Log near π; J_r^{-1} evaluated beyond π; feature at zero depth; Cholesky of a non-PD S
Corrections systematically make things worse	sign of H or of the residual; injection on the wrong side of the quaternion; global vs local convention mixed
Covariance loses symmetry or positive definiteness	simple update form instead of Joseph; missing symmetrization; aliasing in Eigen block products
Nearly all features fail the gate	overconfident P; T_{bc} inverted; pixel noise used in normalized units (or vice versa); distortion not removed; timestamps misaligned
Gate rejects nothing	R_z or σ_{uv} far too large; gate threshold indexed with the wrong dimension
Slowly growing yaw error with healthy NIS	expected EKF inconsistency along unobservable yaw (Chapter 20)
Scale drift	little acceleration (degenerate motion); accelerometer noise or bias badly tuned
Works on synthetic data, fails on EuRoC	extrinsics, time offset, distortion, noise inflation, initialization, IMU gaps
Results differ between runs	uninitialized memory (run the sanitizers); unseeded random generators

Table 19.1: Symptoms and their most frequent causes.

19.6 What to expect on EuRoC

Published monocular filter-based estimators reach absolute trajectory errors of the order of 0.1 to 0.5 m on the machine-hall sequences, depending on features, tuning and initialization, with the difficult sequences (fast motion, motion blur, dark regions) considerably worse [12]. Milestone 10b sets a first-pass target of 0.5 m on MH_01 and a stretch target of 0.2 m. Yaw and position drift without bound; roll and pitch remain accurate throughout; velocity and biases converge within the first seconds of excitation.

Key takeaways

- One time-ordered stream; propagate to each event's time before applying it.
- At each image: process lost tracks; if the window is full, process the oldest clone's tracks and marginalize it; then clone.
- The core costs about 10^7 flops per image; the front end usually dominates the run time.
- Dependencies point one way: `vio_core` knows nothing about files, YAML or OpenCV.
- Most failures map to a small set of sign, frame, unit and timing errors; test each layer against truth.

Exercises

19.1 Write the event-loop pseudocode for the alternative ordering (clone first, then process and marginalize). What capacity does it require, and what does it gain?

19.2 Instrument a run to record the number of features processed per update, their observation counts, and the stacked row count d. Choose the compile-time row cap from the distribution.

19.3 For each row of Table 19.1, design a unit or integration test that would have caught the cause before the full system was assembled.

Part VII

Beyond the Baseline

Chapter 20

Observability and Consistency of VIO

In this chapter

- Define observability for nonlinear and linearized systems.
- Prove that VIO has four unobservable directions and write them in the project's error coordinates.
- Understand why the standard EKF gains spurious information about yaw and becomes overconfident.
- Know the remedies (FEJ, observability constraints, invariant filtering) and the degenerate motions that remove further information.

20.1 Observability

A system is *observable* if its state can be determined from its inputs and outputs. For the nonlinear systems of navigation the relevant notion is local: a state is *locally weakly observable* if it can be distinguished from all its neighbors using the outputs over some time interval. For an estimator the linearized version is the operative one.

Definition 20.1 (Observability matrix)**.** For the linearized time-varying system $\delta x_{k+1} = \Phi_k \delta x_k$, $r_k = H_k \delta x_k$, the observability matrix over steps $1 \dots k$ is

$$\mathcal{O} = \begin{bmatrix} H_1 \\ H_2 \Phi_1 \\ H_3 \Phi_2 \Phi_1 \\ \vdots \\ H_k \Phi_{k-1} \cdots \Phi_1 \end{bmatrix}. \tag{20.1}$$

An initial error $\delta x_1 \in \text{null}(\mathcal{O})$ produces no change in any measurement, so it cannot be detected. $\text{null}(\mathcal{O})$ is the *unobservable subspace*.

A Kalman filter can only gain information in directions outside $\text{null}(\mathcal{O})$. Along unobservable directions, its covariance may grow (from process noise) but must never shrink. This is the property the standard EKF violates.

20.2 The four unobservable directions of VIO

Theorem 20.2 (Unobservable directions of VIO). *For monocular VIO with point features, applying the same global translation t and the same rotation $R_z(\alpha)$ about the gravity direction to all positions, velocities, attitudes, clones and features leaves every IMU and camera measurement unchanged. The state is therefore unobservable along these four directions.*

Proof. Under $p \mapsto R_z p + t$, $v \mapsto R_z v$, $R_{wb} \mapsto R_z R_{wb}$, $p_f \mapsto R_z p_f + t$, with biases unchanged:

- the gyroscope measures $[\omega_b]_\times = R_{wb}^\mathsf{T} \dot{R}_{wb}$, which is invariant because R_z is constant;
- the accelerometer measures $R_{wb}^\mathsf{T} R_z^\mathsf{T}(R_z a_w - g_w) = R_{wb}^\mathsf{T}(a_w - R_z^\mathsf{T} g_w) = R_{wb}^\mathsf{T}(a_w - g_w)$, because a rotation about gravity leaves g_w unchanged;
- the camera sees $R_{bc}^\mathsf{T}\big(R_{wb}^\mathsf{T} R_z^\mathsf{T}(R_z p_f + t - R_z p - t) - p_{bc}\big) = R_{bc}^\mathsf{T}\big(R_{wb}^\mathsf{T}(p_f - p) - p_{bc}\big)$.

All measurements are identical, so the transformed and original trajectories cannot be distinguished. ∎

Rotations about horizontal axes are *not* in this family, because they would rotate gravity in the accelerometer equation; roll and pitch are observable. Scaling is not either, because it would scale the accelerations the accelerometer measures; scale is observable as long as the vehicle accelerates.

Linearizing the transformation gives the unobservable directions in the project's error coordinates. With e_z the vertical unit vector, a small yaw rotation α and a small translation t change the state by (verified numerically):

$$\underbrace{\begin{bmatrix} \delta p \\ \delta v \\ \delta\theta \\ \delta b_a \\ \delta b_g \\ \delta p_i \\ \delta\theta_i \\ \delta p_f \end{bmatrix}}_{\text{error state}} = \underbrace{\begin{bmatrix} I_3 & -[\hat{p}]_\times e_z \\ 0 & -[\hat{v}]_\times e_z \\ 0 & \hat{R}^\mathsf{T} e_z \\ 0 & 0 \\ 0 & 0 \\ I_3 & -[\hat{p}_i]_\times e_z \\ 0 & \hat{R}_i^\mathsf{T} e_z \\ I_3 & -[\hat{p}_f]_\times e_z \end{bmatrix}}_{N} \begin{bmatrix} t \\ \alpha \end{bmatrix}. \tag{20.2}$$

The last row applies only to formulations that keep features in the state. The attitude entry is $\hat{R}^\mathsf{T} e_z$ because the error is local: a rotation about the world vertical, expressed in body axes. Every column of N *depends on the estimate*: the yaw column involves $\hat{p}$, $\hat{v}$, $\hat{R}$ and the clone and feature estimates. That dependence is the root of the problem.

20.3 Why the standard EKF becomes overconfident

If every Jacobian were evaluated at the true state, the linearized system would satisfy $O N = 0$ at all times, and the filter would never gain information along N. The EKF instead evaluates each Jacobian at the best estimate available when it is computed. The transition matrix Φ_k is evaluated at the propagated estimate of step k, while the next measurement Jacobian is

evaluated at an estimate of the same state that has since been corrected by an update. The same physical state enters the observability matrix linearized at two different points, and the products $H_k\Phi_{k-1}\cdots\Phi_1 N$ no longer cancel.

The translation columns survive this, because all Jacobians depend only on *differences* of positions. The yaw column does not: with estimate-dependent Jacobians, $\mathcal{O}N_{\text{yaw}} \neq 0$, and the unobservable subspace of the EKF's linearized model has dimension three instead of four [26, 31, 23]. The filter then extracts *spurious information* about the global yaw from measurements that contain none. Its yaw variance shrinks when it should not, it becomes overconfident in yaw, and because yaw errors couple into position through the trajectory, its position covariance becomes too small as well. On long trajectories the NEES of yaw and position grows steadily while the NIS can look perfectly healthy: the inconsistency lives in directions the measurements do not see (Table 12.1).

Key idea: an inconsistency that no unit test catches

EKF yaw inconsistency is not a bug in the usual sense: every Jacobian can pass its finite-difference test. It is a property of linearizing at estimates that change over time. Milestone 10b therefore expects it, and asks for it to be measured and documented rather than fixed.

20.4 Remedies

First-Estimates Jacobians (FEJ). Evaluate every Jacobian that involves a state at the *first* estimate of that state, the value it had when it entered the filter, rather than at the latest one [25, 31]. Each state then appears in the observability matrix linearized at a single point, the products telescope, and $\mathcal{O}N = 0$ holds again with N built from first estimates. In the MSCKF this means using the propagated (pre-update) estimates of position and velocity at both ends of each propagation interval in Φ, and the first estimates of the clone poses and of the triangulated feature in the measurement Jacobians. The price is a somewhat larger linearization error; in exchange the filter is markedly more consistent and, in practice, more accurate. FEJ is the project's stretch goal.

Observability-constrained EKF (OC-EKF). Modify each Jacobian minimally, in the least-squares sense, so that the correct unobservable subspace is enforced explicitly [23].

Invariant EKF. Define the error on the extended pose group $\mathrm{SE}_2(3)$ with a right-invariant error. The error dynamics and the unobservable directions then become independent of the estimate, and consistency with respect to these directions holds by construction [4].

Robocentric formulations. Express the state relative to the current body frame, so that the unobservable global quantities are not estimated at all.

20.5 Degenerate motions

Theorem 20.2 assumes *general* motion. Some motions remove further directions [52].

Motion	What becomes unobservable
constant velocity (no acceleration)	metric scale
pure rotation (no translation)	feature depths and translation scale
no rotation	accelerometer bias inseparable from the gravity direction (tilt)
at rest	both of the above, plus everything that needs parallax

The filter does not fail during such motions, provided it is consistent: its covariance simply stops shrinking along the affected directions until informative motion returns. An inconsistent filter, by contrast, may "learn" these quantities from noise. When a vehicle is known to be stationary, a *zero-velocity update* (the pseudo-measurement $v = 0$) supplies real information and stops drift, which is why many systems detect stillness explicitly.

Key takeaways

- The unobservable subspace is the nullspace of the observability matrix; a filter must not gain information along it.
- VIO has four unobservable directions: global translation and rotation about gravity. Roll, pitch and scale are observable under general motion.
- In local error coordinates the yaw direction is $\hat{R}^\mathsf{T} e_z$ for the attitude and $-[\hat{p}]_\times e_z$ for positions: it depends on the estimate.
- Linearizing the same state at different estimates makes yaw spuriously observable, so the EKF becomes overconfident in yaw and position.
- FEJ, OC-EKF and invariant filtering restore consistency; degenerate motions remove further directions.

Exercises

20.1 Verify (20.2) numerically: apply a small yaw rotation and translation to a random state and compute $x' \boxminus x$.

20.2 Show that the translation columns of N remain in the nullspace of the EKF's observability matrix even when Jacobians are evaluated at changing estimates.

20.3 For a planar robot with odometry and range-bearing measurements to a landmark, show that the EKF gains spurious heading information. (This is the classic EKF-SLAM inconsistency result.)

20.4 Implement a zero-velocity update as an EKF measurement and derive its H. When is it safe to apply?

20.5 Read [31] and list which Jacobians must be evaluated at first estimates in the MSCKF. Sketch how you would store the first estimates in the project's data structures without allocation.

Chapter 21

Engineering a Real-Time Estimator in C++20

In this chapter

- Understand why a real-time estimator must be deterministic, and how the project's memory tiers achieve it.
- Use fixed-max-size Eigen matrices, member workspaces and allocation guards correctly.
- Avoid the Eigen pitfalls that silently corrupt numbers: dangling expression templates and aliasing.
- Design a test strategy in which every test can fail, and a systematic procedure for debugging a diverging filter.

The mathematics of the previous chapters is necessary but not sufficient. An estimator that is mathematically correct can still miss its deadline because of an allocation, produce garbage because of an Eigen expression template, or pass all its tests because the tests cannot fail. This chapter collects the engineering practices that the project encodes in AGENTS.md.

21.1 Determinism and the memory tiers

A flight controller consumes the estimator's output at a fixed rate. A late estimate is a wrong estimate. The main source of unpredictable latency in C++ is dynamic memory allocation: `new`, a `std::vector` that grows, a dynamic-size Eigen temporary. The allocator may take locks, search free lists or ask the operating system for pages, with latency that is usually small and occasionally large. The project therefore separates three tiers (AGENTS.md E4).

Tier	Scope	Rule
Load	parsing, building event arrays, constructing the filter	allocate freely, before streaming starts
Core	`vio_core`, per IMU sample and per update	zero heap allocations
Frontend	OpenCV tracking	may allocate, must reuse buffers; allocations are measured and logged

21.1.1 Fixed-max-size matrices

The MSCKF covariance changes size as clones come and go, but never exceeds $15 + 6N_{max}$. Eigen supports exactly this case: a matrix whose *runtime* size is dynamic but whose *maximum*

size is a compile-time constant. Its storage is an inline array of the maximum size, so resizing within the bound never allocates.

```
inline constexpr int kMaxClones = 10;                  // decided in M10a
inline constexpr int kMaxDim    = 15 + 6 * kMaxClones;

using CovMatrix = Eigen::Matrix<double, Eigen::Dynamic, Eigen::Dynamic,
                                Eigen::ColMajor, kMaxDim, kMaxDim>;

CovMatrix P;          // inline storage for 75 x 75 doubles, no heap
P.resize(15, 15);     // allowed: within the compile-time maximum
```

Large workspaces (S, K, stacked H, temporaries for ΦP) are class members, created once, rather than function-local variables. Function-local fixed-max matrices would not allocate either, but they would live on the stack, and several of them at once can exceed it (Chapter 17).

21.1.2 Proving it: allocation guards

"It shouldn't allocate" is not evidence. The project overrides the global `operator new` in its tests to count allocations (`tests/support/alloc_guard.hpp`), and wraps every core hot-path test with a guard that asserts a count of zero. In debug builds, `EIGEN_RUNTIME_NO_MALLOC` together with `Eigen::internal::set_is_malloc_allowed(false)` makes Eigen itself abort on any internal allocation and reports where it happened. The two mechanisms are complementary: the guard catches allocations anywhere, Eigen's flag pinpoints those inside Eigen.

21.2 Eigen pitfalls

Expression templates and `auto`. Eigen operations return lightweight *expression* objects that are evaluated lazily when assigned to a matrix. Storing one with `auto` keeps references to its operands, which may be temporaries that no longer exist when the expression is finally evaluated.

```
auto bad = R * (a + b);             // expression refers to the temporary (a + b)
Eigen::Vector3d x = bad;            // undefined behavior: the temporary is gone
Eigen::Vector3d good = R * (a + b); // evaluated immediately into a concrete type
```

The rule is simple: never use `auto` with Eigen expressions (AGENTS.md E5). Name the concrete type, or call `.eval()` when a temporary really is wanted. AddressSanitizer catches many of these bugs, but not all.

Aliasing. Eigen assumes that the destination of a matrix product may alias its operands and evaluates products into a temporary. With dynamic sizes that temporary allocates. When the destination is known not to alias, `.noalias()` writes directly into it. When it does alias, the temporary is required: write into a member workspace explicitly.

```
tmp_15xN_.noalias() = Phi * P.topRightCorner(15, nc);  // reads P, writes workspace
P.topRightCorner(15, nc) = tmp_15xN_;                   // copy back
P.bottomLeftCorner(nc, 15) = tmp_15xN_.transpose();     // keep symmetry
```

Coefficient-wise operations do not alias-check. `A = A.transpose()` is the classic bug; use `A.transposeInPlace()` or symmetrize with a workspace.

Passing and alignment. Pass Eigen objects by `const&` (or `Eigen::Ref` for blocks). Do not add `EIGEN_MAKE_ALIGNED_OPERATOR_NEW`: since C++17, aligned `new` handles over-aligned types. The real alignment hazard is mixing `-march` or alignment-changing flags between translation units or with dependencies, which the project forbids.

Decompositions without allocation. Eigen's decompositions normally own their storage. Fixed-max decomposition objects held as members, or the in-place variants that factor a matrix in its own storage, avoid per-call allocation. Whatever the API, verify with the allocation guard.

21.3 Time and events

```
using Clock = std::chrono::nanoseconds;
struct ImuSample   { Clock t; Eigen::Vector3d gyro, accel; };
struct ImageEvent  { Clock t; int index; };
using SensorEvent = std::variant<ImuSample, ImageEvent /*, PoseMeasurement */>;

// Delta t: subtract in integers, convert only the difference.
const double dt = std::chrono::duration<double>(t1 - t0).count();
```

Absolute time stays in 64-bit integer nanoseconds (Section 2.9). Events form one presorted sequence built at load time, so the streaming loop is a single pass over contiguous memory: no priority queue, no allocation, no reordering surprises.

21.4 Numerical robustness checklist

- Normalize every quaternion after every multiplicative update (propagation, injection, cloning).
- Symmetrize P after propagation and after every update.
- Use the Joseph form; factor S with Cholesky or LDL^T; treat a failed factorization as an error to report, not to hide.
- Guard every $0/0$ with Taylor branches; use atan2 for angles; handle Log near π.
- Reject features with non-positive or tiny depth, poor parallax or large reprojection error before linearizing.
- Count and log gate rejections and triangulation failures.
- Never clamp negative eigenvalues of P to hide a problem; find out why they appeared.
- In debug builds, assert finiteness at module boundaries so that a `NaN` is caught where it is born.

21.5 Testing

Key idea: a test must be able to fail

For every test, ask: would it still pass if I flipped the sign of X, transposed Y, or swapped two blocks? If yes, the test is decoration. Run it once against a deliberately broken implementation to make sure it fails.

The project's test strategy layers several kinds of evidence.

1. **Algebraic unit tests**: identities such as $\mathrm{Log}(\mathrm{Exp}(\phi)) = \phi$ at zero, tiny, generic and near-π angles.
2. **Finite-difference Jacobian tests** built on $\boxplus/\boxminus$, at non-trivial states, with tolerances justified in a comment.
3. **Closed-form trajectory tests**: synthetic trajectories with analytic IMU data and convergence-order checks.
4. **Statistical tests**: Monte Carlo covariance checks and NEES/NIS with χ^2 bounds, seeded with `std::mt19937_64`. Because `std::normal_distribution` differs between standard libraries, never hard-code values from one run; test statistical properties.
5. **Invariance tests**: cloning enabled versus disabled must give identical IMU estimates when no visual updates occur.
6. **Resource tests**: allocation guards around every core hot path.
7. **Sanitizers**: AddressSanitizer and UndefinedBehaviorSanitizer, with `-fno-sanitize-recover=all` so that any report fails the test run.

21.6 Debugging a diverging filter

When the full system misbehaves, resist the urge to tune. Work down the layers instead.

1. Run on **synthetic data** with exactly modeled noise. If it fails there, the bug is in the code.
2. Disable visual updates. Does IMU-only propagation from ground truth behave as Figure 10.1 predicts, with errors inside 3σ?
3. Replace visual updates by **pose updates** (Milestone 6). If those are consistent, the back end is sound.
4. Feed the MSCKF **ground-truth feature tracks** from a synthetic scene. If that works, the problem is in the front end or the calibration.
5. On real data, plot each residual component over time and look for bias, oscillation or outliers.
6. Only then adjust noise parameters, and judge the result on consistency statistics.

21.7 Going further

Once the baseline MSCKF runs, natural extensions are First-Estimates Jacobians (Chapter 20); online calibration of the camera-IMU extrinsics and time offset [31, 32]; stereo observations from `cam1`; keeping long-lived features as SLAM landmarks; zero-velocity updates; and evaluation on the harder EuRoC sequences (V1_03, MH_05) and other datasets. Each extension adds states and Jacobians, and each Jacobian gets its derivation and its finite-difference test.

Key takeaways

- Determinism requires zero allocation in the core: fixed-max-size matrices, member workspaces, proven by allocation guards.
- Never use `auto` with Eigen expressions; use `.noalias()` only when there is no aliasing.
- Keep time in integer nanoseconds and process one presorted event stream.
- Every test must be able to fail; layer algebraic, Jacobian, closed-form, statistical and resource tests.
- Debug by removing layers until the failure disappears, not by tuning.

Exercises

21.1 Write a minimal allocation guard by overriding the global `operator new` and `operator delete`, and show that it detects a `std::vector::push_back` beyond capacity but not a fixed-max Eigen `resize`.

21.2 Construct an example in which `auto` with an Eigen expression produces wrong numbers without crashing. Does AddressSanitizer report it?

21.3 Measure the time of 10 000 covariance propagation steps with fixed-max and with fully dynamic Eigen matrices, in both build presets. Explain the difference.

21.4 Take one of your Jacobian tests, break the Jacobian in three different ways, and record which breakages the test detects.

Appendices

Appendix A

Notation and Conventions

This appendix restates the frozen conventions of `AGENTS.md` in one place, together with the notation of these notes.

Frames and transformations

Symbol	Code name	Meaning
W	—	world frame: fixed, gravity-aligned, z up; EuRoC `R`
B	—	body frame = IMU `imu0`; EuRoC `S` in `imu0`
C	—	left camera `cam0`: z optical axis, x right, y down
R_{wb}, q_{wb}	`R_wb`, `q_wb`	attitude, maps body to world: $v_w = R_{wb} v_b$
p_{wb}	`p_wb`	position of B's origin in W
v_{wb}	`v_wb`	velocity of B relative to W, expressed in W
$T_{bc} = (R_{bc}, p_{bc})$	`T_bc`	camera-to-body: $p_b = R_{bc} p_c + p_{bc}$; EuRoC `T_BS` of `cam0`
g_w	—	gravity, $[0, 0, -9.81]^\mathsf{T}$ m/s^2

Naming rule: `<quantity>_<to><from>`; adjacent subscripts of a product must match ($R_{wb} R_{bc} = R_{wc}$).

State, error state and noise

Error index	Block	Definition
0–2	δp	$p = \hat{p} + \delta p$ (world)
3–5	δv	$v = \hat{v} + \delta v$ (world)
6–8	$\delta\theta$	$q = \hat{q} \otimes \mathrm{Exp}_q(\delta\theta)$ (body, local)
9–11	δb_a	$b_a = \hat{b}_a + \delta b_a$
12–14	δb_g	$b_g = \hat{b}_g + \delta b_g$
$15 + 6i \ldots$	δp_i	clone i position
$18 + 6i \ldots$	$\delta\theta_i$	clone i attitude (local)

Noise index	Block
0–2	n_a
3–5	n_g
6–8	n_{b_a}
9–11	n_{b_g}

Operators and symbols

Symbol	Meaning
a^T, A^{-1}, $A^{-\mathsf{T}}$	transpose, inverse, inverse transpose
$[a]_\times$, $a^\wedge$	skew-symmetric (cross-product) matrix of $a \in \mathbb{R}^3$
$(\cdot)^\vee$	inverse of the hat map
$\otimes$	Hamilton quaternion product
q^*	quaternion conjugate
exp, log	matrix exponential and logarithm
Exp, Log	$\mathbb{R}^3 \leftrightarrow$ SO(3); Exp_q, Log_q for unit quaternions
J_r, J_l	right and left Jacobians of SO(3)
$\boxplus$, $\boxminus$	retraction and local difference (AGENTS.md C4)
$\hat{x}$, δx, x	nominal (estimated), error, and true state
P, Q_c, Q_d, R_z	error covariance, continuous and discrete process noise, measurement noise
F_c, G_c, Φ	continuous error dynamics, noise input matrix, discrete transition
H, S, K, r	measurement Jacobian, innovation covariance, gain, residual
$\mathcal{N}(\mu, \Sigma)$	Gaussian distribution
χ^2_k, $\chi^2_p(k)$	chi-squared distribution with k DOF; its quantile at probability p
$\pi(\cdot)$	pinhole projection $(X/Z, Y/Z)$
u, $\varepsilon_{\mathrm{mach}}$	unit roundoff 2^{-53}, machine epsilon 2^{-52}

Abbreviations

ATE / RPE	absolute trajectory error / relative pose error
BCH	Baker–Campbell–Hausdorff formula
DOF	degrees of freedom
EKF / ESKF	extended / error-state Kalman filter
FEJ	first-estimates Jacobians
IMU	inertial measurement unit
KLT / LK	Kanade–Lucas–Tomasi tracker / Lucas–Kanade optical flow
MSCKF	multi-state constraint Kalman filter
NEES / NIS	normalized estimation error squared / normalized innovation squared
PSD / PD	positive semi-definite / positive definite
RANSAC	random sample consensus
SLAM	simultaneous localization and mapping
VIO	visual-inertial odometry

Appendix B

Formula Sheet

Every formula below follows the project conventions (Hamilton quaternions, local perturbations) and was verified numerically. Let $\theta = \|\phi\|$.

Cross products

$$[a]_\times b = a \times b = -[b]_\times a, \qquad [a]_\times^\mathsf{T} = -[a]_\times, \qquad [a]_\times^2 = aa^\mathsf{T} - \|a\|^2 I, \qquad [a]_\times^3 = -\|a\|^2 [a]_\times,$$
$$[a]_\times [b]_\times = ba^\mathsf{T} - (a^\mathsf{T} b) I, \qquad [a \times b]_\times = [a]_\times [b]_\times - [b]_\times [a]_\times, \qquad [Ra]_\times = R[a]_\times R^\mathsf{T}.$$

Exponential and logarithm

$$\mathrm{Exp}(\phi) = I + \frac{\sin\theta}{\theta}[\phi]_\times + \frac{1-\cos\theta}{\theta^2}[\phi]_\times^2, \qquad \mathrm{Exp}_q(\phi) = \left[\cos\tfrac{\theta}{2},\ \tfrac{\sin(\theta/2)}{\theta}\phi\right],$$
$$\theta = \mathrm{atan2}\left(\tfrac{1}{2}\|(R - R^\mathsf{T})^\vee\|,\ \tfrac{1}{2}(\mathrm{tr}\, R - 1)\right), \qquad \mathrm{Log}(R) = \frac{\theta}{2\sin\theta}(R - R^\mathsf{T})^\vee \quad (0 < \theta < \pi),$$
$$\text{near } \pi:\ uu^\mathsf{T} = \frac{\frac{1}{2}(R + R^\mathsf{T}) - \cos\theta\, I}{1 - \cos\theta}, \qquad \mathrm{Log}_q(q) = 2\,\mathrm{atan2}(\|v\|, w)\frac{v}{\|v\|} \ \text{ after } w \geq 0.$$

Coefficient functions and their series

Function	Closed form	Series
$A(\theta)$	$\sin\theta/\theta$	$1 - \theta^2/6 + \theta^4/120$
$B(\theta)$	$(1-\cos\theta)/\theta^2$	$1/2 - \theta^2/24 + \theta^4/720$
$C(\theta)$	$(\theta - \sin\theta)/\theta^3$	$1/6 - \theta^2/120 + \theta^4/5040$
$D(\theta)$	$1/\theta^2 - \cot(\theta/2)/(2\theta)$	$1/12 + \theta^2/720 + \theta^4/30240$
$\cos(\theta/2)$		$1 - \theta^2/8 + \theta^4/384$
$\sin(\theta/2)/\theta$		$1/2 - \theta^2/48 + \theta^4/3840$
$\theta/\sin\theta$		$1 + \theta^2/6 + 7\theta^4/360$

Jacobians of SO(3)

$$J_r(\phi) = I - B\,[\phi]_\times + C\,[\phi]_\times^2, \qquad J_r^{-1}(\phi) = I + \tfrac{1}{2}[\phi]_\times + D\,[\phi]_\times^2,$$
$$J_l(\phi) = J_r(-\phi) = J_r(\phi)^\mathsf{T} = \mathrm{Exp}(\phi)J_r(\phi), \qquad J_r(\phi) = \int_0^1 \mathrm{Exp}(-s\phi)\,\mathrm{d}s,$$
$$\mathrm{Exp}(\phi+\delta) \approx \mathrm{Exp}(\phi)\,\mathrm{Exp}(J_r(\phi)\delta), \qquad \mathrm{Log}(\mathrm{Exp}(\phi)\,\mathrm{Exp}(\delta)) \approx \phi + J_r^{-1}(\phi)\delta,$$
$$R\,\mathrm{Exp}(\phi)R^\mathsf{T} = \mathrm{Exp}(R\phi), \qquad \mathrm{Log}(\mathrm{Exp}(a)\,\mathrm{Exp}(b)) = a + b + \tfrac{1}{2}a \times b + \cdots$$

Elementary Jacobians (local perturbation $R\,\mathrm{Exp}(\delta\theta)$)

Rv	$-R[v]_\times$	R^{-1}	$-R$
$R^\mathsf{T}v$	$[R^\mathsf{T}v]_\times$	R_1R_2	R_2^T (w.r.t. R_1), I (w.r.t. R_2)
$\mathrm{Exp}(\phi)$	$J_r(\phi)$	$\mathrm{Log}(R)$	$J_r^{-1}(\mathrm{Log}\,R)$

Quaternions

$$p \otimes q = \left[p_w q_w - p_v^\mathsf{T} q_v,\ p_w q_v + q_w p_v + p_v \times q_v\right], \qquad R(q) = (w^2 - v^\mathsf{T}v)I + 2vv^\mathsf{T} + 2w[v]_\times,$$
$$\dot{q} = \tfrac{1}{2}q \otimes [0, \omega_b], \qquad \dot{R} = R[\omega_b]_\times, \qquad R(p \otimes q) = R(p)R(q).$$

Kalman filter

$$P \leftarrow \Phi P \Phi^\mathsf{T} + Q_d, \qquad \Phi \approx I + F_c\Delta t, \qquad Q_d \approx G_c Q_c G_c^\mathsf{T}\Delta t,$$
$$r = z \boxminus h(\hat{x}), \quad H = \tfrac{\partial}{\partial \delta x}\left[h(\hat{x} \boxplus \delta x) \boxminus h(\hat{x})\right]_0, \quad S = HPH^\mathsf{T} + R_z, \quad K = PH^\mathsf{T}S^{-1},$$
$$P \leftarrow (I - KH)P(I - KH)^\mathsf{T} + KR_zK^\mathsf{T}, \qquad \hat{x} \leftarrow \hat{x} \boxplus Kr, \qquad P \leftarrow GPG^\mathsf{T},\ G_\theta = I - \tfrac{1}{2}[\delta\hat{\theta}]_\times.$$

Strapdown propagation with constant inputs ($\phi = \hat{\omega}\Delta t$)

$$R_{k+1} = R_k\,\mathrm{Exp}(\phi), \qquad v_{k+1} = v_k + g\Delta t + R_k\Delta t J_l(\phi)\hat{a},$$
$$p_{k+1} = p_k + v_k\Delta t + \tfrac{1}{2}g\Delta t^2 + R_k\Delta t^2 N(\phi)\hat{a},$$
$$N(\phi) = \tfrac{1}{2}I + \frac{\theta - \sin\theta}{\theta^3}[\phi]_\times + \frac{\theta^2/2 + \cos\theta - 1}{\theta^4}[\phi]_\times^2.$$

Noise

$$\sigma_d = \sigma/\sqrt{\Delta t} \ \text{(white, per sample)}, \qquad \sigma_{b,d} = \sigma_b\sqrt{\Delta t} \ \text{(random walk, per step)},$$

$$\text{Var}(m\text{-fold integral of white noise}) = \sigma^2 \frac{t^{2m-1}}{(2m-1)((m-1)!)^2},$$

$$1\,^\circ/\sqrt{\text{h}} = 2.909 \times 10^{-4}\ \text{rad/s}/\sqrt{\text{Hz}}.$$

Geometry

$$\bar{x}_2^{\mathsf{T}}[t]_\times R\bar{x}_1 = 0, \qquad \frac{\partial \pi}{\partial p_c} = \frac{1}{Z}\begin{bmatrix} 1 & 0 & -X/Z \\ 0 & 1 & -Y/Z \end{bmatrix},$$

$$\sigma_Z \approx \sqrt{2}\frac{Z^2}{b}\sigma_{uv}, \qquad N_{\text{RANSAC}} = \frac{\log(1-p)}{\log(1-w^s)}.$$

Appendix C

Statistical and Numerical Tables

The tables are generated by `scripts/make_tables.py` with SciPy, so they can be regenerated or extended. The project's `constexpr` χ^2 table (AGENTS.md F3) should be generated the same way, offline, rather than typed by hand.

Quantiles of the χ^2 distribution

$\chi^2_p(k)$ is the value below which a χ^2_k variable falls with probability p. Gates use $\chi^2_{0.95}$ or $\chi^2_{0.99}$; two-sided 95% intervals use $\chi^2_{0.025}$ and $\chi^2_{0.975}$. For other probability levels, rerun the script. An MSCKF feature with M observations has $k = 2M - 3$.

k	$\chi^2_{0.025}$	$\chi^2_{0.95}$	$\chi^2_{0.975}$	$\chi^2_{0.99}$	k	$\chi^2_{0.025}$	$\chi^2_{0.95}$	$\chi^2_{0.975}$	$\chi^2_{0.99}$
1	0.001	3.841	5.024	6.635	26	13.844	38.885	41.923	45.642
2	0.051	5.991	7.378	9.210	27	14.573	40.113	43.195	46.963
3	0.216	7.815	9.348	11.345	28	15.308	41.337	44.461	48.278
4	0.484	9.488	11.143	13.277	29	16.047	42.557	45.722	49.588
5	0.831	11.070	12.833	15.086	30	16.791	43.773	46.979	50.892
6	1.237	12.592	14.449	16.812	31	17.539	44.985	48.232	52.191
7	1.690	14.067	16.013	18.475	32	18.291	46.194	49.480	53.486
8	2.180	15.507	17.535	20.090	33	19.047	47.400	50.725	54.776
9	2.700	16.919	19.023	21.666	34	19.806	48.602	51.966	56.061
10	3.247	18.307	20.483	23.209	35	20.569	49.802	53.203	57.342
11	3.816	19.675	21.920	24.725	36	21.336	50.998	54.437	58.619
12	4.404	21.026	23.337	26.217	37	22.106	52.192	55.668	59.893
13	5.009	22.362	24.736	27.688	38	22.878	53.384	56.896	61.162
14	5.629	23.685	26.119	29.141	39	23.654	54.572	58.120	62.428
15	6.262	24.996	27.488	30.578	40	24.433	55.758	59.342	63.691
16	6.908	26.296	28.845	32.000	45	28.366	61.656	65.410	69.957
17	7.564	27.587	30.191	33.409	50	32.357	67.505	71.420	76.154
18	8.231	28.869	31.526	34.805	60	40.482	79.082	83.298	88.379
19	8.907	30.144	32.852	36.191	75	52.942	96.217	100.839	106.393
20	9.591	31.410	34.170	37.566	100	74.222	124.342	129.561	135.807
21	10.283	32.671	35.479	38.932	150	117.985	179.581	185.800	193.208
22	10.982	33.924	36.781	40.289	200	162.728	233.994	241.058	249.445
23	11.689	35.172	38.076	41.638	300	253.912	341.395	349.874	359.906
24	12.401	36.415	39.364	42.980	450	393.118	500.456	510.670	522.717
25	13.120	37.652	40.646	44.314	600	534.019	658.094	669.769	683.516

Average NEES acceptance intervals

Two-sided 95% interval $\left[\chi^2_{0.025}(Nn)/N,\ \chi^2_{0.975}(Nn)/N\right]$ for the average NEES of an n-dimensional error over N independent Monte Carlo runs (Chapter 12). The same intervals apply to the time-average of N independent NIS values of dimension n.

$n \backslash N$	1	10	25	50	100	200
1	[0.00, 5.02]	[0.32, 2.05]	[0.52, 1.63]	[0.65, 1.43]	[0.74, 1.30]	[0.81, 1.21]
2	[0.05, 7.38]	[0.96, 3.42]	[1.29, 2.86]	[1.48, 2.59]	[1.63, 2.41]	[1.73, 2.29]
3	[0.22, 9.35]	[1.68, 4.70]	[2.12, 4.03]	[2.36, 3.72]	[2.54, 3.50]	[2.67, 3.35]
6	[1.24, 14.45]	[4.05, 8.33]	[4.72, 7.43]	[5.08, 7.00]	[5.34, 6.70]	[5.53, 6.49]
9	[2.70, 19.02]	[6.56, 11.81]	[7.41, 10.74]	[7.86, 10.21]	[8.19, 9.85]	[8.42, 9.60]
15	[6.26, 27.49]	[11.80, 18.58]	[12.93, 17.22]	[13.52, 16.56]	[13.95, 16.09]	[14.25, 15.77]

Monte Carlo variance ratio

If M runs produce a sample variance s^2 of an error whose true variance is σ^2, then $s^2/\sigma^2 \sim \chi^2_{M-1}/(M-1)$. The 95% intervals below are the acceptance bounds for a covariance-propagation check (Milestone 5).

runs M	lower bound	upper bound
50	0.644	1.433
100	0.741	1.297
200	0.813	1.206
500	0.880	1.128
1000	0.914	1.090
2000	0.939	1.063

RANSAC iterations

Number of iterations $N = \lceil \log(1-p) / \log(1-w^s) \rceil$ for confidence $p = 0.99$, minimal sample size s and inlier ratio w.

$s \backslash w$	0.9	0.8	0.7	0.6	0.5	0.4	0.3
1	2	3	4	6	7	10	13
2	3	5	7	11	17	27	49
3	4	7	11	19	35	70	169
5	6	12	26	57	146	448	1893
7	8	20	54	163	588	2809	21055
8	9	26	78	272	1177	7025	70188

Appendix D

The EuRoC MAV Dataset

The EuRoC MAV dataset [7] was recorded with an AscTec Firefly hexacopter carrying a VI-Sensor: two global-shutter MT9V034 cameras (752×480, 20 Hz) and an ADIS16448 MEMS IMU (200 Hz), hardware time-synchronized and calibrated with Kalibr. The values below are those of the published calibration files; check them against your copy.

Directory layout

The project keeps sequences under `data/<sequence>/mav0/` (git-ignored). Each sequence contains:

Path	Content
`imu0/data.csv`	`timestamp [ns], w_x, w_y, w_z [rad/s], a_x, a_y, a_z [m/s²]`: ω_m, a_m in B
`imu0/sensor.yaml`	IMU noise densities (continuous time), rate, `T_BS` (identity for the IMU)
`cam0/data.csv`	`timestamp [ns], filename`
`cam0/data/`	PNG images named by timestamp
`cam0/sensor.yaml`	`T_BS` (row-major 4×4), `intrinsics`, `distortion_coefficients`, resolution, rate
`cam1/`	right camera (unused; stereo is a stretch goal)
`state_groundtruth_estimate0/data.csv`	`timestamp, p_RS_R (3), q_RS (w,x,y,z), v_RS_R (3), b_w_RS_S (3), b_a_RS_S (3)`

Pitfall: ground-truth column order

In the ground-truth file the quaternion is stored w first, and the *gyroscope* bias precedes the *accelerometer* bias. The file starts later than the IMU file, and its velocities and biases come from the dataset authors' own estimator that fused the tracking-system data with the IMU, so they are reference values, not perfect truth. The project maps the columns to $p_{wb}, q_{wb}, v_{wb}, b_g, b_a$ (AGENTS.md C7).

IMU parameters (`imu0/sensor.yaml`)

Key	Value	Units
`gyroscope_noise_density`	1.6968×10^{-4}	$\mathrm{rad/s}/\sqrt{\mathrm{Hz}}$
`gyroscope_random_walk`	1.9393×10^{-5}	$\mathrm{rad/s^2}/\sqrt{\mathrm{Hz}}$
`accelerometer_noise_density`	2.0×10^{-3}	$\mathrm{m/s^2}/\sqrt{\mathrm{Hz}}$
`accelerometer_random_walk`	3.0×10^{-3}	$\mathrm{m/s^3}/\sqrt{\mathrm{Hz}}$
`rate_hz`	200	Hz

Camera parameters (`cam0/sensor.yaml`)

`intrinsics` $[f_u, f_v, c_u, c_v]$	$[458.654,\ 457.296,\ 367.215,\ 248.375]$
`distortion_model`	`radial-tangential`
`distortion_coefficients` $[k_1, k_2, p_1, p_2]$	$[-0.28340811,\ 0.07395907,\ 0.00019359,\ 1.76187114 \times 10^{-5}]$
`resolution`	752×480
`rate_hz`	20

The extrinsic calibration `T_BS` of `cam0` is the project's T_{bc}:

$$T_{bc} \approx \begin{bmatrix} 0.0148655 & -0.9998809 & 0.0041403 & -0.0216401 \\ 0.9995572 & 0.0149672 & 0.0257155 & -0.0646770 \\ -0.0257744 & 0.0037562 & 0.9996607 & 0.0098107 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Its third column shows that the optical axis z_c is almost exactly the body z-axis, and its first two columns that the camera x-axis is close to body y and the camera y-axis close to body $-x$. The camera center sits about 7 cm from the IMU. At rest, the EuRoC accelerometer reads close to $+g$ on its x-axis, so the body x-axis points roughly up. Verify both facts on your data: they are exactly the kind of sanity check that catches a transposed extrinsic.

Sequences

Sequence	Environment	Difficulty	Ground truth
MH_01_easy, MH_02_easy	machine hall	easy	laser tracker (position)
MH_03_medium	machine hall	medium	laser tracker (position)
MH_04_difficult, MH_05_difficult	machine hall	difficult (fast, dark)	laser tracker (position)
V1_01_easy, V2_01_easy	Vicon room	easy	motion capture (pose)
V1_02_medium, V2_02_medium	Vicon room	medium	motion capture (pose)
V1_03_difficult, V2_03_difficult	Vicon room	difficult (fast, blur)	motion capture (pose)

MH_01_easy is the project's development sequence. The difficult sequences contain fast rotations, motion blur and poorly lit regions; they are where robust front ends and consistent estimators make the difference.

Further Reading

Estimation theory. Bar-Shalom, Li and Kirubarajan [2] is the standard reference for Kalman filtering, consistency testing (NEES, NIS) and tracking practice. Maybeck [36] remains the clearest treatment of continuous-time stochastic models and their discretization, and Simon [43] is a modern, readable textbook.

Rotations and Lie theory. Solà's quaternion notes [44] derive the error-state filter with exactly the conventions of this project and are the best single companion to Chapters 7 and 10. The micro Lie theory paper [45] is a short, practical introduction to manifolds, Jacobians and $\boxplus/\boxminus$. Barfoot [3] develops estimation on matrix Lie groups in depth. Sommer et al. [46] explain the Hamilton/JPL confusion once and for all.

Inertial navigation. Groves [19] covers inertial sensors, error models and strapdown algorithms comprehensively. El-Sheimy et al. [13] is a good introduction to the Allan variance.

Computer vision. Hartley and Zisserman [22] is the reference for multi-view geometry; Ma et al. [35] gives a more geometric, calibrated-camera treatment closer to how VIO uses vision. Baker and Matthews [1] unify the many variants of Lucas–Kanade.

Visual-inertial odometry. Read the original MSCKF paper [37] first, then MSCKF 2.0 [31] for consistency and calibration. Huang's review [27] and Scaramuzza and Zhang [41] survey the field. The OpenVINS documentation and paper [17] and the MSCKF-VIO code [48] are excellent engineering references. For the optimization-based alternative, start with preintegration [15], VINS-Mono [39] and the factor-graph tutorial of Dellaert and Kaess [11].

Numerical linear algebra. Golub and Van Loan [18] for factorizations, Higham [24] for floating-point error analysis.

References

[1] S. Baker and I. Matthews. Lucas–Kanade 20 years on: A unifying framework. *International Journal of Computer Vision*, 56(3):221–255, 2004.

[2] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan. *Estimation with Applications to Tracking and Navigation*. Wiley, 2001.

[3] T. D. Barfoot. *State Estimation for Robotics*. Cambridge University Press, 2017 (2nd ed. 2024).

[4] A. Barrau and S. Bonnabel. The invariant extended Kalman filter as a stable observer. *IEEE Transactions on Automatic Control*, 62(4):1797–1812, 2017.

[5] M. Bloesch, S. Omari, M. Hutter, and R. Siegwart. Robust visual inertial odometry using a direct EKF-based approach. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2015.

[6] J.-Y. Bouguet. Pyramidal implementation of the affine Lucas Kanade feature tracker: description of the algorithm. Technical report, Intel Corporation, Microprocessor Research Labs, 2001.

[7] M. Burri, J. Nikolic, P. Gohl, T. Schneider, J. Rehder, S. Omari, M. W. Achtelik, and R. Siegwart. The EuRoC micro aerial vehicle datasets. *The International Journal of Robotics Research*, 35(10):1157–1163, 2016.

[8] C. Campos, R. Elvira, J. J. Gómez Rodríguez, J. M. M. Montiel, and J. D. Tardós. ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multimap SLAM. *IEEE Transactions on Robotics*, 37(6):1874–1890, 2021.

[9] J. Civera, A. J. Davison, and J. M. M. Montiel. Inverse depth parametrization for monocular SLAM. *IEEE Transactions on Robotics*, 24(5):932–945, 2008.

[10] A. J. Davison, I. D. Reid, N. D. Molton, and O. Stasse. MonoSLAM: Real-time single camera SLAM. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 29(6):1052–1067, 2007.

[11] F. Dellaert and M. Kaess. Factor graphs for robot perception. *Foundations and Trends in Robotics*, 6(1–2):1–139, 2017.

[12] J. Delmerico and D. Scaramuzza. A benchmark comparison of monocular visual-inertial odometry algorithms for flying robots. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.

[13] N. El-Sheimy, H. Hou, and X. Niu. Analysis and modeling of inertial sensors using Allan variance. *IEEE Transactions on Instrumentation and Measurement*, 57(1):140–149, 2008.

[14] M. A. Fischler and R. C. Bolles. Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, 1981.

[15] C. Forster, L. Carlone, F. Dellaert, and D. Scaramuzza. On-manifold preintegration for real-time visual-inertial odometry. *IEEE Transactions on Robotics*, 33(1):1–21, 2017.

[16] P. Furgale, J. Rehder, and R. Siegwart. Unified temporal and spatial calibration for multi-sensor systems. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2013.

[17] P. Geneva, K. Eckenhoff, W. Lee, Y. Yang, and G. Huang. OpenVINS: A research platform for visual-inertial estimation. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2020.

[18] G. H. Golub and C. F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4th edition, 2013.

[19] P. D. Groves. *Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems*. Artech House, 2nd edition, 2013.

[20] M. Grupp. evo: Python package for the evaluation of odometry and SLAM. https://github.com/MichaelGrupp/evo, 2017.

[21] C. Harris and M. Stephens. A combined corner and edge detector. In *Proceedings of the Alvey Vision Conference*, 1988.

[22] R. Hartley and A. Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2nd edition, 2004.

[23] J. A. Hesch, D. G. Kottas, S. L. Bowman, and S. I. Roumeliotis. Consistency analysis and improvement of vision-aided inertial navigation. *IEEE Transactions on Robotics*, 30(1):158–176, 2014.

[24] N. J. Higham. *Accuracy and Stability of Numerical Algorithms*. SIAM, 2nd edition, 2002.

[25] G. P. Huang, A. I. Mourikis, and S. I. Roumeliotis. A first-estimates Jacobian EKF for improving SLAM consistency. In *Experimental Robotics (ISER 2008)*, Springer Tracts in Advanced Robotics, vol. 54, 2009.

[26] G. P. Huang, A. I. Mourikis, and S. I. Roumeliotis. Observability-based rules for designing consistent EKF SLAM estimators. *The International Journal of Robotics Research*, 29(5):502–528, 2010.

[27] G. Huang. Visual-inertial navigation: A concise review. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2019.

[28] IEEE Std 952-1997. *IEEE Standard Specification Format Guide and Test Procedure for Single-Axis Interferometric Fiber Optic Gyros* (includes the Allan variance method). IEEE, 1997.

[29] R. E. Kalman. A new approach to linear filtering and prediction problems. *Transactions of the ASME, Journal of Basic Engineering*, 82(1):35–45, 1960.

[30] S. Leutenegger, S. Lynen, M. Bosse, R. Siegwart, and P. Furgale. Keyframe-based visual-inertial odometry using nonlinear optimization. *The International Journal of Robotics Research*, 34(3):314–334, 2015.

[31] M. Li and A. I. Mourikis. High-precision, consistent EKF-based visual-inertial odometry. *The International Journal of Robotics Research*, 32(6):690–711, 2013.

[32] M. Li and A. I. Mourikis. Online temporal calibration for camera-IMU systems: Theory and algorithms. *The International Journal of Robotics Research*, 33(7):947–964, 2014.

[33] B. D. Lucas and T. Kanade. An iterative image registration technique with an application to stereo vision. In *International Joint Conference on Artificial Intelligence (IJCAI)*, 1981.

[34] T. Lupton and S. Sukkarieh. Visual-inertial-aided navigation for high-dynamic motion in built environments without initial conditions. *IEEE Transactions on Robotics*, 28(1):61–76, 2012.

[35] Y. Ma, S. Soatto, J. Košecká, and S. S. Sastry. *An Invitation to 3-D Vision*. Springer, 2004.

[36] P. S. Maybeck. *Stochastic Models, Estimation, and Control*, volume 1. Academic Press, 1979.

[37] A. I. Mourikis and S. I. Roumeliotis. A multi-state constraint Kalman filter for vision-aided inertial navigation. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 3565–3572, 2007.

[38] D. Nistér. An efficient solution to the five-point relative pose problem. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26(6):756–770, 2004.

[39] T. Qin, P. Li, and S. Shen. VINS-Mono: A robust and versatile monocular visual-inertial state estimator. *IEEE Transactions on Robotics*, 34(4):1004–1020, 2018.

[40] S. I. Roumeliotis and J. W. Burdick. Stochastic cloning: A generalized framework for processing relative state measurements. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2002.

[41] D. Scaramuzza and Z. Zhang. Visual-inertial odometry of aerial robots. In *Encyclopedia of Robotics*. Springer, 2019. arXiv:1906.03289.

[42] J. Shi and C. Tomasi. Good features to track. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 1994.

[43] D. Simon. *Optimal State Estimation: Kalman, H_∞, and Nonlinear Approaches*. Wiley, 2006.

[44] J. Solà. Quaternion kinematics for the error-state Kalman filter. arXiv:1711.02508, 2017.

[45] J. Solà, J. Deray, and D. Atchuthan. A micro Lie theory for state estimation in robotics. arXiv:1812.01537, 2018.

[46] H. Sommer, I. Gilitschenski, M. Bloesch, S. Weiss, R. Siegwart, and J. Nieto. Why and how to avoid the flipped quaternion multiplication. *Aerospace*, 5(3):72, 2018.

[47] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers. A benchmark for the evaluation of RGB-D SLAM systems. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.

[48] K. Sun, K. Mohta, B. Pfrommer, M. Watterson, S. Liu, Y. Mulgaonkar, C. J. Taylor, and V. Kumar. Robust stereo visual inertial odometry for fast autonomous flight. *IEEE Robotics and Automation Letters*, 3(2):965–972, 2018.

[49] N. Trawny and S. I. Roumeliotis. Indirect Kalman filter for 3D attitude estimation. Technical Report 2005-002, University of Minnesota, 2005.

[50] S. Umeyama. Least-squares estimation of transformation parameters between two point patterns. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 13(4):376–380, 1991.

[51] C. F. Van Loan. Computing integrals involving the matrix exponential. *IEEE Transactions on Automatic Control*, 23(3):395–404, 1978.

[52] Y. Yang, P. Geneva, K. Eckenhoff, and G. Huang. Degenerate motion analysis for aided INS with online spatial and temporal sensor calibration. *IEEE Robotics and Automation Letters*, 4(2):2070–2077, 2019.

[53] Z. Zhang and D. Scaramuzza. A tutorial on quantitative trajectory evaluation for visual(-inertial) odometry. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.